

NUMERICAL METHODS FOR OPTIMAL TRANSPORT PROBLEMS

A Dissertation
Presented to
The Academic Faculty

By

Daniyar Omarov

In Partial Fulfillment
of the Requirements for the Degree
Doctor of Philosophy in the
School of Mathematics
College of Sciences

Georgia Institute of Technology

May 2024

© Daniyar Omarov 2024

NUMERICAL METHODS FOR OPTIMAL TRANSPORT PROBLEMS

Thesis committee:

Dr. Edmond Chow
School of Computational Science
and Engineering
Georgia Institute of Technology

Dr. Wenjing Liao
School of Mathematics
Georgia Institute of Technology

Dr. Luca Dieci
School of Mathematics
Georgia Institute of Technology

Dr. Haomin Zhou
School of Mathematics
Georgia Institute of Technology

Dr. Sung Ha Kang
School of Mathematics
Georgia Institute of Technology

Date approved: April 12, 2024

To my grandfather.

ACKNOWLEDGMENTS

First and foremost, I would like to express my gratitude to my advisor, Luca Dieci, for his unwavering support and invaluable guidance throughout my PhD journey. Our weekly meetings, spanning hundreds of hours over the past six years, have been a highlight of my doctoral experience. Under his mentorship, I have honed my research skills, cultivated a meticulous attention to details, and learned to navigate challenges with resilience. I am truly fortunate to have had such a dedicated advisor and mentor by my side.

I extend my heartfelt thanks to Sung Ha Kang and Haomin Zhou for their assistance and insightful discussions during my PhD studies. Their perspectives have helped me see the bigger picture and encouraged me to remain curious. Additionally, I am grateful to Edmond Chow and Wenjing Liao for generously sharing their time and expertise. I am appreciative of the support and prompt assistance provided by all the faculty and staff members of the School of Mathematics over the past six years.

I am deeply grateful to my family and friends for their unwavering moral support. My journey into the world of mathematics was ignited by my grandfather, whose unyielding belief in me exceeded even my own self-confidence. I owe a debt of gratitude to my mother for devoting countless hours to studying with me and fostering my passion for science. Finally, I wish to express my heartfelt appreciation to my brother, who has consistently been a rock of support whenever I needed it.

TABLE OF CONTENTS

Acknowledgments	iv
List of Tables	viii
List of Figures	x
Chapter 1: Introduction	1
1.1 Background	1
1.2 Applications	4
Chapter 2: Discrete OT Problems	6
2.1 Linear Programming Formulation and Sinkhorn Method	6
2.2 Linear Assignment Formulation and Auction Method	8
2.3 Two Points with Obstacle	12
2.3.1 Numerical examples	19
Chapter 3: Continuous OT problems	23
3.1 Formulations	23
3.2 Algorithms	24
3.2.1 Basic Algorithms Description	24
3.2.2 Numerical Implementation	28

3.2.3	Hybrid technique: From $v(0, x)$ to the full geodesic evolution	36
3.3	Separable Class of OT Problems	37
3.3.1	Separation Process	37
3.3.2	Separable Monge-Ampère	38
3.3.3	Separable Densities with Benamou-Brenier Formulation	43
3.3.4	Numerical Solution of 1-d Monge Ampère	47
3.3.5	Optimal map on 1-d torus	48
3.4	Computational Examples and Numerical Results	49
3.4.1	Problems and Results in 1D	49
3.4.2	Problems and Results for Separable 2D Examples	53
3.4.3	Problems and Results for Non-Separable 2D Examples	56
Chapter 4:	Semi-discrete OT problems	61
4.1	Formulation	61
4.2	Theoretical Results	64
4.2.1	Star-shapedness Results	66
4.2.2	Smoothness Results	72
4.2.3	Minimization Problem Results	86
4.3	Laguerre cell computation in 2D	89
4.3.1	Boundary tracing technique	92
4.3.2	2-d integral computation	95
4.4	Newton's Method and Conditioning of the problem	97
4.4.1	The Hessian matrix computation	98

4.4.2	Taking a Newton step: solving a system with the Hessian matrix . .	100
4.4.3	Getting an initial guess	103
4.5	Computational examples	108
4.6	Semi-discrete OT problems in $\mathbb{R}^n$ ($n \geq 3$)	120
4.6.1	Computational examples in 3-d	121
4.7	Application on the congressional districts of Georgia	124
4.7.1	Problem statement	124
4.7.2	Algorithm and results	124
Chapter 5: Conclusion		132
Appendices		134
Appendix A: Algorithms Pseudocodes		135
References		143

LIST OF TABLES

2.1	Total cost of mapping 100 points	19
3.1	Errors in Optimal Map for Problem 3.4.1, obtained by the Regularized Benamou-Brenier method with different β values and varying levels of discretization dt and dx	27
3.2	Total number of Newton's iterations, number of homotopy steps if applicable, and execution times, when the nonlinear Monge-Ampère discretization is solved with different initial guesses for Newton's method.	32
3.3	Numerical Results for Example 3.4.1	51
3.4	Numerical Results for Example 3.4.2	51
3.5	Numerical Results for Example 3.4.3	52
3.6	Numerical Results for Example 3.4.4	53
3.7	Numerical Results for Example 3.4.5	54
3.8	Numerical Results for Example 3.4.6	55
3.9	Numerical Results for Example 3.4.7	55
3.10	Numerical Results for Example 3.4.8	56
3.11	Numerical Results for Example 3.4.9	58
3.12	Numerical Results for Example 3.4.10	59
4.1	Iterations refer to Newton's steps, Difference is the ∞ -norm of the difference between analytic and divided differences Hessians.	99

4.2	Different strategies to give an initial guess for Newton's method	106
4.3	Different Methods	110
4.4	Example with Different Source Densities	111
4.5	Example with Different p -Norm Cost Functions	112
4.6	Approaching the ∞ -norm. Cost is $\ x - y\ _{2^k}$	113
4.7	Approaching the 1-norm. Cost is $\ x - y\ _{1+2^{-k}}$	113
4.8	Impact of κ on Newton iterations and time	115
4.9	Interplay between Source and Target Densities with Location of Points . . .	116
4.10	Different domains and uniform source density	117
4.11	Impact of rescaling	118
4.12	Problem with $\rho_1(x) = 1$	122
4.13	Problem with $\rho_2(x) = 8x_1x_2x_3$	122
4.14	Problem with $\rho_3(x) = \frac{1}{3}(x_1 + 2x_2 + 3x_3)$	123
4.15	Problem with $\rho_4(x) = ce^{-(x_1-0.25)^2-(x_2-0.5)^2-(x_3-0.75)^2}$	123
4.16	Population after Step 1	127
4.17	Population after Step 2	127
4.18	Population after Step 3	130

LIST OF FIGURES

2.1	Generic path when $k \leq 1$	13
2.2	Generic path when $1 < k < \frac{\pi}{2}$	15
2.3	Generic path when $k \geq \frac{\pi}{2}$	17
2.4	Assignment pairs and optimal paths for Example 2.3.10	21
2.5	Assignment pairs and optimal paths for Example 2.3.11	22
3.1	Sup-norm of the difference on the grid between $\rho(t, x)$ obtained by the techniques based on (3.4) (with $\beta = 10^{-5}$) and (3.5) on Problem 3.4.2. . . .	36
3.2	Checking separability of $\rho(t, x)$. Problem 3.4.8, multiple shooting method, $dx = \frac{1}{32}$	46
3.3	Evolution of probability density in Example 3.4.2 with $dx = \frac{1}{64}$, solved with Regularized Benamou-Brenier method	52
3.4	Evolution of probability density in Example 3.4.3 with $dx = \frac{1}{128}$	53
3.5	Evolution of probability density for Example 3.4.8, solved with the multiple shooting method, $dx = \frac{1}{64}$. Here, we first obtained the initial velocity $v(0, x)$ solving by multiple shooting two 1-d problems, and then integrated in time for ρ and v the system (3.5) using the time integrator from [29]. . . .	57
3.6	Evolution of probability density in Example 3.4.9 with $dx = \frac{1}{32}$, solved with multiple shooting method.	59
3.7	Evolution of probability density in Example 3.4.10 with $dx = \frac{1}{32}$, solved with multiple shooting method.	60

4.1	For the same target points, w , and cost: (A) Laguerre cells in $\mathbb{R}^2$ and (B) Restricted to the unit square.	67
4.2	Counterexamples to uniqueness $w = 0$ The shaded regions are in both $A(y_1)$ and $A(y_2)$	85
4.5	Different p -norm costs	114
4.6	p -norm costs for growing p	114
4.7	p -norm costs for p decreasing toward 1	115
4.8	Impact of small κ on the tessellation	116
4.9	Interplay between Source and Target Densities with Location of Points . . .	117
4.10	Examples with Different domains and uniform source density	118
4.11	Effect of Rescaling	119
4.12	Problem with uniform source density	123
4.13	Locations of the target points	126
4.14	Laguerre cells and districts after Step 1	128
4.15	Districts after Step 2	129
4.16	Districts after Step 3 for Example (C1)	129
4.17	Districts after Step 3 for Example (C2)	130
4.18	Districts based on the 2020 presidential election results	131

CHAPTER 1

INTRODUCTION

1.1 Background

Optimal transport theory is a fascinating mathematical framework that deals with the optimal way to transport goods from one location to another. Currently it has applications in a wide range of fields, including economics, geography, and even in the development of efficient transportation systems. At its core, the optimal transport (OT) problem seeks to minimize the cost of moving resources while satisfying certain constraints, making it a powerful tool for understanding and solving real-world problems related to resource allocation and logistics. The first OT problem was formulated by Gaspard Monge in 1781, [67]. In modern mathematical language one can formulate the Monge problem as following:

Problem 1.1.1 (Monge). *Given two compactly supported probability measures μ and ν on $\mathbb{R}^n$, $n > 1$, absolutely continuous with respect to Lebesgue measure, and with respective supports X and Y , and given a cost function $c : X \times Y \rightarrow \mathbb{R}^+$. The problem is to find an optimal invertible map $T(x)$ such that:*

$$\inf_{T: T_{\#}\mu=\nu} \int_X c(x, T(x)) d\mu(x), \quad (1.1)$$

where $T_{\#}$ is the push-forward of μ through T , that is $(T_{\#}\mu)(A) = \mu(T^{-1}(A))$, for any Borel subset $A \subseteq Y$.

Remark 1.1.2. *If μ and ν are supported on $X, Y \subset \mathbb{R}^n$ with $T^{-1}(Y) = X$, then using the change of variables one can transform the constraint of the Monge problem into a PDE. Let $\rho_0(x)$ and $\rho_1(y)$ be probability densities corresponding to measures μ and ν , respectively.*

Then, the constraint reads as follows:

$$\int_{y \in Y} f(y) \rho_1(y) dy = \int_{x \in X} f(T(x)) \rho_0(x) dx,$$

where $f(x)$ is any measurable function. Using the relation $y = T(x)$ one gets that:

$$\begin{aligned} \int_{T(x) \in Y} f(T(x)) \rho_1(T(x)) dT(x) &= \int_{x \in X} f(T(x)) \rho_0(x) dx, \\ \int_{x \in T^{-1}(Y)} f(T(x)) \rho_1(T(x)) \det(DT(x)) dx &= \int_{x \in X} f(T(x)) \rho_0(x) dx, \\ \int_{x \in X} f(T(x)) \left[\rho_1(T(x)) \det(DT(x)) - \rho_0(x) \right] dx &= 0, \end{aligned}$$

where $DT(x)$ is a Jacobian matrix. Since the above equation must hold for any measurable function we get the following PDE:

$$\rho_1(T(x)) \det(DT(x)) = \rho_0(x). \quad (1.2)$$

Unfortunately, the existence of the solution to the Monge problem is not guaranteed. For example, let $\rho_0(x) = \delta_0(x)$ and $\rho_1(y) = \frac{1}{2}[\delta_{-1}(y) + \delta_1(y)]$. Then, there is no map $T(x)$ which will satisfy the constraint $T_{\#}\mu = \nu$. As a result, no significant progress was made in the analysis of the OT problem for over 150 years.

In 1940s a major breakthrough occurred with the work of Kantorovich, [50]. The key was to relax the problem and reformulate it in terms of a linear programming problem. The new problem (known as Monge-Kantorovich, MK) aimed to determine an *optimal transport plan* instead of an optimal map $T(x)$.

Definition 1.1.3. Given two compactly supported probability measures μ and ν on $\mathbb{R}^n$, $n > 1$, absolutely continuous with respect to Lebesgue measure, and with respective supports

X and Y . The set of transport plans between the measures μ and ν is defined as follows:

$$\Pi(\mu, \nu) = \{\pi \in \mathcal{P}(X \times Y) \mid \pi(A \times Y) = \mu(A), \pi(X \times B) = \nu(B), \forall A \subset X, \forall B \subset Y\},$$

where $\mathcal{P}(X \times Y)$ is the set of joint probability measures on the product space.

Problem 1.1.4 (Monge-Kantorovich). *Given a cost function $c(x, y) : X \times Y \rightarrow \mathbb{R}^+$. The problem is to find an optimal transport plan $\pi \in \Pi(\mu, \nu)$, which realizes the inf of the functional $J(\pi)$ in (1.3) below:*

$$J := \int_{X \times Y} c(x, y) d\pi. \quad (1.3)$$

The key difference between the Monge problem and the Monge-Kantorovich (MK) problem is that the MK problem allows mass splitting. As a result, an existence result was derived under mild restrictions on the cost function.

Theorem 1.1.5. [80, Theorem 1.5]. *Let $c(x, y)$ be lower semi-continuous and bounded from below. Then Problem 1.1.4 admits a solution.*

In addition, one can derive the dual formulation of the MK problem. Given a cost function $c(x, y) : X \times Y \rightarrow \mathbb{R}^+$, consider the set of functions $D_c(\mu, \nu)$ given by:

$$D_c(\mu, \nu) = \{\phi \in L^1(\mu), \psi \in L^1(\nu) \mid \phi(x) + \psi(y) \leq c(x, y), \forall x \in X, \forall y \in Y\},$$

where L^1 is the space of absolutely integrable functions.

Problem 1.1.6 (Dual Monge-Kantorovich). *The problem is to find an optimal pair $(\phi, \psi) \in D_c(\mu, \nu)$, which realizes the sup of the functional $K(\phi, \psi)$ in (1.4):*

$$K(\phi, \psi) := \int_X \phi(x) d\mu + \int_Y \psi(y) d\nu. \quad (1.4)$$

In certain cases, such as when $c(x, y) = \|x - y\|_2^2$, Brenier proved in [20] that the optimal plan does not permit mass splitting. In these instances, the Monge and MK formulations are equivalent, the map $T(x)$ is the gradient of a smooth and convex potential $u(x)$, and (1.2) reduces to the Monge-Ampere (MA) partial differential equation:

$$\rho_1(\nabla u(x)) \det(D^2 u(x)) = \rho_0(x). \quad (1.5)$$

Furthermore, the uniqueness of the optimal map T has been established in the case of the quadratic cost, $c(x, y) = \|x - y\|_2^2$, as shown in [80, Theorem 1.17]. Consequently, in this case solving the original constrained minimization problem 1.1.1 is equivalent to solving the nonlinear elliptic PDE (1.5). Later Ma-Trudinger-Wang in [62] determined the conditions, now known as Ma-Trudinger-Wang (MTW) conditions, under which the Monge PDE in (1.2) is well-posed in the sense of Hadamard. The main two classes of functions that satisfy MTW conditions are strictly convex functions (e.g., $c(x, y) = \|x - y\|_2^2$) and c -convex functions (see Section 4.2. in [78]).

1.2 Applications

Over its nearly 250-year history, OT has found applications in a wide range of fields. In the field of economics, the OT theory has been used to study resource allocation, market equilibrium, and cost minimization in the supply chain management. These applications have been discussed in depth by Galichon in [42].

In climate science, OT has been applied to analyze semigeostrophic equations [6] and to understand the transport of the air pollution in the atmosphere [1]. Furthermore, in neuroscience, OT theory has been instrumental in modeling the neural activity and information transfer within the brain [5], and training brain-computer interfaces [73].

OT theory has made significant contributions to image processing [72] and computer vision [14], as well as for improvement of color transfer problem [37] and texture synthe-

sis [85] problems. Moreover, in machine learning [86], OT has been utilized for domain adaptation [27], clustering [55], and generative modeling tasks [77].

Research related to the optimal transportation has also had a positive impact on various other subjects: signal processing [53], geophysics [64], fluid mechanics [7, 21], PDEs [48, 71], optimization [16, 15], geometric measure theory [36, 68]. These diverse applications highlight the versatility and broad impact of OT theory in advancing our understanding of complex systems.

Finally, it is convenient to classify three distinct types of the MK problem (1.3), which are based on the type of marginal densities. The first case is when the measures μ and ν are discrete (discussed in Chapter 2). This problem is known as a the *discrete* OT problem and can be solved using linear assignment and/or linear programming techniques. The second variation is when both densities are continuous (discussed in Chapter 3). It is known as the *continuous* OT problem, in which case one typically uses the 2-norm squared cost function, $c(x, y) = ||x - y||_2^2$, and wants to determine the optimal map $T(x) = \nabla\phi(x)$ from (1.5). Finally, the last situation is when the source density is a continuous measure and the the target density is a discrete measure (discussed in Chapter 4). This setting is called the *semi-discrete* OT problem and the objective is to find an optimal partition of the source domain.

CHAPTER 2

DISCRETE OT PROBLEMS

2.1 Linear Programming Formulation and Sinkhorn Method

Probably the most studied type of OT problems in the literature is the *discrete* case. Recall that the MK problem seeks the optimal plan $\pi \in \mathcal{P}(X \times Y)$, with marginals μ and ν , which realizes the inf of the functional $J(\pi)$ in (1.3). Now, suppose μ and ν are discrete probability measures on $X = \{x_i\}_{i=1}^{k_0}$ and $Y = \{y_j\}_{j=1}^{k_1}$:

$$\mu := \sum_{i=1}^{k_0} a_i \delta(x_i), \quad \nu := \sum_{j=1}^{k_1} b_j \delta(y_j).$$

Then the discrete MK problem can be formulated as following.

Problem 2.1.1. *Given a cost function $c(x, y)$, find an optimal transport plan $\pi \in \mathbb{R}_{\geq 0}^{k_0 \times k_1}$ that realizes the inf of the functional $J(\pi)$ in (2.1):*

$$\begin{aligned} J(\pi) &:= \sum_i^{k_0} \sum_j^{k_1} c(x_i, y_j) \pi_{ij}, \\ \text{subject to } \sum_j^{k_1} \pi_{ij} &= a_i, \quad i = 1, \dots, k_0, \\ \text{and } \sum_i^{k_0} \pi_{ij} &= b_j, \quad j = 1, \dots, k_1, \end{aligned} \tag{2.1}$$

where $x_i \in X$ and $a_i \geq 0, \forall i$, $y_j \in Y$ and $b_j \geq 0, \forall j$, and $\sum_i^{k_0} a_i = \sum_j^{k_1} b_j = 1$.

The discrete MK problem (2.1) is widely used in image processing tasks such as color transfer between images [76], image registration [46], and texture mixing [38].

The problem (2.1) is a linear programming (LP) problem with linear constraints and, in general, we have that the bipartite graph describing the plan is complete, that is the matrix π

has no 0-entries. But standard LP techniques, such as the simplex method or interior-point algorithms (e.g., see [26]), become computationally inefficient as the number of points increase. The state-of-the-art method is to use an entropic regularization term to smooth out the problem, as it was done by Cuturi in [30]. The resulting algorithm is known as the *Sinkhorn* method.

The first step of the Sinkhorn technique is to add an entropic regularization term to the objective function in (2.1), for the purpose of obtaining a strict convex function of π_{ij} , J_ε :

$$\begin{aligned} J_\varepsilon &:= \sum_i^{k_0} \sum_j^{k_1} c(x_i, y_j) \pi_{ij} + \varepsilon \sum_i^{k_0} \sum_j^{k_1} \pi_{ij} \log \pi_{ij}, \\ &\text{subject to } \sum_j^{k_1} \pi_{ij} = a_i, \quad i = 1, \dots, k_0, \\ &\text{and } \sum_i^{k_0} \pi_{ij} = b_j, \quad j = 1, \dots, k_1, \end{aligned} \tag{2.2}$$

where $\varepsilon > 0$ is a fixed constant. Using Lagrange multipliers $\alpha \in \mathbb{R}^{k_0}$, $\beta \in \mathbb{R}^{k_1}$, the Lagrangian of the above problem can be formulated as:

$$L(\pi, \alpha, \beta) = \sum_i^{k_0} \sum_j^{k_1} c(x_i, y_j) \pi_{ij} + \varepsilon \sum_i^{k_0} \sum_j^{k_1} \pi_{ij} \log \pi_{ij} + \sum_i^{k_0} \alpha_i \left(\sum_j \pi_{ij} - a_i \right) + \sum_j^{k_1} \beta_j \left(\sum_i \pi_{ij} - b_j \right).$$

By taking partial derivative of $L(\pi, \alpha, \beta)$ with respect to π_{ij} one gets:

$$\begin{aligned} \frac{\partial L(\pi, \alpha, \beta)}{\partial \pi_{ij}} &= c(x_i, y_j) + \varepsilon (\log \pi_{ij} + 1) + \alpha_i + \beta_j = 0 \implies \pi_{ij} = e^{-\frac{c(x_i, y_j) + \alpha_i + \beta_j + \varepsilon}{\varepsilon}}, \\ \text{or } \pi_{ij} &= \left(e^{-\frac{1}{2} - \frac{\alpha_i}{\varepsilon}} \right) \left(e^{-\frac{c(x_i, y_j)}{\varepsilon}} \right) \left(e^{-\frac{1}{2} - \frac{\beta_j}{\varepsilon}} \right) \implies \pi = \text{diag}(u) K \text{diag}(v), \end{aligned}$$

where $u \in \mathbb{R}^{k_0}$, $v \in \mathbb{R}^{k_1}$, and $K \in \mathbb{R}^{k_0 \times k_1}$ such that

$$u_i = e^{-\frac{1}{2} - \frac{\alpha_i}{\varepsilon}}, \quad v_j = e^{-\frac{1}{2} - \frac{\beta_j}{\varepsilon}}, \quad K_{ij} = e^{-\frac{c(x_i, y_j)}{\varepsilon}}.$$

In addition, taking partial derivative of $L(\pi, \alpha, \beta)$ with respect to α and β we get that:

$$\begin{aligned}\frac{\partial L(\pi, \alpha, \beta)}{\partial \alpha_i} &= \sum_j^{k_1} \pi_{ij} - a_i = 0 \implies \sum_j^{k_1} u_i K_{ij} v_j = a_i \implies u_i = \frac{a_i}{\sum_j^{k_1} K_{ij} v_j}, \\ \frac{\partial L(\pi, \alpha, \beta)}{\partial \beta_j} &= \sum_i^{k_0} \pi_{ij} - b_j = 0 \implies \sum_i^{k_0} u_i K_{ij} v_j = b_j \implies v_j = \frac{b_j}{\sum_i^{k_0} u_i K_{ij}}.\end{aligned}$$

Note that for fixed $\varepsilon > 0$ an optimal plan π_ε^* for (2.2) exists and is unique due to the strict convexity of J_ε (e.g., see [30, Lemma 2]). As a result, to recover the optimal plan π_ε^* of (2.2), for fixed $\varepsilon > 0$, one needs to perform a sufficient number of iterations on the vectors u and v to converge to π_ε^* :

$$v_j^{(k+1)} = \frac{b_j}{\{K^T u^{(k)}\}_j} \implies u_i^{(k+1)} = \frac{a_i}{\{K v^{(k+1)}\}_i} \implies \pi_\varepsilon^{(k+1)} = \text{diag}(u^{(k+1)}) K \text{diag}(v^{(k+1)}), \quad (2.3)$$

where the convergence of vectors $u^{(k+1)}$ and $v^{(k+1)}$ is proved to be linear (see [30, page 5]).

Remark 2.1.2. Clearly, as $\varepsilon \rightarrow 0$, J_ε in (2.2) converges to J in (2.1). However, in practice, there is always some value ε_0 , which depends on the problem statement, such that even for $0 < \varepsilon$, but $\varepsilon < \varepsilon_0$, iterations in (2.3) will not converge due to numerical instabilities created by the $\frac{1}{\varepsilon}$ term. One potential solution to address this problem could be implementing log-domain stabilization within iterations, as it was done in [25].

2.2 Linear Assignment Formulation and Auction Method

Alternatively, one can formulate a variation of the discrete MK problem in a form known as *linear assignment problem* or *matching problem*.

Problem 2.2.1. Find an optimal assignment plan $\pi \in \mathbb{R}_{\geq 0}^{k \times k}$ that realizes the inf of the

functional $J(\pi)$ in (2.4):

$$\begin{aligned}
J(\pi) &:= \sum_i^k \sum_j^k c(x_i, y_j) \pi_{ij}, \\
\text{subject to } &\sum_j^k \pi_{ij} = a_i, i = 1, \dots, k, \\
\text{and } &\sum_i^k \pi_{ij} = b_j, j = 1, \dots, k,
\end{aligned} \tag{2.4}$$

where $x_i \in X$ and $a_i = \frac{1}{k}, \forall i, y_j \in Y$ and $b_j = \frac{1}{k}, \forall j$.

Remark 2.2.2. The constraints in Problem 2.2.1 restrict π to be a non-negative doubly stochastic matrix up to a scalar multiplication. In addition, it is a well-known result that the minima of a linear functional over a set of non-negative doubly stochastic matrices will be at the vertex of the set, which corresponds to a permutation matrix (this is the Birkhoff–von Neumann theorem; e.g., see [4, Theorem 5.2]). As a result, Problem 2.2.1 can be rewritten as an unconstrained minimization problem over a set of permutation matrices of dimension $k \times k, \mathbb{P}_k$:

$$\inf_{\pi \in \mathbb{P}_k} J := \frac{1}{k^2} \sum_i^k \sum_j^k c(x_i, y_j) \pi_{ij}.$$

Applications of the assignment problem can be found in dynamical systems [49], optics [34], and robotics [61]. Common techniques used to solve the MK assignment problem are the *Hungarian* algorithm [54, 69] and the *auction* method [11, 12, 88], which we now review. The general auction method solves the assignment Problem 2.2.1 using the dual formulation of the discrete problem. The dual MK problem from Problem 1.1.6 in discrete

form is:

$$\begin{aligned} & \max_{\phi, \psi} \sum_i^k \phi_i a_i + \sum_j^k \psi_j b_j, \\ & \text{subject to } \phi_i + \psi_j \leq c(x_i, y_j), \forall i, j, \end{aligned} \quad (2.5)$$

where the cost function is a lower semi-continuous function and is bounded from below.

Lemma 2.2.3 (Observation 4.3.2 in [58]). *Let $\psi_i^c = \min_j [c(x_i, y_j) - \psi_j]$. If the pair (ϕ, ψ) is admissible in (2.5), then the pair (ψ^c, ψ) is admissible as well.*

Proof.

$$\psi_i^c + \psi_j = \min_j [c(x_i, y_j) - \psi_j] + \psi_j \leq c(x_i, y_j) - \psi_j + \psi_j = c(x_i, y_j). \quad \square$$

Lemma 2.2.4 (Observation 4.3.3 in [58]). *If the pair (ϕ, ψ) is admissible for (2.5), then one obtains a better pair in terms of the objective function value by replacing ϕ with ψ^c .*

Proof.

$$\left\{ \begin{array}{l} \psi_i^c = \min_j [c(x_i, y_j) - \psi_j] \\ \forall j, \phi_i \leq c(x_i, y_j) - \psi_j \end{array} \right\} \implies \phi_i \leq \psi_i^c. \quad \square$$

Using the above two lemmas, the dual MK formulation in (2.5) can be formulated as an unconstrained optimization problem over one variable:

$$\max_{\psi} \sum_i^k \min_j [c(x_i, y_j) - \psi_j] a_i + \sum_j^k \psi_j b_j. \quad (2.6)$$

Next we are going to rewrite (2.6) to be consistent with the auction method terminology:

$$\min_p \sum_i^k \max_j [c_{ij} - p_j] a_i + \sum_j^k p_j b_j, \quad (2.7)$$

where $p_j = -\psi_j$ and $c_{ij} = -c(x_i, y_j)$. The minimization variable p_j is referred to as the

price of an object j and the term $c_{ij} - p_j$ is a total expense for a person i to buy the object j . Then the auction algorithm consists of two phases: *bidding* phase and *assignment* phase. For a fixed bidding step size ε , the bidding step is the following, for each person i :

1. Find the best object j_i and corresponding expense v_i :

$$j_i = \arg \max_j c_{ij} - p_j, \quad v_i = \max_j c_{ij} - p_j.$$

Also, find the second-best offer:

$$w_i = \max_{j \neq j_i} c_{ij} - p_j.$$

2. Compute the bid amount B_{ij_i} :

$$B_{ij_i} = c_{ij_i} - w_i + \varepsilon.$$

Next we move to the assignment phase. For each object j , let I_j be a set of people who bid on the object j in the previous phase. Then:

1. Find a person offering the best bid and find the corresponding bid:

$$i_j = \arg \max_{i \in I_j} b_{ij}, \quad b_{i_j j} = \max_{i \in I_j} b_{ij}.$$

2. Set the price of object j to be the best bid: $p_j = b_{i_j j}$.
3. Update the assignment list by removing any pair (i, j) , $i \neq i_j$, and adding the (i_j, j) pair.

The auction method is guaranteed to terminate with an assignment list that is feasible and is within $k\varepsilon$ of being optimal, if at least one feasible assignment exists (e.g., see Proposition 7.2 in [10]).

2.3 Two Points with Obstacle

In general, given a cost matrix, one could solve the discrete MK problem fairly efficiently, as it was described above. However, in practice forming the cost matrix might not be straightforward. In this section we will discuss the situation when there is a circular obstacle in the domain, with different permeability coefficient.

Definition 2.3.1. Define a circular obstacle $\mathcal{O} \subset \mathbb{R}^2$ as an open disk centered at the origin with radius r :

$$\mathcal{O} = \{x \mid \sqrt{x_1^2 + x_2^2} < r\}.$$

In addition, let $k > 0$ be a permeability coefficient inside the obstacle $\mathcal{O}$ such that given any two points $x, y \in \mathcal{O}$, distance between x and y is $d(x, y) = k\|x - y\|_2$.

Problem 2.3.2. Given an obstacle $\mathcal{O}$ with permeability coefficient k and points $P, Q \notin \overline{\mathcal{O}}$, find a minimum path $z(t)$ from P to Q such that:

$$\begin{aligned} \arg \min_{z(t)} \int_0^1 K(z(t)) \|z'(t)\|_2 dt, \\ z(0) = P, \quad z(1) = Q, \end{aligned}$$

where $z(t)$ is function of a.e. differentiable paths from P to Q and $K(\cdot)$ is piecewise obstacle parameter:

$$K(x) = \begin{cases} k & x \in \mathcal{O} \\ 1 & x \notin \mathcal{O} \end{cases}.$$

Remark 2.3.3. Without loss of generality we can assume that $P = \begin{pmatrix} p_1 \\ p_2 \end{pmatrix}$ and $Q = \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}$ with $p_2 \geq 0$, $q_1 > r$, and $q_2 = 0$. Let $\{P_{tan}^1, P_{tan}^2\}$ and $\{Q_{tan}^1, Q_{tan}^2\}$ be the two points, whose line from P , respectively Q , is tangent to the obstacle. In addition, let P_{norm} be the

point on the boundary of $\mathcal{O}$ which is closest to the point P .

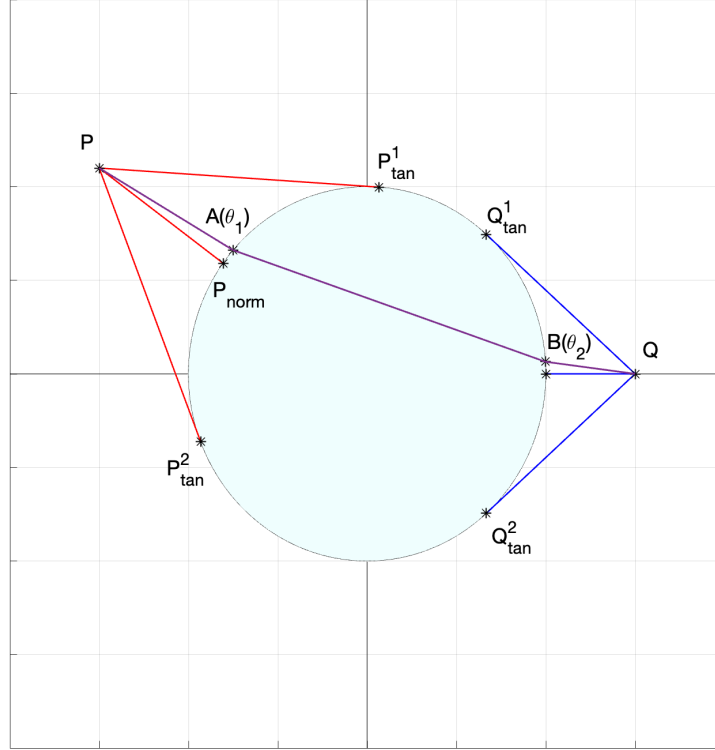


Figure 2.1: Generic path when $k \leq 1$.

Theorem 2.3.4. *Let $k \leq 1$ and let the straight line PQ intersect $\mathcal{O}$. Then the optimal path from P to Q will be a broken line (see Figure 2.1). In addition, the optimal cost will be the solution of the following minimization problem:*

$$c(P, Q) = \min_{\theta_1, \theta_2} \|P - A(\theta_1)\|_2 + k \|A(\theta_1) - B(\theta_2)\|_2 + \|B(\theta_2) - Q\|,$$

$$\text{where } \min\{\angle P_{tan}^1, \angle P_{tan}^2\} \leq \theta_1 \leq \angle P_{norm}, \quad (2.8)$$

$$0 \leq \theta_2 \leq \max\{\angle Q_{tan}^1, \angle Q_{tan}^2\},$$

where $A(\theta_1)$ and $B(\theta_2)$ are points on the boundary of $\mathcal{O}$ such that $\theta_1 \geq \theta_2$ and

$$A(\theta_1) = r \begin{pmatrix} \cos(\theta_1) \\ \sin(\theta_1) \end{pmatrix}, \quad B(\theta_2) = r \begin{pmatrix} \cos(\theta_2) \\ \sin(\theta_2) \end{pmatrix}.$$

Proof. To show that the path will be a broken line we need to rule out the possibility of moving along the boundary. Given any two points $x, y \in \partial\mathcal{O}$

$$c_{arc}(x, y) = \text{Arc}(x, y) = r\alpha,$$

where $\alpha \in (0, \pi]$ is an angle between the points x and y . Similarly,

$$c_{line}(x, y) = k\|x - y\|_2 = 2kr \sin\left(\frac{\alpha}{2}\right).$$

Then, note that when $k \leq 1$, $f(\alpha) = r\alpha - 2kr \sin(\frac{\alpha}{2}) > 0$ for $\alpha \in (0, \pi]$. Hence we get that

$$c_{arc}(x, y) > c_{line}(x, y),$$

and it is always more efficient to move inside the obstacle than along the boundary. Further, the triangle inequality guarantees that there will be only one straight line segment inside the obstacle. In addition, geometrically one can see that entry point to the obstacle will be between the normal and the tangent points of P and exit point from the obstacle will be between the normal point (which is on the x -axis) and the tangent points of Q . $\square$

Lemma 2.3.5. *Let $k \leq 1$ and let the straight line PQ do not intersect $\mathcal{O}$. Then the optimal path will be either the straight line PQ , or the broken line from Theorem 2.3.4. In addition, the optimal cost will be the minimum between $d(P, Q) = \|P - Q\|_2$ and $c(P, Q)$ in (2.8).*

Theorem 2.3.6. *Let $1 < k < \frac{\pi}{2}$ and let the straight line PQ intersect $\mathcal{O}$. Then in general the optimal path from P to Q will be a combination of a straight line segment from P to $\partial\mathcal{O}$,*

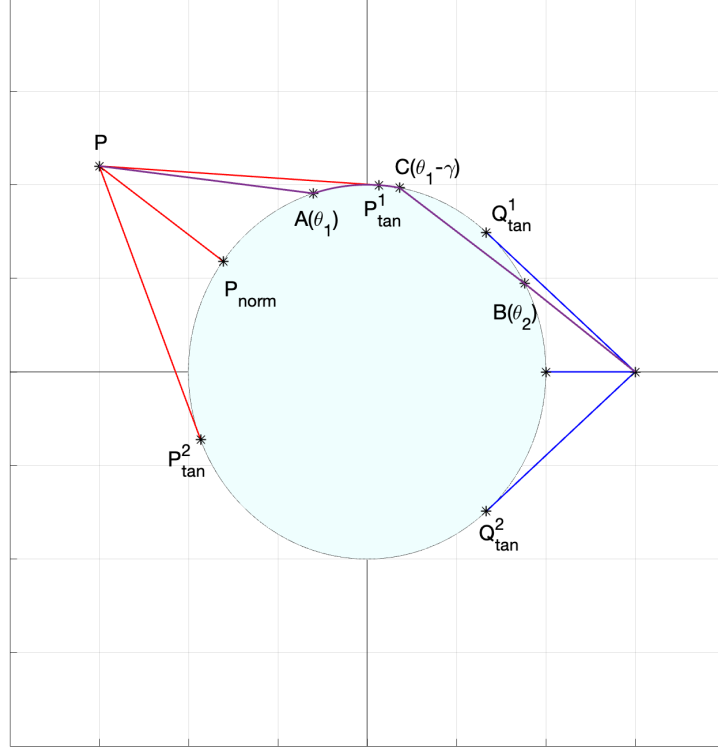


Figure 2.2: Generic path when $1 < k < \frac{\pi}{2}$.

an arc along $\partial\mathcal{O}$, a line segment inside $\mathcal{O}$, and a segment from $\partial\mathcal{O}$ to Q (see Figure 2.2).

In addition, the optimal cost will be the solution of the following minimization problem:

$$c(P, Q) = \begin{cases} \min_{\theta_1, \theta_2} \|P - A(\theta_1)\|_2 + \text{Arc}(A(\theta_1), B(\theta_2)) + \|B(\theta_2) - Q\|, & \text{if } \theta_1 - \theta_2 \leq \gamma, \\ \min_{\theta_1, \theta_2} \|P - A(\theta_1)\|_2 + r\gamma + 2kr \sin\left(\frac{\theta_1 - \theta_2 - \gamma}{2}\right) + \|B(\theta_2) - Q\|, & \text{if } \theta_1 - \theta_2 > \gamma, \end{cases} \quad (2.9)$$

$$\text{where } \min\{\angle P^1_{tan}, \angle P^2_{tan}\} \leq \theta_1 \leq \angle P_{norm},$$

$$0 \leq \theta_2 \leq \max\{\angle Q^1_{tan}, \angle Q^2_{tan}\},$$

and $A(\theta_1)$ and $B(\theta_2)$ are points on the boundary of $\mathcal{O}$ such that $\theta_1 \geq \theta_2$ and

$$A(\theta_1) = r \begin{pmatrix} \cos(\theta_1) \\ \sin(\theta_1) \end{pmatrix}, \quad B(\theta_2) = r \begin{pmatrix} \cos(\theta_2) \\ \sin(\theta_2) \end{pmatrix}.$$

In addition, $\gamma \in (0, \pi]$ is the root of the function $f(\theta)$:

$$f(\theta) = r\theta - 2kr \sin\left(\frac{\theta}{2}\right).$$

Proof. Given any two points $x, y \in \partial\mathcal{O}$

$$c_{arc}(x, y) = \text{Arc}(x, y) = r\alpha,$$

where $\alpha \in (0, \pi]$ is an angle between points x and y . Similarly,

$$c_{line}(x, y) = k\|x - y\|_2 = 2kr \sin\left(\frac{\alpha}{2}\right).$$

Now define the following function $f(\theta)$:

$$f(\theta) = r\theta - 2kr \sin\left(\frac{\theta}{2}\right),$$

and let $\gamma \in (0, \pi]$ be a root of the function $f(\theta)$. If $\gamma \geq \alpha$, then it follows that:

$$f(\theta) \leq 0, \theta \in (0, \alpha] \implies c_{arc}(x, y) \leq c_{line}(x, y).$$

As a result, it is more efficient to move along the boundary. Otherwise, assume than $\alpha > \gamma$.

Then it can be shown that:

$$f(\theta) \leq 0, \theta \in (0, \gamma],$$

$$f(\theta) > 0, \theta \in (\alpha - \gamma, \alpha].$$

Hence the optimal path inside the obstacle will be a combination of a boundary movement and a straight line. Overall, the optimal cost is given in (2.9). In addition, geometrically one can see that the entry point to the obstacle will be between the normal and the tangent points of P and the exit point from the obstacle will be between the normal and the tangent

points of $\mathcal{Q}$. □

Lemma 2.3.7. *Let $1 < k < \frac{\pi}{2}$ and suppose that the straight line PQ do not intersect $\mathcal{O}$. Then the optimal path will be the straight line PQ and the optimal cost will be $d(P, Q) = \|P - Q\|_2$.*

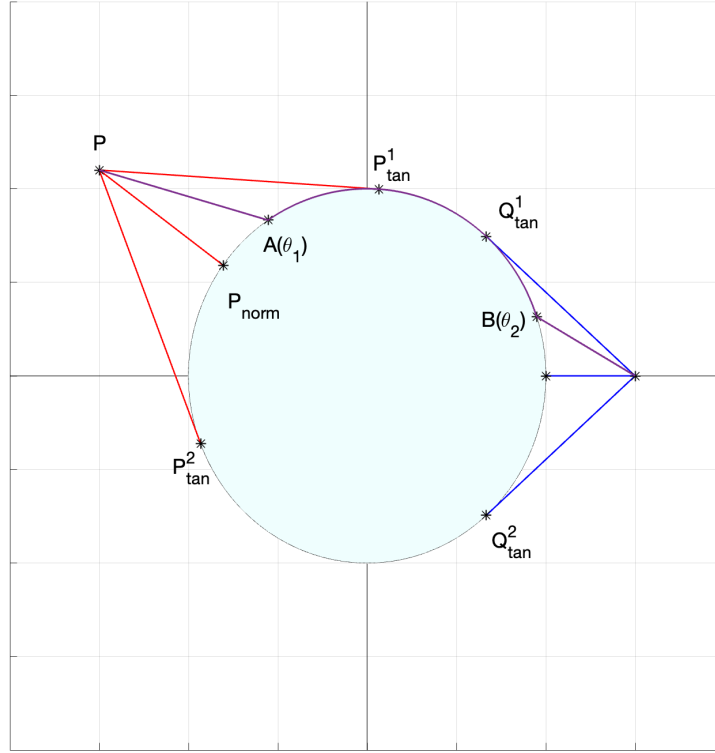


Figure 2.3: Generic path when $k \geq \frac{\pi}{2}$.

Theorem 2.3.8. *Let $k \geq \frac{\pi}{2}$ and let the straight line PQ intersect $\mathcal{O}$. Then the optimal path from P to Q will move along the boundary of $\mathcal{O}$ (see Figure 2.3). In addition, the optimal*

cost will be the solution of the following minimization problem:

$$\begin{aligned}
c(P, Q) &= \min_{\theta_1, \theta_2} \|P - A(\theta_1)\|_2 + \text{Arc}(A(\theta_1), B(\theta_2)) + \|B(\theta_2) - Q\|, \\
\text{where } \min\{\angle P_{tan}^1, \angle P_{tan}^2\} &\leq \theta_1 \leq \angle P_{norm}, \\
0 \leq \theta_2 &\leq \max\{\angle Q_{tan}^1, \angle Q_{tan}^2\},
\end{aligned} \tag{2.10}$$

and $A(\theta_1)$ and $B(\theta_2)$ are points on the boundary of $\mathcal{O}$ such that $\theta_1 \geq \theta_2$ and

$$A(\theta_1) = r \begin{pmatrix} \cos(\theta_1) \\ \sin(\theta_1) \end{pmatrix}, \quad B(\theta_2) = r \begin{pmatrix} \cos(\theta_2) \\ \sin(\theta_2) \end{pmatrix}.$$

Proof. To show that the path will go along the boundary we need to rule out the possibility of moving on a straight line inside the obstacle. Given any two points $x, y \in \partial\mathcal{O}$

$$c_{arc}(x, y) = \text{Arc}(x, y) = r\alpha,$$

where $\alpha \in (0, \pi]$ is an angle between points x and y . Similarly,

$$c_{line}(x, y) = k\|x - y\|_2 = 2kr \sin\left(\frac{\alpha}{2}\right).$$

Then note that when $k \geq \frac{\pi}{2}$, $f(\alpha) = r\alpha - 2kr \sin\left(\frac{\alpha}{2}\right) \leq 0$ for $\alpha \in (0, \pi]$. Hence we get that

$$c_{arc}(x, y) < c_{line}(x, y), \quad \forall \alpha \in (0, \pi),$$

$$c_{arc}(x, y) = c_{line}(x, y), \quad \text{when } \alpha = \pi,$$

and it is always more efficient to move along the boundary than inside the obstacle, except when the angle between points is π . In this case it is the same cost. In addition, geometrically one can see that the entry point to the obstacle will be between the normal and the tangent points of P and the exit point from the obstacle will be between the normal and the

tangent points of Q . □

Lemma 2.3.9. *Let $k \geq \frac{\pi}{2}$ and the straight line PQ does not intersect $\mathcal{O}$. Then the optimal path will be straight line PQ and the optimal cost will be $d(P, Q) = \|P - Q\|_2$.*

2.3.1 Numerical examples

Let $\mathcal{O}$ be an obstacle centered at the origin with the radius 1 and with the permeability coefficient k . In addition, let $\rho_0(x)$ and $\rho_1(y)$ be probability densities supported on X and Y , respectively, such that $X \cap \mathcal{O} = \emptyset$ and $Y \cap \mathcal{O} = \emptyset$. Then, we sample $N = 100$ points x_i and y_j from $\rho_0(x)$ and $\rho_1(y)$ with respect to the corresponding densities, such that $\rho_0(x_i) = \rho_1(y_j) = \frac{1}{N}$, $\forall i, j = 1, 2, \dots, N$. Next, we solve the linear assignment problem using the MATLAB function `matchpairs`, where each x_i points is mapped to the unique y_j points. Since there is the obstacle $\mathcal{O}$ in the region, we use above results to form the cost matrix.

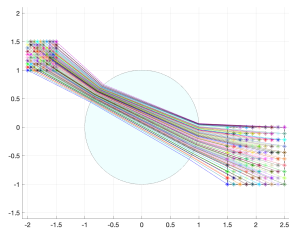
Example 2.3.10. *Let $\rho_0(x) = 4$ and $\rho_1(y) = 1$, where $X = [-2, -1.5] \times [1, 1.5]$ and $Y = [1.5, 2.5] \times [-1, 0]$. In addition, let x_i and y_j be points on a uniform grid. Then, Table 2.1 shows the total cost for the problem, which increases as the constant k grows and stabilizes after some value of k . Moreover, Figure 2.4 illustrates the mapping and the optimal paths for different permeability values.*

Table 2.1: Total cost of mapping 100 points

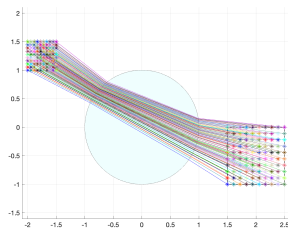
	$k = 0.25$	$k = 0.5$	$k = 0.75$	$k = 0.9$	$k = 1$	$k = 1.01$	$k = 1.05$	$k = 1.25$	$k = 2$
Example 2.3.10	272.9166	321.0642	368.4267	396.1746	414.1313	432.7493	435.0483	435.0483	435.0483
Example 2.3.11	285.4642	331.1290	374.9101	399.4764	414.0982	422.5685	422.9262	422.9262	422.9262

Example 2.3.11. *Let $\rho_0(x) = 1$ and $\rho_1(y) = 4(y_1 - \frac{3}{2})(y_2 + 1)$, where $X = [-2, -1] \times [1, 2]$ and $Y = [1.5, 2.5] \times [-1, 0]$. In addition, let x_i and y_j be randomly generated points with respect to their corresponding densities. Similar to the previous example, from the second*

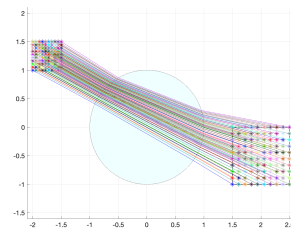
row of the Table 2.1 one can see that the total cost of the mapping increases with the constant k and it converges to some value for $k \geq 1.05$. Also, Figure 2.5 demonstrates the optimal paths for different values of the permeability constant k .



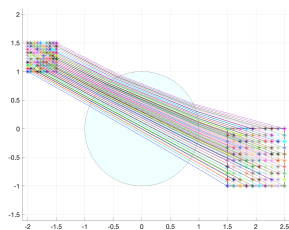
(a) $k = 0.25$



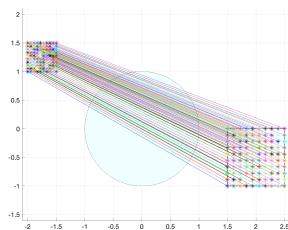
(b) $k = 0.5$



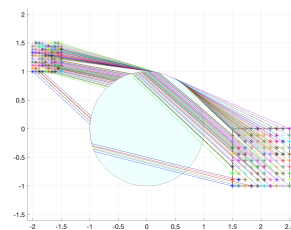
(c) $k = 0.75$



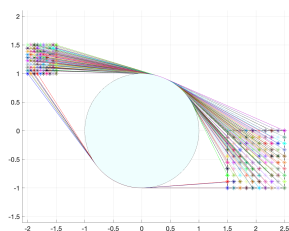
(d) $k = 0.9$



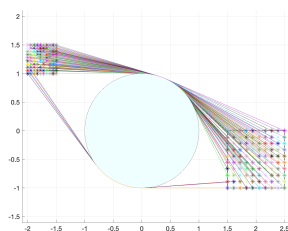
(e) $k = 1$



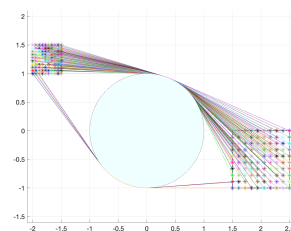
(f) $k = 1.01$



(g) $k = 1.05$

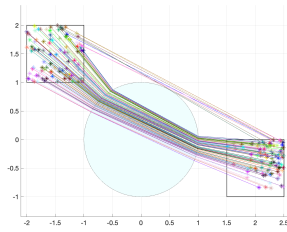


(h) $k = 1.25$

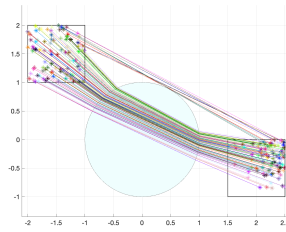


(i) $k = 2$

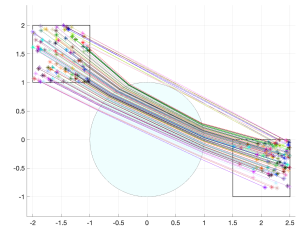
Figure 2.4: Assignment pairs and optimal paths for Example 2.3.10



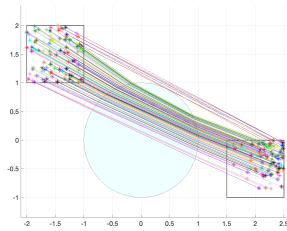
(a) $k = 0.25$



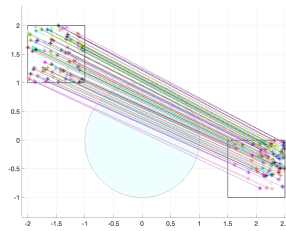
(b) $k = 0.5$



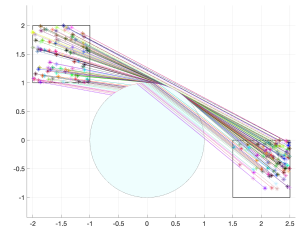
(c) $k = 0.75$



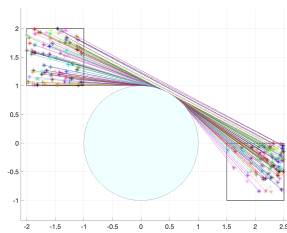
(d) $k = 0.9$



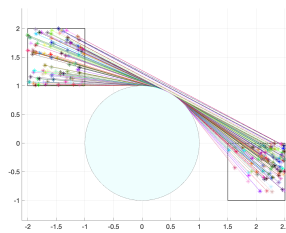
(e) $k = 1$



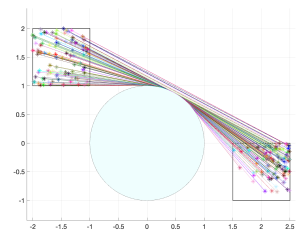
(f) $k = 1.01$



(g) $k = 1.05$



(h) $k = 1.25$



(i) $k = 2$

Figure 2.5: Assignment pairs and optimal paths for Example 2.3.11

CHAPTER 3

CONTINUOUS OT PROBLEMS

3.1 Formulations

In this section we discuss the continuous OT problem with the 2-norm squared cost function, $c(x, y) = \|x - y\|_2^2$. Unless stated otherwise, assume $X = Y = \Omega$ is a convex and compact subset of $\mathbb{R}^2$ and let $\rho_0(x)$ and $\rho_1(y)$ be the probability density functions of the measures μ and ν .

As it was mentioned in introduction, in this case the optimal map $T(x)$ exists and is unique. As a result, one way to solve this problem is by solving the MA-PDE in (1.5). We will shortly discuss appropriate boundary conditions (BCs) for this PDE, at which point a possible method will suggest itself.

An alternative formulation of the OT problem was given by Benamou & Brenier in [8], where they showed that the OT problem is equivalent to the following constrained variational problem (see [80, Theorem 5.28]):

$$\begin{aligned} & \inf_v \int_0^1 \int_{\Omega} \|v(t, x)\|^2 \rho(t, x) dx dt, \\ & \text{such that } \frac{\partial \rho(t, x)}{\partial t} + \nabla \cdot (v(t, x) \rho(t, x)) = 0, \\ & \text{and } \rho(0, x) = \rho_0(x), \quad \rho(1, x) = \rho_1(x), \end{aligned} \tag{3.1}$$

where $v(t, x)$ is a smooth velocity field, and $\rho(t, x)$ is a density interpolation function. The minimization problem (3.1) is known as Benamou-Brenier formulation or fluid dynamics formulation. The relation between the minimization problem (3.1) and the result giving the optimal map T as gradient of a convex function u is that $\nabla u(x) = x + v(0, x)$. The optimal value of the functional in (3.1) is the square of the (L^2 -)Wassertstein distance between ρ_0 and ρ_1 , and it is usually written as $g_W^2(\rho_0, \rho_1)$. This distance g_W has become a popular

way to measure distance between distribution, thanks to its increasing use in statistics and machine learning, e.g., in generative adversarial networks (GAN).

Remark 3.1.1. $[\Omega = \mathbb{R}^d]$ *An important case is when the problem is set on the full space $\mathbb{R}^d$, that is $\Omega = \mathbb{R}^d$ in the above. Now we will need to require that ρ_0 and ρ_1 have finite second moments: $\int_{\mathbb{R}^d} \|x\|^2 \rho_0(x) dx, \int_{\mathbb{R}^d} \|x\|^2 \rho_1(x) dx < \infty$. With this, the existence (and uniqueness) of the optimal map $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ continues to hold; see [80, Theorem 1.22]. Likewise, the previous equivalence result, that is $\nabla u(x) = x + v(0; x)$, about the formulations (1.5) and (3.1) holds as well.*

3.2 Algorithms

Below, we review three basic techniques to solve the OT problem based on either solving the Monge-Ampere PDE, or using the Benamou-Brenier formulation, or using a system of PDEs also resulting from the Benamou-Brenier formulation.

3.2.1 Basic Algorithms Description

3.2.1.1 Monge-Ampere PDE

The most direct approach to find the Monge map is to solve (a discretized version of) (1.5). This has been done many times before; e.g., see [70, 22] for early work, [60] as well as [24, 41, 40, 66] for more recent discretization efforts, as well as the reformulation as a steady-state of an associated parabolic problem of [84]. In this work, we will closely follow the algorithm proposed by Froese in [40]. Before discussing more details, including the choice of BCs, we note that there are two other issues of concern. First of all, since uniqueness is guaranteed only for the map $T(x)$, a solution $u(x)$ of (1.5) is unique only up to a constant. In practice, one can remove this non-uniqueness in several ways: for example, by enforcing that the solution has mean zero ($\int_{\Omega} u(x) dx = 0$) or by fixing a specific value a-priori (say, $u(x_0) = 0$). In [40], the mean zero condition was implemented. The second concern is that we are interested in a convex solution u . A main contribution of [40] on this aspect is related to the discretization of the determinant operator. In fact, a straightforward discretization of (1.5) by centered differences methods will not conserve convexity of the solution during

iterations. Froese in [40] resolves this issue by replacing the determinant operator with the following, and then using centered difference (see Section 3.2.2.1)

$$\det^+(D^2u(x)) = \min_{(v_1, v_2) \in V} \left\{ \max(u_{v_1 v_1}, 0) \max(u_{v_2 v_2}, 0) + \min(u_{v_1 v_2}, 0) + \min(u_{v_2 v_1}, 0) \right\}, \quad (3.2)$$

where V is the set of all orthonormal bases of $\mathbb{R}^2$. Under this specific discretization, Froese gives some convergence results for the finite difference scheme (see [40, Theorem 5]).

3.2.1.2 Regularized Benamou-Brenier

The next algorithm we considered is based upon the optimization problem expressed by (3.1). The basic approach based on solving this optimization problem was discussed already in the original paper [8] where the authors confronted the difficulties arising from lack of strict convexity of the functional. In our presentation, we will follow the recent work of Li et al., [59], where the authors expand upon the work [8].

To begin with, the authors of [59] replace (as already done in [8]) the velocity field function v with the flux function m :

$$v(t, x) = \frac{m(t, x)}{\rho(t, x)},$$

and (3.1) becomes

$$\begin{aligned} \min_{m, \rho} \int_0^1 \int_{\Omega} \left\| \frac{m(t, x)}{\rho(t, x)} \right\|_2^2 \rho(t, x) dx dt, \\ \frac{\partial \rho(t, x)}{\partial t} + \nabla \cdot m(t, x) = 0, \\ \rho(0, x) = \rho_0(x), \quad \rho(1, x) = \rho_1(x). \end{aligned} \quad (3.3)$$

Next, motivated by the lack of strict convexity and by the fact that if $\rho(t, x)$ becomes 0 (or even close to 0), a numerical scheme will run into difficulties, the authors in [59] regularize (3.3) as follows. They perform the change of variable $m(t, x) = \tilde{m}(t, x) + \beta \nabla \rho(t, x)$,

where β is a regularization parameter, and rewrite (3.3) as:

$$\begin{aligned} \min_{\tilde{m}, \rho} \int_0^1 \int_{\Omega} \left\{ \frac{||\tilde{m}(t, x)||^2}{\rho(t, x)} + \beta^2 (\nabla \log \rho(t, x))^2 \rho(t, x) \right\} dx dt + 2\beta \mathcal{D}(\rho_1 | \rho_0), \\ \frac{\partial \rho(t, x)}{\partial t} + \nabla \cdot \tilde{m}(t, x) = 0, \\ \rho(0, x) = \rho_0(x), \quad \rho(1, x) = \rho_1(x), \end{aligned} \quad (3.4)$$

where $\mathcal{D}(\rho_1 | \rho_0)$ is the constant

$$\mathcal{D}(\rho_1 | \rho_0) = \int_{\Omega} \rho_1(x) \log \rho_1(x) dx - \int_X \rho_0(x) \log \rho_0(x) dx.$$

We note that with this reformulation the Wasserstein distance $g_W(\rho_0, \rho_1)$ is thus approximated by the square root of the minimum value in (3.4).

The chief difference between (3.3) and (3.4) is the *Fisher information* term $\mathcal{I}(\rho)$:

$$\mathcal{I}(\rho)(t, x) = \int_{\Omega} (\nabla \log \rho(t, x))^2 \rho(t, x).$$

The purpose of this regularization is that adding the Fisher information term maintains the probability densities strictly positive (it acts as a barrier function), and further introduces strict convexity into the original minimization problem. Some convergence results as $\beta \rightarrow 0$ of the solution of (3.4) to that of (3.3) are in [56]. But, of course, in practice one cannot let $\beta \rightarrow 0$ and balancing the appropriate value of β with the discretization level in x and t will be important; see Table 3.1, which clearly shows the need to decrease β up to the discretization error level in space and time, but that there is no benefit beyond that.

3.2.1.3 Geodesic

The last technique we present is based on the recent work by Cui et al. [29]. The technique is based on a reformulation of the Benamou-Brenier problem (3.1) posed in $\mathbb{R}^d$; see Remark

Table 3.1: Errors in Optimal Map for Problem 3.4.1, obtained by the Regularized Benamou-Brenier method with different β values and varying levels of discretization dt and dx .

	dt = 1/6			dt = 1/11			dt = 1/21		
β	$dx = \frac{1}{16}$	$dx = \frac{1}{32}$	$dx = \frac{1}{64}$	$dx = \frac{1}{16}$	$dx = \frac{1}{32}$	$dx = \frac{1}{64}$	$dx = \frac{1}{16}$	$dx = \frac{1}{32}$	$dx = \frac{1}{64}$
10^{-1}	0.550	0.563	0.567	0.553	0.564	0.567	0.555	0.566	0.569
10^{-2}	0.053	0.046	0.050	0.054	0.052	0.056	0.056	0.054	0.058
10^{-4}	0.028	0.020	0.017	0.031	0.017	0.010	0.036	0.021	0.011
10^{-8}	0.028	0.021	0.018	0.031	0.017	0.010	0.036	0.021	0.011

3.1.1. The authors of [29] reformulate the problem as the Hamiltonian PDE system

$$\begin{aligned}
\frac{\partial \rho(t, x)}{\partial t} + \nabla \cdot (\rho(t, x) \nabla S(t, x)) &= 0, \\
\frac{\partial S(t, x)}{\partial t} + \frac{1}{2} \|\nabla S(t, x)\|^2 &= 0, \\
\rho(0, x) &= \rho_0(x), \quad \rho(1, x) = \rho_1(x),
\end{aligned} \tag{3.5}$$

where $\nabla S = v$. Here, the Hamiltonian is given by $H(\rho, S) = \frac{1}{2} \int_{\mathbb{R}^d} \|\nabla S\|^2 \rho dx$. When the reformulation holds (see [29]), then, if $S(0, x)$ is known, for the Wasserstein distance the equality $g_W(\rho_0, \rho_1) = \sqrt{2H(\rho_0(x), S(0, x))}$ holds as well. Furthermore, the optimal map has the form $T(x) = x + v(0, x)$. The basic idea of [29] is to find an initial $S(0, x)$ (or $v(0, x)$) such that the trajectory starting at $(\rho_0, S(0, x))$ arrives at ρ_1 at $t = 1$. This is the well-known *geodesic equation* between the two densities ρ_0 and ρ_1 on the Wasserstein manifold (see [87]). But, since S is defined only up to an arbitrary constant, the boundary value problem (3.5) cannot have a unique solution. Because of this fact, we end up (just like the authors of [29] did, albeit at the discrete level) using the formulation based on ρ

and v :

$$\begin{aligned}\frac{\partial \rho(t, x)}{\partial t} + \nabla \cdot (\rho(t, x)v(t, x)) &= 0, \\ \frac{\partial v(t, x)}{\partial t} + \frac{1}{2} \nabla \|v(t, x)\|^2 &= 0, \\ \rho(0, x) &= \rho_0(x), \quad \rho(1, x) = \rho_1(x).\end{aligned}\tag{3.6}$$

As far as the Wasserstein distance is concerned, one can obtain it from the square root of

$$\int_{\mathbb{R}^d} \|v(0, x)\|^2 \rho_0(x) dx.\tag{3.7}$$

3.2.2 Numerical Implementation

Now we discuss some implementation issues of the basic algorithms from Section 3.2.1.

Restriction 3.2.1. *We will consider two types of problems. (a) Periodic problems, which are naturally formulated on the torus, and (b) Problems given on finite rectangular domains: $\Omega = [a_0, a_1] \times [b_0, b_1]$. In fact, we could also consider two different rectangular domains for the two different densities, say $X = [a_0, a_1] \times [b_0, b_1]$ for ρ_0 and $Y = [\alpha_0, \alpha_1] \times [\beta_0, \beta_1]$ for ρ_1 , but it is a simple linear change of variable to reduce Y to X .*

Remark 3.2.2. *Note that consideration of a finite rectangular domain poses some nontrivial restrictions on the approach of Section 3.2.1.3. In particular, the formulation based on (3.6) of [29] on a finite domain Ω imposes zero-flux conditions, more specifically homogeneous boundary conditions for v and homogeneous Neumann conditions for $\rho(t, x)$: $v = 0$ and $\frac{\partial \rho}{\partial n} = 0$ on $\partial\Omega$ (here, n is the outer normal). If these are severely violated, the use of this approach on a truncated domain is of dubious value.*

3.2.2.1 Monge-Ampere Discretization

As mentioned in Section 3.2.1.1, we follow the approach of [40] for discretizing (1.5).

The first thing to decide is how to choose the orthonormal vectors in the monotone discretization of the derivatives (see (3.2)). Rather than considering all possible orthonormal

bases of $\mathbb{R}^2$, we restrict the set V of (3.2) to these two bases:

$$\{\tilde{v}_1, \tilde{v}_2\} = \left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right\}, \quad \{\hat{v}_1, \hat{v}_2\} = \left\{ \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right\}.$$

Using these sets, we derived centered difference formulas for both first and second derivatives. (The first derivatives of course are needed to evaluate $\rho_1(\nabla u)$.) For example, for the first derivatives we have

$$\begin{aligned} [D_{\tilde{v}_1} u]_{ij} &= \frac{u_{i+1,j} - u_{i-1,j}}{2h}, & [D_{\tilde{v}_2} u]_{ij} &= \frac{u_{i,j+1} - u_{i,j-1}}{2h}, \\ [D_{\hat{v}_1} u]_{ij} &= \frac{u_{i+1,j+1} - u_{i-1,j-1}}{2\sqrt{2}h}, & [D_{\hat{v}_2} u]_{ij} &= \frac{u_{i,+1,j-1} - u_{i-1,j+1}}{2\sqrt{2}h}, \end{aligned}$$

where h is the mesh size of the grid. Similarly for the second derivative approximations (see [40]).

The second key aspect of the algorithm is how to select the boundary conditions. For periodic problems, we cannot use (1.5) as given, since there cannot be any periodic convex function u (except the constants) satisfying $\rho_1 \det D^2 u = \rho_0$ with ρ_0 and ρ_1 periodic. The appropriate setup now is not for u to be convex but rather $\frac{1}{2}\|x\|^2 - u$ to be (e.g., see [60] and [80, pp. 211–214]), so that the the optimal map will be its gradient, that is $T(x) = x - \nabla u(x)$ (in the periodic sense), and now u satisfies this Monge-Ampere PDE:

$$\rho_1(x - \nabla u) \det(I - D^2 u) = \rho_0(x). \quad (3.8)$$

For rectangular domains, we followed the approach of [40] and implemented the so-called *transport boundary conditions*: “The gradient of u maps the boundary onto the boundary”. That is:

$$\nabla u(\partial\Omega) = \partial\Omega. \quad (3.9)$$

These are particularly easy to implement when we have two rectangular regions (in fact, for

more general polygonal regions as well) $X = [a_0, a_1] \times [b_0, b_1]$ and $Y = [\alpha_0, \alpha_1] \times [\beta_0, \beta_1]$.

Then (3.9) become:

$$\begin{aligned} u_{x_1}(a_0, x_2) &= \alpha_0, \quad \forall x_2 \in [b_0, b_1], & u_{x_1}(a_1, x_2) &= \alpha_1, \quad \forall x_2 \in [b_0, b_1], \\ u_{x_2}(x_1, b_0) &= \beta_0, \quad \forall x_1 \in [a_0, a_1], & u_{x_2}(x_1, b_1) &= \beta_1, \quad \forall x_1 \in [a_0, a_1]. \end{aligned} \quad (3.10)$$

Let h be a mesh size of the $N_1 \times N_2$ grid, such that $x_1^1 = a_0$, $x_1^{N_1} = a_1$, $x_2^1 = b_0$, and $x_2^{N_2} = b_1$. Then (3.10) are discretized as:

$$\begin{aligned} \frac{u_{3,j} - u_{1,j}}{2h} &= \alpha_0, \quad \forall j \in [2, N_2 - 1], & \frac{u_{N_1,j} - u_{N_1-2,j}}{2h} &= \alpha_1, \quad \forall j \in [2, N_2 - 1], \\ \frac{u_{i,3} - u_{i,1}}{2h} &= \beta_0, \quad \forall i \in [2, N_1 - 1], & \frac{u_{i,N_2} - u_{i,N_2-2}}{2h} &= \beta_1, \quad \forall i \in [2, N_1 - 1], \end{aligned}$$

and at the corners:

$$\begin{aligned} \frac{u_{3,3} - u_{1,1}}{2\sqrt{2}h} &= \frac{\alpha_0 + \beta_0}{\sqrt{2}}, & \frac{u_{N_1,1} - u_{N_1-2,3}}{2\sqrt{2}h} &= \frac{\alpha_1 - \beta_0}{\sqrt{2}}, \\ \frac{u_{1,N_2} - u_{3,N_2-2}}{2\sqrt{2}h} &= \frac{-\alpha_0 + \beta_1}{\sqrt{2}}, & \frac{u_{N_1,N_2} - u_{N_1-2,N_2-2}}{2\sqrt{2}h} &= \frac{\alpha_1 + \beta_1}{\sqrt{2}}. \end{aligned}$$

Finally, we add the condition $u_{11} = 0$ to remove the ambiguity resulting from the fact that $u(x)$ is defined only up to a constant.

Remark 3.2.3. *To be able to handle non-polygonal domains X and Y , in [9] the authors proposed to use boundary conditions inherited from the signed distance function $\sigma(\nabla u(x))$:*

$$\sigma(\nabla u(x)) = \text{dist}(\nabla u(x), \partial Y) = 0, \quad \forall x \in \partial X.$$

Unfortunately, we were not successful in implementing this type of boundary conditions and to replicate the results of [9].

Once the previous discretization is adopted, the Monge-Ampère PDE is replaced by its difference analog and the discretized system is solved using Newton's method. As

usual, for Newton's method it is important to give a good initial guess. To this end, we experimented with three different approaches.

Initial Guess for Newton.

- (i) *Trivial.* Here we use the identity as the approximation to the optimal map, that is

$$u(x) = \frac{1}{2}\|x\|^2.$$

- (ii) *Interpolation.* We interpolate a solution obtained on a coarse grid to obtain an initial guess for the fine grid. In other words, the initial guess u^0 on the $N_1 \times N_2$ grid is the solution u^* defined on $\frac{N_1}{2} \times \frac{N_2}{2}$ grid, which is interpolated on the $N_1 \times N_2$ grid (we used the MATLAB built-in function `interp2` which does linear interpolation). This is done recursively until $\max N_1, N_2 \leq 8$. And for the coarsest grid we used the identity map as an initial guess.

- (iii) *Homotopy.* Here, let $X = Y$. We introduce an artificial source probability density

$$\tilde{\rho}_0(\mu, x):$$

$$\tilde{\rho}_0(\mu, x) = (1 - \mu)\rho_1(x) + \mu\rho_0(x),$$

and transport $\tilde{\rho}_0$ to ρ_1 as μ goes from zero to one. At $\mu = 0$ we are mapping ρ_1 to ρ_1 , which corresponds to the identity mapping. We begin with $\Delta^0 = 1/10$ or $1/100$ and update it based on the number of Newton iterations in the previous step:

$$\Delta^i = 2^{\frac{4-k}{3}} \Delta^{i-1}, \quad \mu^i = \mu^{i-1} + \Delta^i,$$

where k is the number of Newton iterations required to solve problem at μ^{i-1} .

The results of Table 3.2 compare performance of the above different options to initialize Newton's method, and they are typical, showing that in the end the simplest option of using the trivial initial guess is also the most efficient. Of course, in all cases the same solution was recovered.

Table 3.2: Total number of Newton's iterations, number of homotopy steps if applicable, and execution times, when the nonlinear Monge-Ampère discretization is solved with different initial guesses for Newton's method.

	Identity		Interpolation		Homotopy	
dx	Newton	Time	Newton	Time	Nsteps, Newton	Time
Problem 3.4.9						
$\frac{1}{8}$	6	0.01	6	0.03	12, 50	0.05
$\frac{1}{16}$	6	0.02	10	0.05	16, 66	0.18
$\frac{1}{32}$	8	0.19	12	0.34	15, 62	1.47
$\frac{1}{64}$	9	2.78	16	5.25	15, 62	20.2
$\frac{1}{128}$	17	78.3	21	101	13, 54	262
Problem 3.4.10						
$\frac{1}{8}$	7	0.23	7	0.19	3, 20	0.06
$\frac{1}{16}$	6	0.50	7	0.66	5, 40	0.16
$\frac{1}{32}$	6	1.91	11	2.65	7, 60	1.60
$\frac{1}{64}$	8	9.17	15	13.8	7, 61	22.1
$\frac{1}{128}$	12	81.9	14	104	4, 32	281

3.2.2.2 Regularized Benamou-Brenier Discretization

Here, we need to solve problem (3.4) by discretizing it on a spatial and temporal grid. Note that the dependence of the discretized constraints on $\tilde{m}$ and ρ is linear when standard finite difference schemes are used for derivative approximations. This method is well suited for rectangular domains, but not when we pose the problem on the torus (periodic problems).

Let the domain $\Omega = [a_0, a_1] \times [b_0, b_1]$ be discretized by a uniform mesh: $h = dx_1 = dx_2$ such that $(x_1)_i = a_0 + h(i - 1)$, $i = 1, 2, \dots, N$, $(x_2)_j = b_0 + h(j - 1)$, $j = 1, 2, \dots, M$. In addition, let L be the number of internal time values such that $t_l = ldt$, $l = 0, 1, \dots, L + 1$ and $dt = \frac{1}{L+1}$. The probability density values $\rho(t_l, (x_1)_i, (x_2)_j)$ are defined on the grid vertices and $\tilde{m}(t_l, (x_1)_{i+0.5}, (x_2)_{j+0.5})$ are defined on the grid edges.

Then, the discretization of (3.4) is:

$$\begin{aligned} & \min_U f(U), \\ & \text{subject to } AU = b, \end{aligned} \quad (3.11)$$

where $U = \begin{pmatrix} m^{x_1}(\cdot) \\ m^{x_2}(\cdot) \\ R(\cdot) \end{pmatrix} \in \mathbb{R}^{((N-1)M+N(M-1))(L+1)+LNM}$ represent the grid analog comprising the two components of $\tilde{m}$ and of ρ and we need to impose/maintain positivity of R .

So, $m^{x_1} \in \mathbb{R}^{(N-1) \times M \times (L+1)}$, $m^{x_2} \in \mathbb{R}^{N \times (M-1) \times (L+1)}$, $R \in \mathbb{R}^{N \times M \times L}$, and the objective function $f(U)$ is given by

$$\begin{aligned} f(U) &= f(m^{x_1}, m^{x_2}, R) = \\ &= \sum_{l=0}^L \sum_{i=1}^{N-1} \sum_{j=1}^{M-2} \frac{2(m_{i+0.5,j,l}^{x_1})^2}{\rho_{i,j,l} + \rho_{i+1,j,l}} + \frac{\beta^2}{h^2} (\log(\rho_{i,j,l}) - \log(\rho_{i+1,j,l}))^2 \frac{\rho_{i,j,l} + \rho_{i+1,j,l}}{2} + \\ &+ \sum_{l=0}^L \sum_{i=1}^{N-2} \sum_{j=1}^{M-1} \frac{2(m_{i,j+0.5,l}^{x_2})^2}{\rho_{i,j,l} + \rho_{i,j+1,l}} + \frac{\beta^2}{h^2} (\log(\rho_{i,j,l}) - \log(\rho_{i,j+1,l}))^2 \frac{\rho_{i,j,l} + \rho_{i,j+1,l}}{2}, \end{aligned} \quad (3.12)$$

and $A \in \mathbb{R}^{((L+1)NM+L) \times (((N-1)M+N(M-1))(L+1)+LNM)}$ and $b \in \mathbb{R}^{(L+1)NM+L}$ are obtained from the constraints:

$$\begin{aligned} & \frac{\rho_{i,j,l+1} - \rho_{i,j,l}}{dt} + \frac{1}{h} ((m_{i+0.5,j,l}^{x_1} - m_{i-0.5,j,l}^{x_1}) + (m_{i,j+0.5,l}^{x_2} - m_{i,j-0.5,l}^{x_2})) = 0, \\ & i = 1, 2, \dots, N; \quad j = 1, 2, \dots, M; \quad l = 0, 1, 2, \dots, L; \\ & \sum_{i=1}^N \sum_{j=1}^M \rho_{i,j,l} = 1, \quad l = 1, 2, \dots, L; \end{aligned} \quad (3.13)$$

and the zero flux condition is used for ghost cells: $m_{-0.5,j,l}^{x_1} = m_{n+0.5,j,l}^{x_1} = 0$ and $m_{i,-0.5,l}^{x_2} = m_{i,m+0.5,l}^{x_2} = 0$. Then, we solve the discrete minimization problem (3.11) by moving from the current iterate in the direction of the best quadratic approximation (a modified Newton's method) subject to linear constraints and positivity constraints on the components R of U

(cfr. with [59]):

$$U^{k+1} = U^k + \alpha d^k, \quad R^{k+1} > 0, \quad (3.14)$$

where

$$d^k = \arg \min_d d^T \nabla f(U^k) + \frac{1}{2} d^T H(u^k) d, \quad (3.15)$$

such that $Ad = 0$,

where $\nabla f(u)$ and $H(u)$ are the gradient and the Hessian of $f(u)$, respectively. This is a classic quadratic minimization problem for d in the kernel of A , subject to positivity constraints on the subset of the variables associated to R . We solve this problem in three steps: (i) First we find an orthonormal basis for the kernel of A , call W its matrix representation; then (ii) we write $d = Wc$ for some c ; (iii) finally, we reformulate the minimization problem for c , by requiring positivity of the relevant variables. We solve this last problem by the tried and true MATLAB function `quadprog`. Finally, we refer to Section 3.4 for the values of α in (3.14) that we used in practice.

3.2.2.3 Geodesic discretization: Multiple shooting method

The authors of [29] provided a `Matlab` code on `github` implementing a multiple shooting approach to solve the discretized version of (3.5)-(3.6), for both 1-d and 2-d. We have used their code in our 2-d experiments, as well as implemented our own versions for 1-d problems.

For completeness, we next describe our discretization for $\rho(t, x)$ and $v(t, x)$. Let the domain $X = Y = [a_0, a_1]$ be discretized by a uniform mesh of n points with spacing dx , and let L be the number of internal time points with spacing $dt = \frac{1}{L+1}$. Density values $\rho(t_l, x_i)$ are defined on grid points and $v(t_l, x_{i+0.5})$ are defined on the grid edges.

Then, the discretized problem (3.6) can be formulated as:

$$\begin{aligned} \frac{\rho_{i,l+1} - \rho_{i,l}}{dt} + \rho_{i,l} \frac{v_{i+0.5,l} - v_{i-0.5,l}}{dx} + \frac{v_{i+0.5,l} + v_{i-0.5,l}}{2} \cdot \frac{\rho_{i+1,l} - \rho_{i-1,l}}{2dx} &= 0, \quad i = 1, 2, \dots, n; \\ \frac{v_{i+0.5,l+1} - v_{i+0.5,l}}{dt} + v_{i+0.5,l} \frac{v_{i+1.5} - v_{i-0.5}}{2dx} &= 0, \quad i = 1, 2, \dots, n-1; \end{aligned}$$

where $l = 0, 1, \dots, L$. For periodic boundary conditions, we then impose

$$\rho_{-1,l} = \rho_{n-1,l}, \quad \rho_{n+1,l} = \rho_{2,l}, \quad l = 0, 1, \dots, L.$$

Instead, zero flux condition on $v(x, t)$ and homogeneous Neumann conditions on $\rho(x, t)$ give

$$\begin{aligned} v_{0.5,l} = v_{n+0.5,l} &= 0, \quad l = 0, 1, \dots, L; \\ \frac{\rho_{2,l} - \rho_{-1,l}}{2dx} = \frac{\rho_{n+1,l} - \rho_{n-1,l}}{2dx} &= 0, \quad l = 0, 1, \dots, L. \end{aligned}$$

Remark 3.2.4 (Wasserstein distance approximation). *In the fluid-flow formulation, the minimum value of (3.1) gives $g_W^2(\rho_0, \rho_1)$, the square of the Wasserstein distance between ρ_0 and ρ_1 . To obtain this distance when we have just the optimal map (say, by solving the Monge-Ampère PDE), one needs to approximate the integral below*

$$g_W^2 = \int_{\Omega} \|T(x) - x\|^2 \rho_0(x) dx, \quad (3.16)$$

and approximating this integral on the grid where we have solved for $T(x)$ renders an approximation scheme for g_W^2 . Of course, this is also a very inexpensive procedure once $T(x)$ is known.

Remark 3.2.5. *Ideally, the two different fluid-flow formulations that we considered, that based on (3.3) and that based on (3.6), should recover the same geodesic trajectory from ρ_0 to ρ_1 . However, in general, this is not borne out in practice, well beyond the discretization error level; e.g., see Figure 3.1 for a plot of the (sup-norm of the) difference between the*

two trajectories obtained by the two different methods. This discrepancy highlights the need to make sure that the assumptions underlying validity of different formulations are satisfied.

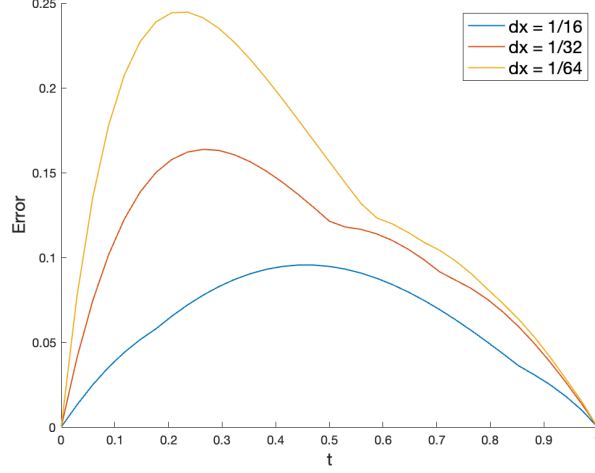


Figure 3.1: Sup-norm of the difference on the grid between $\rho(t, x)$ obtained by the techniques based on (3.4) (with $\beta = 10^{-5}$) and (3.5) on Problem 3.4.2.

3.2.3 Hybrid technique: From $v(0, x)$ to the full geodesic evolution

One more method with which we have experimented rests on a very natural idea. Suppose we are able, for example by exploiting separability (see Section 3.3 and especially Section 3.3.3.1), to obtain the initial velocity $v(0, x)$ in the fluid-flow formulation. Then, we can approximate the geodesic by solving the PDE system (3.5), with no need to iterate to find the correct initial velocity. The resulting hybrid method allows to obtain the full evolution of the density without the need to solve a nonlinear problem to get the correct initial velocity. For example, the density evolution of Figure 3.5 was obtained using this hybrid technique, with the initial velocity obtained from the approach in Section 3.3.3.1. In principle, for as long as the conditions relating the optimal map T to the initial velocity $v(0, x)$ hold, namely $T(x) = x + v(0, x)$, then one may obtain the initial velocity also from the solution of the Monge-Ampère PDE, that is from T ; the advantage of this point of view

is that it applies even if the problem is not separable.

3.3 Separable Class of OT Problems

As it turns out, many of the problems that arise in the study of optimal transport are of a special type, which we will call *separable*, in analogy to the technique of separation of variables commonly performed for PDEs. To witness, examples of separable problems are in the works [40, 59, 29], and in fact the common case of Gaussian distributions ρ_0 and/or ρ_1 , as well as the case of uniform distribution are all separable. Below, for these separable problems we will give some new theoretical results and devise much more accurate and efficient versions of the previously examined numerical techniques, specifically for the solution of the Monge-Ampère PDE. By contrast, in the above cited works, the separability of the OT problems was neither recognized nor exploited.

3.3.1 Separation Process

We describe the process on rectangular domains, but with minimal changes everything carries over to $\mathbb{R}^d$ or $\mathbb{T}^p$. Let the rectangular domains of $\rho_0(x)$ and $\rho_1(y)$ be $X = [a_0, a_1] \times [b_0, b_1]$ and $Y = [\alpha_0, \alpha_1] \times [\beta_0, \beta_1]$.

Definition 3.3.1. *We say that the pair of positive probability densities ρ_0 and ρ_1 is a separable pair of densities on X , respectively Y , if*

$$\rho_0(x) = R_1(x_1)R_2(x_2), \quad x \in X, \quad \rho_1(y) = S_1(y_1)S_2(y_2), \quad y \in Y. \quad (3.17)$$

Equivalently, we will just say that ρ_0 and ρ_1 are separable.

Lemma 3.3.2. *Let ρ_0 and ρ_1 be a separable pair of densities as in Definition 3.3.1. Then,*

we can always assume that

$$\begin{aligned} R_1(x_1) > 0, \quad \int_{a_0}^{a_1} R_1(x_1) dx_1 = 1, \quad R_2(x_2) > 0, \quad \int_{b_0}^{b_1} R_2(x_2) dx_2 = 1, \\ S_1(y_1) > 0, \quad \int_{\alpha_0}^{\alpha_1} S_1(y_1) dy_1 = 1, \quad S_2(y_2) > 0, \quad \int_{\beta_0}^{\beta_1} S_2(y_2) dy_2 = 1. \end{aligned} \quad (3.18)$$

Proof. The claim follows from the fact that $\rho_0(x_1, x_2)$ and $\rho_1(y_1, y_2)$ are probability densities and therefore:

$$\begin{aligned} 1 &= \int_{b_0}^{b_1} \int_{a_0}^{a_1} \rho_0(x_1, x_2) dx_1 dx_2 = \int_{b_0}^{b_1} \int_{a_0}^{a_1} R_1(x_1) R_2(x_2) dx_1 dx_2 = \int_{b_0}^{b_1} \int_{a_0}^{a_1} \frac{m}{m} R_1(x_1) R_2(x_2) dx_1 dx_2, \\ 1 &= \int_{b_0}^{b_1} \int_{a_0}^{a_1} m R_1(x_1) \frac{1}{m} R_2(x_2) dx_1 dx_2 = \left[\int_{a_0}^{a_1} m R_1(x_1) dx_1 \right] \left[\int_{b_0}^{b_1} \frac{1}{m} R_2(x_2) dx_2 \right], \\ 1 &= \int_{\beta_0}^{\beta_1} \int_{\alpha_0}^{\alpha_1} \rho_1(y_1, y_2) dy_1 dy_2 = \int_{\beta_0}^{\beta_1} \int_{\alpha_0}^{\alpha_1} S_1(y_1) S_2(y_2) dy_1 dy_2 = \int_{\beta_0}^{\beta_1} \int_{\alpha_0}^{\alpha_1} \frac{n}{n} S_1(y_1) S_2(y_2) dy_1 dy_2, \\ 1 &= \int_{\beta_0}^{\beta_1} \int_{\alpha_0}^{\alpha_1} \frac{1}{n} S_1(y_1) n S_2(y_2) dy_1 dy_2 = \left[\int_{\alpha_0}^{\alpha_1} \frac{1}{n} S_1(y_1) dy_1 \right] \left[\int_{\beta_0}^{\beta_1} n S_2(y_2) dy_2 \right], \end{aligned}$$

for some constants m and n . Therefore, we can always renormalize R_1, R_2, S_1 and S_2 , as stated in (3.18). $\square$

Next, we look at the implication of having separable densities on the two formulation considered in this work: the Monge Ampère PDE and the Benamou-Brenier formulation.

3.3.2 Separable Monge-Ampère

Recall that, on the rectangular domains X and Y , we need to solve the following problem for positive ρ_0 and ρ_1 :

$$\begin{aligned} \det(D^2 u(x)) &= \frac{\rho_0(x)}{\rho_1(\nabla u(x))}, \\ u_{x_1}(a_0, x_2) &= \alpha_0, \quad \forall x_2 \in [b_0, b_1], \quad u_{x_1}(a_1, x_2) = \alpha_1, \quad \forall x_2 \in [b_0, b_1], \\ u_{x_2}(x_1, b_0) &= \beta_0, \quad \forall x_1 \in [a_0, a_1], \quad u_{x_2}(x_1, b_1) = \beta_1, \quad \forall x_1 \in [a_0, a_1], \end{aligned} \quad (3.19)$$

and that the solution $u(x)$ is unique, up to a constant.

Theorem 3.3.3. *Let ρ_0 and ρ_1 be separable densities as in Definition 3.3.1. Then, the 2-d Monge-Ampère problem (3.19) can be reduced to the two 1-d Monge-Ampère problems (3.20).*

Proof. Since the densities are separable, we seek a solution of (3.19) of the form $u(x) = u_1(x_1) + u_2(x_2)$. If so, the BCs in (3.19) rewrite as

$$u'_1(a_0) = \alpha_0, \quad u'_1(a_1) = \alpha_1, \quad u'_2(b_0) = \beta_0, \quad u'_2(b_1) = \beta_1.$$

The differential part of (3.19) rewrites as

$$\det(D^2u(x_1, x_2)) = u_{x_1x_1}u_{x_2x_2} - u_{x_1x_2}^2 = \frac{\rho_0(x_1, x_2)}{\rho_1(u_{x_1}, u_{x_2})} \iff u''_1(x_1)u''_2(x_2) = \frac{R_1(x_1)R_2(x_2)}{T_1(u'_1(x_1))T_2(u'_2(x_2))},$$

which we can formally rewrite as

$$\frac{u''_1(x_1)S_1(u'_1(x_1))}{R_1(x_1)} = \frac{R_2(x_2)}{u''_2(x_2)S_2(u'_2(x_2))} = c \neq 0,$$

where c is some constant (this is because the left, respectively right, hand sides above is a function only of x_1 , respectively of x_2), and R_1, R_2, S_1, S_2 , are normalized as in Lemma 3.3.2. By selecting the constant $c = 1$, we get

$$u''_1(x_1) = \frac{R_1(x_1)}{S_1(u'_1(x_1))}, \quad u''_2(x_2) = \frac{R_2(x_2)}{S_2(u'_2(x_2))}.$$

As a result, we obtain the two 1-d Monge-Ampère equations:

$$\begin{aligned} S_1(u'_1(x_1))u''_1(x_1) &= R_1(x_1), & u'_1(a_0) &= \alpha_0, & u'_1(a_1) &= \alpha_1 \\ S_2(u'_2(x_2))u''_2(x_2) &= R_2(x_2), & u'_2(b_0) &= \beta_0, & u'_2(b_1) &= \beta_1. \end{aligned} \tag{3.20}$$

In conclusion, we can take the solution $u(x)$ of (3.19) given by the representation $u(x) = u_1(x_1) + u_2(x_2)$ with u_1 and u_2 satisfying (3.20). $\square$

Corollary 3.3.4. *Under the conditions of Theorem 3.3.3, the unique optimal map $T(x) = \nabla u(x)$ rewrites as*

$$T(x) = \begin{bmatrix} T_1(x_1) \\ T_2(x_2) \end{bmatrix} = \begin{bmatrix} u_1'(x_1) \\ u_2'(x_2) \end{bmatrix}, \quad (3.21)$$

where u_1 and u_2 solve (3.20). □

We reiterate that when the domains of ρ_0 and ρ_1 are rectangles, and the probability densities form a separable pair, it is possible to simplify the task of solving the original 2-d Monge-Ampère problem by solving two 1-d problems.

3.3.2.1 Well-posedness

Consider the two-point boundary value problem:

$$\begin{cases} \rho_1(y)y' = \rho_0(x), \\ y(a) = \alpha, \quad y(b) = \beta, \end{cases} \quad (3.22)$$

where

$$\int_a^b \rho_0(x)dx = \int_\alpha^\beta \rho_1(y)dy = 1, \\ \rho_0(x) > 0, \quad \forall x \in [a, b], \quad \rho_1(y) > 0, \quad \forall y \in [\alpha, \beta].$$

Claim 3.3.5. *The boundary value problem (3.22) is well-posed in the sense of Hadamard.*

Proof. The existence and uniqueness of the solution is a consequence of results for the Monge-Ampère equation. We observe that the solution y of (3.22) can be expressed implicitly using the cumulative distribution functions $R_0(x) = \int_a^x \rho_0(s)ds$ and $R_1(y) = \int_\alpha^y \rho_1(z)dz$, that is from the relation

$$R_1(y) = R_0(x), \quad (3.23)$$

where $R_0(a) = 0$, $R_0(b) = 1$, $R_1(\alpha) = 0$, and $R_1(\beta) = 1$.

We show the continuous dependence on the data, therefore consider three possible perturbations, in α , in β and in ρ_0 and ρ_1 .

- Perturbation in α . Consider

$$\begin{aligned}\rho_1(\tilde{y})\tilde{y}' &= \rho_0(x), \\ \tilde{y}(a) &= \alpha + \varepsilon, \quad \tilde{y}(b) = \tilde{\beta},\end{aligned}\tag{3.24}$$

where $|\varepsilon|$ is small, and $\int_{\alpha+\varepsilon}^{\tilde{\beta}} \rho_1(z)dz = 1$ holds. Then, the solution of (3.24) satisfies:

$$\int_{\alpha+\varepsilon}^{\tilde{y}} \rho_1(z)dz = \int_a^x \rho_0(s)ds.\tag{3.25}$$

From (3.23): $\int_{\alpha}^y \rho_1(z)dz = \int_a^x \rho_0(s)ds$. As a result, it follows that:

$$\begin{aligned}0 &= \int_{\alpha+\varepsilon}^{\tilde{y}} \rho_1(z)dz - \int_{\alpha}^y \rho_1(z)dz = \int_{\alpha}^{\tilde{y}} \rho_1(z)dz - \int_{\alpha}^{\alpha+\varepsilon} \rho_1(z)dz - \int_{\alpha}^y \rho_1(z)dz, \\ 0 &= \int_y^{\tilde{y}} \rho_1(z)dz - \int_{\alpha}^{\alpha+\varepsilon} \rho_1(z)dz \implies \int_y^{\tilde{y}} \rho_1(z)dz = \int_{\alpha}^{\alpha+\varepsilon} \rho_1(z)dz, \\ \left| \int_y^{\tilde{y}} \rho_1(z)dz \right| &\leq \int_{\min(y,\tilde{y})}^{\max(y,\tilde{y})} \rho_1(z)dz \leq |\varepsilon| \max_z |\rho_1(z)| \implies |y - \tilde{y}| \leq |\varepsilon| \frac{\max_z |\rho_1(z)|}{\min_z |\rho_1(z)|}.\end{aligned}$$

- Perturbation in β . This proceeds similarly to the above. Considering

$$\begin{aligned}\rho_1(\tilde{y})\tilde{y}' &= \rho_0(x), \\ \tilde{y}(a) &= \tilde{\alpha}, \quad \tilde{y}(b) = \beta + \varepsilon,\end{aligned}\tag{3.26}$$

where $|\varepsilon|$ is small and $\int_{\tilde{\alpha}}^{\beta+\varepsilon} \rho_1(z)dz = 1$ holds, one now obtains

$$|y - \tilde{y}| \leq |\varepsilon| \frac{\max_z |\rho_1(z)|}{\min_z |\rho_1(z)|}.$$

- Perturbation of $\rho_0(x)$. Now we need to consider small zero average perturbations of ρ_0 .

That is, we consider

$$\begin{aligned}\rho_1(\tilde{y})\tilde{y}' &= \rho_0(x) + \varepsilon(x), \\ \tilde{y}(a) &= \alpha, \quad \tilde{y}(b) = \beta,\end{aligned}\tag{3.27}$$

where $\int_a^b \varepsilon(s)ds = 0$ and $|\varepsilon(x)|$ is small for all x . The solution of (3.27) satisfies:

$$\int_{\alpha}^{\tilde{y}} \rho_1(z)dz = \int_a^x \rho_0(s)ds + \int_a^x \varepsilon(s)ds.\tag{3.28}$$

From (3.23): $\int_{\alpha}^y \rho_1(z)dz = \int_a^x \rho_0(s)ds$. As a result, it follows that

$$\begin{aligned}\int_{\alpha}^{\tilde{y}} \rho_1(z)dz - \int_{\alpha}^y \rho_1(z)dz &= \int_a^x \varepsilon(s)ds \implies \int_y^{\tilde{y}} \rho_1(z)dz = \int_a^x \varepsilon(s)ds, \\ \left| \int_y^{\tilde{y}} \rho_1(z)dz \right| &= \rho_1(\hat{z})|y(x) - \tilde{y}(x)| \implies |y(x) - \tilde{y}(x)| \leq |x - a| \frac{\max_s |\varepsilon(s)|}{\rho_1(\hat{z})}.\end{aligned}$$

where $\min(y, \tilde{y}) \leq \hat{z} \leq \max(y, \tilde{y})$.

• **Perturbation of $\rho_1(\tilde{y})$.** This is similar to the previous case. We consider small zero average perturbations of ρ_1 :

$$\begin{aligned}[\rho_1(\tilde{y}) + \varepsilon(\tilde{y})]\tilde{y}' &= \rho_0(x), \\ \tilde{y}(a) &= \alpha, \quad \tilde{y}(b) = \beta,\end{aligned}\tag{3.29}$$

where $\int_{\alpha}^{\beta} \varepsilon(s)ds = 0$ and $|\varepsilon(\tilde{y})|$ is small for all $\tilde{y}$. The solution of (3.29) satisfies:

$$\int_{\alpha}^{\tilde{y}} \rho_1(z)dz + \int_{\alpha}^{\tilde{y}} \varepsilon(z)dz = \int_a^x \rho_0(s)ds.\tag{3.30}$$

From (3.23): $\int_{\alpha}^y \rho_1(z)dz = \int_a^x \rho_0(s)ds$. As a result, it follows that

$$\begin{aligned} \int_{\alpha}^y \rho_1(z)dz - \int_{\alpha}^{\tilde{y}} \rho_1(z)dz &= \int_{\alpha}^{\tilde{y}} \varepsilon(z)dz \implies \int_{\tilde{y}}^y \rho_1(z)dz = \int_{\alpha}^{\tilde{y}} \varepsilon(z)dz, \\ \left| \int_y^{\tilde{y}} \rho_1(z)dz \right| &= \rho_1(\hat{z})|y(x) - \tilde{y}(x)| \implies |y(x) - \tilde{y}(x)| \leq |\beta - \alpha| \frac{\max_s |\varepsilon(s)|}{\rho_1(\hat{z})}, \end{aligned}$$

where $\min(y, \tilde{y}) \leq \hat{z} \leq \max(y, \tilde{y})$. □

As a result, (3.20) is well posed in the sense of Hadamard. This imply that it must be possible to devise robust numerical methods for its solution, and hence for solving the Monge-Ampère equation (3.19) relative to separable densities.

3.3.3 Separable Densities with Benamou-Brenier Formulation

Here we discuss the impact of having a separable density pair with respect to the Benamou-Brenier formulation (3.1).

Definition 3.3.6. *We say that the velocity field $v(t, x)$ in (3.1) is separable if it has the form $v(t, x) = \begin{bmatrix} v_1(t, x_1) \\ v_2(t, x_2) \end{bmatrix}$ for all (t, x_1, x_2) . Similarly, we say that the density $\rho(t, x)$ is separable if it can be written as product of 1-d densities: $\rho(t, x) = R_1(t, x_1)R_2(t, x_2)$.*

Lemma 3.3.7. *Let ρ_0 and ρ_1 be separable densities as in Definition 3.3.1. Then, the velocity field at $t = 0$, $v^0(x) = v(0, x)$, is separable.*

Proof. This is a consequence of the relation between the optimal map $T(x_1, x_2)$ and the velocity field $v(x_1, x_2, t)$:

$$T(x_1, x_2) = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + v(x_1, x_2, 0) = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{bmatrix} v_1(0, x_1, x_2) \\ v_2(0, x_1, x_2) \end{bmatrix}.$$

Therefore, as a consequence of Corollary 3.3.4, the initial velocity field is separable:

$$v(0, x_1, x_2) = \begin{bmatrix} v_1^0(x_1) \\ v_2^0(x_2) \end{bmatrix}.$$

□

Theorem 3.3.8. *Let ρ_0 and ρ_1 be separable densities as in Definition 3.3.1. Consider the formulation (3.6) of the OT problem. Then, the velocity field is separable for all $t \in [0, 1]$:*

$$v(t, x) = \begin{bmatrix} v_1(t, x_1) \\ v_2(t, x_2) \end{bmatrix}.$$

Proof. Using the Benamou-Brenier formulation (3.6) the velocity field can be found by solving the PDE

$$\frac{\partial v(t, x)}{\partial t} + \nabla \left(\frac{1}{2} \|v(t, x)\|^2 \right) = 0. \quad (3.31)$$

Writing

$$v(x_1, x_2, t) = \begin{pmatrix} v_1(t, x_1, x_2) \\ v_2(t, x_1, x_2) \end{pmatrix},$$

(3.31) can be explicitly rewritten as

$$\begin{pmatrix} \frac{\partial v_1(t, x_1, x_2)}{\partial t} \\ \frac{\partial v_2(t, x_1, x_2)}{\partial t} \end{pmatrix} + \begin{pmatrix} \frac{\partial v_1(t, x_1, x_2)}{\partial x_1} v_1(t, x_1, x_2) + \frac{\partial v_2(t, x_1, x_2)}{\partial x_1} v_2(t, x_1, x_2) \\ \frac{\partial v_1(t, x_1, x_2)}{\partial x_2} v_1(t, x_1, x_2) + \frac{\partial v_2(t, x_1, x_2)}{\partial x_2} v_2(t, x_1, x_2) \end{pmatrix} = 0.$$

Now, from Lemma 3.3.7 we know that the velocity field is separable at $t = 0$: $v(0, x_1, x_2) =$

$\begin{bmatrix} v_1^0(x_1) \\ v_2^0(x_2) \end{bmatrix}$. But then, the evolution in time of v will preserve this property. This is a consequence of the fact that the derivative at $t = 0$ of $v(t, x)$ is separable. In fact, from (3.31)

one has that

$$\left[\frac{\partial}{\partial t} v(t, x_1, x_2) \right]_{t=0} = - \begin{pmatrix} \frac{\partial v_1(0, x_1, x_2)}{\partial x_1} v_1(0, x_1, x_2) + \frac{\partial v_2(0, x_1, x_2)}{\partial x_1} v_2(0, x_1, x_2) \\ \frac{\partial v_1(0, x_1, x_2)}{\partial x_2} v_1(0, x_1, x_2) + \frac{\partial v_2(0, x_1, x_2)}{\partial x_2} v_2(0, x_1, x_2) \end{pmatrix},$$

that is, using Lemma Lemma 3.3.7:

$$\left[\frac{\partial}{\partial t} v(t, x_1, x_2) \right]_{t=0} = - \begin{pmatrix} (\partial_{x_1} v_1^0(x_1)) v_1^0(x_1) \\ (\partial_{x_2} v_2^0(x_2)) v_2^0(x_2) \end{pmatrix},$$

and thus

$$\begin{pmatrix} \frac{\partial}{\partial t} v_1(t, x_1) \\ \frac{\partial}{\partial t} v_2(t, x_2) \end{pmatrix} = - \begin{pmatrix} (\partial_{x_1} v_1(t, x_1)) v_1(t, x_1) \\ (\partial_{x_2} v_2(t, x_2)) v_2(t, x_2) \end{pmatrix}. \quad (3.32)$$

□

Remark 3.3.9. [Is $\rho(t, x)$ separable?] *Under the same conditions of Theorem 3.3.8, it is unclear whether the probability density $\rho(t, x_1, x_2)$ is separable for all t , even when at times $t = 0$ and $t = 1$ one has a separable pair. That is, if $\rho(0, x_1, x_2) = R_1(x_1)R_2(x_2)$ and $\rho(1, x_1, x_2) = S_1(x_1)S_2(x_2)$, can we conclude that $\rho(t, x_1, x_2) = R_1(t, x_1)R_2(t, x_2)$ for all $t \in (0, 1)$? In general, this is not clear to us. However, we point out that in order for it to be true then one would need the following to hold (using Theorem 3.3.8)*

$$\begin{aligned} \frac{\partial}{\partial t} R_1(t, x_1) + \frac{\partial}{\partial x} (v_1(t, x_1) R_1(t, x_1)) &= 0, \\ \frac{\partial}{\partial t} R_2(t, x_2) + \frac{\partial}{\partial x} (v_2(t, x_2) R_2(t, x_2)) &= 0, \end{aligned} \quad (3.33)$$

along with the velocity equations from (3.32). Below, we give some evidence that this fact does not hold, by comparing the solutions obtained by assuming that ρ is separable, and using (3.33), versus solving the problem for $\rho(t, x)$ without assuming separability of ρ explicitly. As it can be seen from Figure 3.2, it appears that the density $\rho(t, x)$ is not separable, in general.

3.3.3.1 Fluid-flow, optimal map, separability

In spite of Remark 3.3.9, it is still possible to take advantage of separability when using the fluid-flow formulation. In fact, from separability of the Monge-Ampère problem, we

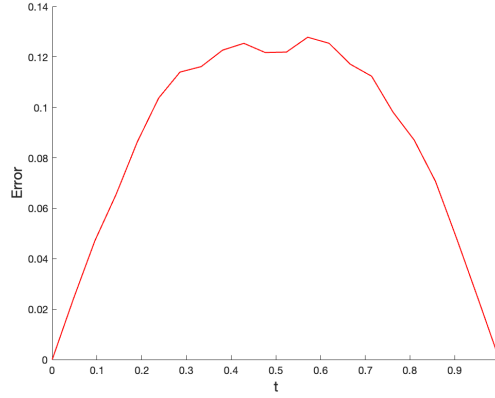


Figure 3.2: Checking separability of $\rho(t, x)$. Problem 3.4.8, multiple shooting method, $dx = \frac{1}{32}$.

know that the optimal $T(x)$ rewrites as $T(x) = \begin{bmatrix} T_1(x_1) \\ T_2(x_2) \end{bmatrix}$, where T_1 (respectively T_2) is the optimal map for the transport from the 1-d density R_1 to S_1 (respectively, R_2 to S_2). Then, one approach is to use the fluid-flow formulation (in either form (3.3) or (3.6)) to perform the 1-d transfers from R_1 to S_1 and from R_2 to S_2 to obtain the initial velocity $\begin{bmatrix} v_1(0, x_1) \\ v_2(0, x_2) \end{bmatrix}$. This can then be used to integrate directly for the density evolution, for example integrating (3.5), but also to approximate the optimal map (for as long as the conditions leading to the equality below hold)

$$T(x) = \begin{bmatrix} T_1(x_1) \\ T_2(x_2) \end{bmatrix} = \begin{bmatrix} x_1 + v_1(0, x_1) \\ x_2 + v_2(0, x_2) \end{bmatrix}.$$

We used this approach to approximate the optimal map on all separable 2-d problems in Section 3.4 when using the methods (3.4) or (3.6) based on the fluid-flow formulation, that is solving 1-d problems.

Finally, motivated by the previous discussion on separability, we conclude this Section with some results on the numerical discretization of the 1-d Monge Ampère PDE and on the impact of periodic boundary conditions, also in this 1-d case.

3.3.4 Numerical Solution of 1-d Monge Ampère

As we saw before, the Monge-Ampère equation in 1-d reduces to solving the two-point boundary value problem:

$$\begin{aligned}\rho_1(y(x))y'(x) &= \rho_0(x), \\ y'(a) &= \alpha, \quad y'(b) = \beta,\end{aligned}\tag{3.34}$$

where $y(x) = u'(x)$ and

$$\begin{aligned}\int_a^b \rho_0(x)dx &= \int_\alpha^\beta \rho_1(y)dy = 1, \\ \rho_0(x) &> 0, \forall x \in [a, b] \quad \text{and} \quad \rho_1(y) > 0, \quad \forall y \in [\alpha, \beta].\end{aligned}$$

Then one approach to solve this problem is to use the analytical form of the exact solution; see [80, Theorem 2.5], which requires computing cumulative distribution functions after performing separation of variables. A more practical way to solve (3.34), and the one we adopted in this work, is to discretize by a Runge-Kutta (RK) scheme the scalar ODE (3.34) with only the initial condition $y'(a) = \alpha$. This way we can recover the terminal condition $y'(b) = \beta$ to any desired accuracy. In our numerical experiments we have used the classical RK4 method to solve (3.34) as an initial value problem from a to b with a fixed stepsize $h = \frac{b-a}{N+1}$. Finally, since we are using RK4, we enforce the normalization of the values of $\rho_0(x_i)$ in the following way:

$$\begin{aligned}\text{let } \gamma &= \frac{h}{6} \sum_{i=0}^N [\rho_0(x_i) + 4\rho_0(x_i + h/2) + \rho_0(x_{i+1})], \\ \text{then renormalize } \rho_0^i &= \frac{1}{\gamma} \rho_0(x_i).\end{aligned}$$

With the above normalization, the final value y_{N+1} will be accurate to β at $\mathcal{O}(h^4)$.

3.3.5 Optimal map on 1-d torus

For one-dimensional MA problem on the torus, the correct formulation now seeks a convex function of the form $x^2/2 - u$ (see (3.8)), and the optimal map will be of the form

$$T(x) = x - u'(x),$$

where u satisfies

$$1 - u''(x) = \frac{\rho_0(x)}{\rho_1(x - u'(x))}, \quad u(0) = u(1), \quad (3.35)$$

for $x \in [0, 1], \text{ mod } 1$.

To solve (3.35), we exploit the fact that the solution is unique only up to a constant and so we fix the value $u(0) = 0$, and rewrite (3.35) as the second-order two-point boundary value problem:

$$\begin{cases} 1 - u''(x) = \frac{\rho_0(x)}{\rho_1(x - u'(x))}, \\ u(0) = 0, \quad u(1) = 0. \end{cases} \quad (3.36)$$

For solving (3.36) we used a single shooting method, by integrating the initial value problem with initial data $u(0) = 0$, $u'(0) = s$, and seeking (via Newton's method) the correct initial velocity $u'(0)$ so that at the end point we get $u(1) = 0$. This is a standard approach, and in our implementation we carried out the numerical integration of the initial value problem with RK4.

Remark 3.3.10. *A major simplification, allowing to bypass using single shooting and Newton's method, occurs when $\rho_1(y)$ is uniform (i.e. $\rho_1(y) = 1$). This is because (3.36) rewrites as a first-order scalar initial value problem by letting $y(x) = u'(x)$:*

$$1 - y'(x) = \rho_0(x), \quad y(0) = \int_0^1 R_0(s) ds - \frac{1}{2}, \quad (3.37)$$

where $R_0(x)$ is the cumulative distribution function of $\rho_0(x)$:

$$R_0(x) = \int_0^x \rho_0(s) ds.$$

It is simple to justify the initial condition for $y(x)$ in (3.37) as follows. Rewrite

$$\begin{aligned} u''(x) = 1 - \rho_0(x) &\implies u'(x) = x - \int_0^x \rho_0(s) ds + u'(0), \\ \implies u(x) &= \frac{1}{2}x^2 - \int_0^x R_0(s) ds + xu'(0) + u(0). \end{aligned}$$

So, enforcing periodicity condition $u(0) = u(1)$ gives $u'(0) = \int_0^1 R(s) ds - \frac{1}{2}$.

3.4 Computational Examples and Numerical Results

In this section we will provide the set of problems which we have used to test accuracy and overall performance of our implementation for the numerical methods previously presented. Whenever we have an expression for the optimal map, we will report on the error in approximating it. Likewise, we will report on the value we obtain for the approximation of the Wasserstein distance $g_W(\rho_0, \rho_1)$, by using the three different approaches we implemented: when solving the Monge-Ampère problem, g_W is approximated by (3.16) as outlined in Remark 3.2.4, for the fluid-flow implementation of (3.4) it is approximated by the square root of the value in (3.4), and for the approach based on the geodesic we take the square root of the value in (3.7).

For all implementations requiring to solve a nonlinear system, we iterated until either the update in norm was less than a preassigned value `TOL`, or we exceeded `maxit` iterations.

3.4.1 Problems and Results in 1D

Below, $X = [a, b]$ and $Y = [\alpha, \beta]$ are the domains of the two densities $\rho_0(x)$ and $\rho_1(y)$.

For these problems, we either have the explicit formula for the optimal map, or we can obtain it implicitly as follows. Since $\rho_1(y)dy = \rho_0(x)dx$, then we get $\int_\alpha^y \rho_1(y)dy = \int_a^x \rho_0(x)dx$ or $R_1(y) = R_0(x)$. So, for any given x , we find $y(x)$ (the optimal map) as the solution of

$$R_1(y(x)) - R_0(x) = 0.$$

To find the root of this equation, we used the routine `fzero` of `Matlab`.

Example 3.4.1. *This is a problem used to compare the numerical solution of the 1-d MA with the solution of the Benamou-Brenier formulation (3.4). The formulation (3.5) does not apply here, since the implied BCs do not hold. The densities are linear, and the map can be found easily. We have*

$$X = Y = [0, 1], \quad \rho_0(x) = \frac{2x+1}{2}, \quad \rho_1(y) = \frac{3-2y}{2},$$

and the optimal transport map is

$$T(x) = \frac{1}{2}(3 - \sqrt{9 - 4x - 4x^2}).$$

As it can be seen from Table 3.3, solving the system (3.11), and exploiting the RK integration for the 1-d Monge-Ampère solution, we recover first and the fourth orders of convergence for the respective methods, as expected. In addition, the execution time of RK4 is very short. Multiple shooting method outputs small error too, which is comparable with Regularized Benamou-Brenier method's error. However, it does not show desired order of convergence. In fairness, Example 3.4.1 is not one where we expected the method based on (3.6) to work well, since the implied no-flux BCs do not hold.

Example 3.4.2. *This is a map from a Gaussian to a two-bumps Gaussian. We have*

$$X = Y = [0, 1], \quad \rho_0(x) = c_1 e^{-5(x-0.5)^2}, \quad \rho_1(y) = c_2 [e^{-50(y-0.25)^2} + e^{-50(y-0.75)^2}],$$

Table 3.3: Numerical Results for Example 3.4.1

dx	Regularized Benamou-Brenier $dt = \frac{1}{21}, \beta = 10^{-5}, \alpha = 0.5, \text{TOL} = 10^{-8}, \text{maxit} = 40$				Multiple Shooting $dt = \frac{1}{128}, \Delta t = \frac{dt}{32}, \text{TOL} = 10^{-12}, \text{maxit} = 20$				Explicit RK-4		
	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
$\frac{1}{16}$	$3.6022 * 10^{-2}$	$2.0241 * 10^{-1}$	4.28	26	$7.0847 * 10^{-2}$	$2.0644 * 10^{-1}$	2.36	5	$2.4340 * 10^{-5}$	$1.7951 * 10^{-1}$	$6.11 * 10^{-3}$
$\frac{1}{32}$	$2.0514 * 10^{-2}$	$1.9079 * 10^{-1}$	19.9	26	$5.7057 * 10^{-2}$	$1.9231 * 10^{-1}$	4.21	5	$2.0546 * 10^{-6}$	$1.7953 * 10^{-1}$	$1.69 * 10^{-3}$
$\frac{1}{64}$	$1.1055 * 10^{-2}$	$1.8512 * 10^{-1}$	117	26	$5.0202 * 10^{-2}$	$1.8529 * 10^{-1}$	14.1	5	$1.4717 * 10^{-7}$	$1.7953 * 10^{-1}$	$1.69 * 10^{-3}$
$\frac{1}{128}$	$5.7552 * 10^{-3}$	$1.8232 * 10^{-1}$	760	26	$4.6811 * 10^{-2}$	$1.8182 * 10^{-1}$	248	5	$9.7792 * 10^{-9}$	$1.7953 * 10^{-1}$	$9.05 * 10^{-4}$

where c_1 and c_2 are normalization constants.

Table 3.4 shows numerical results for this example. No method performs well for this problem, and the main culprit appears to be the fact that ρ_1 is near 0 at the endpoints.

Table 3.4: Numerical Results for Example 3.4.2

dx	Regularized Benamou-Brenier $dt = \frac{1}{21}, \beta = 10^{-5}, \alpha = 0.5, \text{TOL} = 10^{-8}, \text{maxit} = 40$				Multiple Shooting $dt = \frac{1}{128}, \Delta t = \frac{dt}{32}, \text{TOL} = 10^{-12}, \text{maxit} = 20$				Explicit RK-4		
	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
$\frac{1}{16}$	$2.4776 * 10^{-2}$	$6.3886 * 10^{-2}$	6.61	26	$3.1463 * 10^{-2}$	$6.6372 * 10^{-2}$	2.51	5	$1.7424 * 10^2$	20.261	$3.40 * 10^{-2}$
$\frac{1}{32}$	$2.1508 * 10^{-2}$	$6.2688 * 10^{-2}$	27.6	27	$3.4132 * 10^{-2}$	$6.5357 * 10^{-2}$	4.28	5	$6.8561 * 10^1$	5.4739	$2.87 * 10^{-2}$
$\frac{1}{64}$	$1.5208 * 10^{-2}$	$6.2503 * 10^{-2}$	143	27	$3.1703 * 10^{-2}$	$6.5075 * 10^{-2}$	16.5	6	$1.5420 * 10^{-3}$	$6.2603 * 10^{-2}$	$6.00 * 10^{-2}$
$\frac{1}{128}$	$1.1248 * 10^{-2}$	$6.2494 * 10^{-2}$	939	27	$3.2718 * 10^{-2}$	$6.5012 * 10^{-2}$	308	6	$9.5382 * 10^{-4}$	$6.2678 * 10^{-2}$	$1.02 * 10^{-3}$

Example 3.4.3. Here we have nonuniform periodic densities

$$X = Y = [0, 1] \mod 1, \quad \rho_0(x) = 1 - \frac{1}{32} \cos(2\pi(x + 0.5)), \quad \rho_1(y) = 1 - \frac{1}{32} \cos(2\pi y).$$

As it can be seen from Table 3.5, all three methods exhibit expected order of convergence for the map. However, show some instability at $x = 1$ for the density evolution, see Figure 3.4. This is because the Regularized Benamou-Brenier method assumes zero flux condition at the boundaries, which is violated for periodic problems. For the discretization of the geodesic equation, we used the scheme in Section 3.2.2.3, and single shooting proved adequate on this problem.

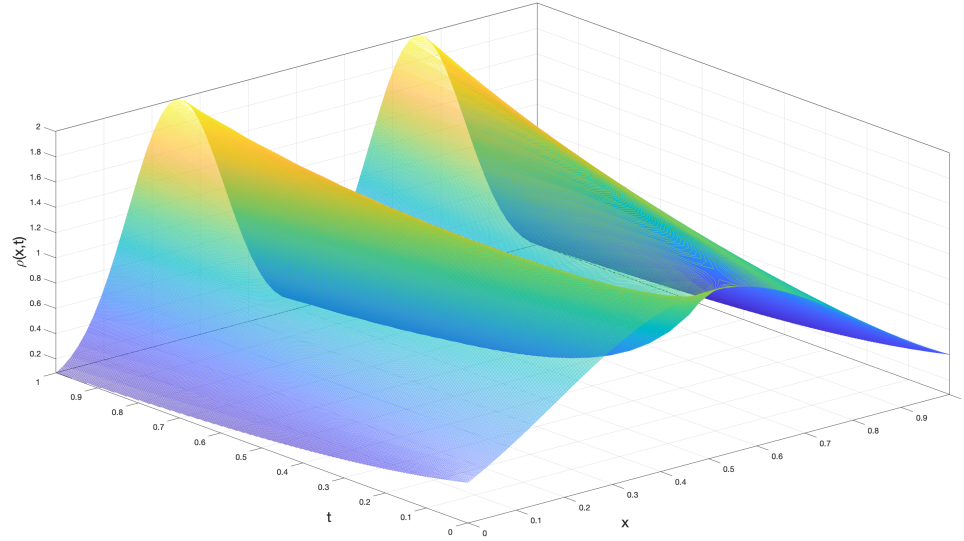


Figure 3.3: Evolution of probability density in Example 3.4.2 with $dx = \frac{1}{64}$, solved with Regularized Benamou-Brenier method

Table 3.5: Numerical Results for Example 3.4.3

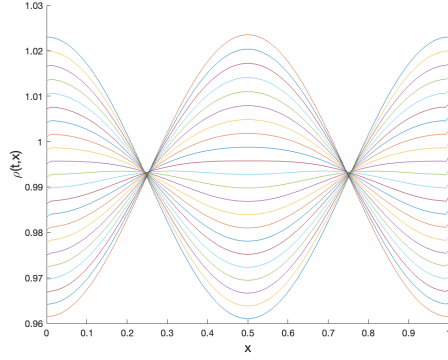
	Regularized Benamou-Brenier				Multiple Shooting				Shooting Method with Explicit RK-4		
	$dt = \frac{1}{21}, \beta = 10^{-5}, \alpha = 0.5, \text{TOL} = 10^{-8}, \text{maxit} = 40$				$dt = \frac{1}{128}, \Delta t = dt, \text{TOL} = 10^{-5}, \text{maxit} = 10$				$\text{TOL} = 10^{-12}$		
dx	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
$\frac{1}{16}$	$1.5127 * 10^{-3}$	$7.7011 * 10^{-3}$	3.96	19	FAIL (Singular Jacobian)				$9.7941 * 10^{-8}$	$7.0329 * 10^{-3}$	1.0595
$\frac{1}{32}$	$8.6542 * 10^{-4}$	$7.3647 * 10^{-3}$	15.6	19	1.4069	1.3346	$1.44 * 10^{-2}$	9	$6.0470 * 10^{-9}$	$7.0329 * 10^{-3}$	1.4939
$\frac{1}{64}$	$4.5942 * 10^{-4}$	$7.1982 * 10^{-3}$	93.5	19	$6.1075 * 10^{-2}$	$5.9121 * 10^{-2}$	$2.40 * 10^{-2}$	4	$3.7646 * 10^{-10}$	$7.0329 * 10^{-3}$	1.9254
$\frac{1}{128}$	$2.3887 * 10^{-4}$	$7.1154 * 10^{-3}$	559	19	$3.4884 * 10^{-3}$	$7.7983 * 10^{-3}$	$8.18 * 10^{-2}$	4	$2.4216 * 10^{-11}$	$7.0329 * 10^{-3}$	1.6242

Example 3.4.4. *This is Example 4.4 of [29]. We have*

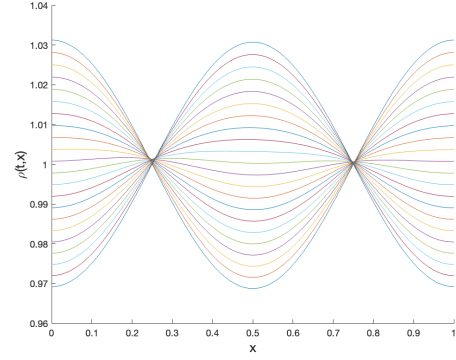
$$X = Y = [0, 2], \quad \rho_0(x) = \frac{1}{2}, \quad \rho_1(y) = ce^{-25(y-1)^2},$$

where c is the normalization constant.

Table 3.6 shows numerical results for this Example 3.4.4. Similar to Example 3.4.2, the explicit RK4 method to solve the Monge-Ampère problem initially gives large error but it eventually shows a fourth order decrease in the error as dx gets smaller. Neither of the Benamou-Brenier methods converges.



(a) Regularized Benamou-Brenier Method



(b) Multiple Shooting Method

Figure 3.4: Evolution of probability density in Example 3.4.3 with $dx = \frac{1}{128}$

Table 3.6: Numerical Results for Example 3.4.4

	Regularized Benamou-Brenier				Multiple Shooting				Explicit RK-4		
	$dt = \frac{1}{21}, \beta = 10^{-5}, \alpha = 0.5, \text{TOL} = 10^{-8}, \text{maxit} = 40$				$dt = \frac{1}{60}, \Delta t = \frac{dt}{20}, \text{TOL} = 10^{-12}, \text{maxit} = 20$						
dx	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
0.1	$4.2075 * 10^{-1}$	$1.6099 * 10^{-1}$	11.3	30	$3.1705 * 10^{-1}$	$5.9022 * 10^{-1}$	3.74	8	$9.8686 * 10^1$	$5.6616 * 10^1$	$5.8265 * 10^{-2}$
0.05	$4.8934 * 10^{-1}$	$1.5549 * 10^{-1}$	52.7	30	$4.3462 * 10^{-1}$	$5.9402 * 10^{-1}$	2.76	8	$9.7865 * 10^1$	$5.5880 * 10^1$	$4.5646 * 10^{-2}$
0.025	$5.6196 * 10^{-1}$	$1.5288 * 10^{-1}$	318	31	$5.7931 * 10^{-1}$	$5.9005 * 10^{-1}$	7.84	9	$2.7722 * 10^{-2}$	$4.3975 * 10^{-1}$	$8.4420 * 10^{-2}$
0.0125	$6.1689 * 10^{-1}$	$1.5163 * 10^{-1}$	2052	30	$6.4925 * 10^{-1}$	$5.8709 * 10^{-1}$	33.4	11	$2.4734 * 10^{-2}$	$4.4027 * 10^{-1}$	$1.6293 * 10^{-1}$
0.00625	FAIL (Out of Time)				FAIL (Singular Jacobian)				$1.2359 * 10^{-3}$	$4.4021 * 10^{-1}$	$3.2514 * 10^{-1}$

3.4.2 Problems and Results for Separable 2D Examples

These are all separable 2-d problems. For all of these, we either have the exact optimal map, or can compute using the technique outlined at the beginning of Section 3.4.1.

Example 3.4.5. *This is similar to Example 3.4.1, and here the formulation (3.5) does not hold, since the implied BCs do not hold. We have*

$$X = Y = [0, 1] \times [-1, 1], \quad \rho_0(x) = \frac{1}{12}[(2x_1 + 1)(x_2 + 3)], \quad \rho_1(y) = \frac{1}{12}[(3 - 2y_1)(3 - y_2)],$$

and the optimal transport map is

$$T(x_1, x_2) = \begin{pmatrix} \frac{1}{2}(3 - \sqrt{9 - 4x_1 - 4x_1^2}) \\ 3 - \sqrt{11 - 6x_2 - x_2^2} \end{pmatrix}.$$

Table 3.7 shows numerical results for this example. Since Example 3.4.5 can be reformulated as two 1D problems, which are similar to Example 3.4.1, the Regularized Benamou-Brenier and RK4 algorithms give first and fourth orders of convergence for the optimal map, respectively. Surprisingly, we see decrease in error for the multiple shooting method, but not of the nominal order of 1.

Table 3.7: Numerical Results for Example 3.4.5

	Regularized Benamou-Brenier $dt = \frac{1}{21}, \beta = 10^{-5}, \alpha = 0.5, \text{TOL} = 10^{-8}, \text{maxit} = 40$				Multiple Shooting $dt = \frac{1}{128}, \Delta t = \frac{dt}{32}, \text{TOL} = 10^{-12}, \text{maxit} = 20$				Explicit RK-4		
dx	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
$\frac{1}{16}$	$4.7129 * 10^{-2}$	$5.1595 * 10^{-2}$	8.26	[26, 25]	$7.0847 * 10^{-2}$	$5.3654 * 10^{-2}$	6.58	[5, 5]	$2.4340 * 10^{-5}$	$4.3364 * 10^{-2}$	$5.01 * 10^{-3}$
$\frac{1}{32}$	$2.3748 * 10^{-2}$	$4.7455 * 10^{-2}$	34.8	[26, 25]	$5.7057 * 10^{-2}$	$4.8140 * 10^{-2}$	17.9	[5, 5]	$2.0546 * 10^{-6}$	$4.3368 * 10^{-2}$	$3.81 * 10^{-3}$
$\frac{1}{64}$	$1.1954 * 10^{-2}$	$4.5355 * 10^{-2}$	215	[26, 25]	$5.0202 * 10^{-2}$	$4.5506 * 10^{-2}$	273	[5, 5]	$1.4717 * 10^{-7}$	$4.3368 * 10^{-2}$	$1.49 * 10^{-2}$
$\frac{1}{128}$	$7.3160 * 10^{-3}$	$4.4313 * 10^{-2}$	1350	[26, 25]	FAIL (Out of Memory)				$9.7792 * 10^{-9}$	$4.3368 * 10^{-2}$	$5.88 * 10^{-3}$

Example 3.4.6. *This is an instance of a mapping of a Gaussian to uniform distribution. This example was also used in [59] and [29], but separability was not exploited in these works. We have*

$$X = Y = [0, 1]^2, \quad \rho_0(x) = \rho_0(x) = c_0 e^{-2[(x_1-0.25)^2 + (x_2-0.75)^2]}, \quad \rho_1(y_1, y_2) = 1,$$

where c_0 is the normalization constant so that $\int_X \rho_0 dx = 1$. The optimal transport map is

$$T(x_1, x_2) = \begin{pmatrix} c_1 \int_0^{x_1} e^{-2(s-0.25)^2} ds \\ c_2 \int_0^{x_2} e^{-2(s-0.75)^2} ds \end{pmatrix}, \quad c_1 = \frac{1}{\int_0^1 e^{-2(s-0.25)^2} ds}, \quad c_2 = \frac{1}{\int_0^1 e^{-2(s-0.75)^2} ds}.$$

As it can be seen from Table 3.8, in Example 3.4.6 all three numerical methods show decrease in optimal map's error as dx gets smaller. However, once again only Regularized Benamou-Brenier and Runge-Kutta methods show the order. Notice that Regularized Benamou-Brenier algorithm does not have linear decrease in the last row, which can be indicating that one needs to decrease dt and/or β values to get better convergence.

Table 3.8: Numerical Results for Example 3.4.6

	Regularized Benamou-Brenier				Multiple Shooting				Explicit RK-4		
	$dt = \frac{1}{21}, \beta = 10^{-5}, \alpha = 0.5, \text{TOL} = 10^{-8}, \text{maxit} = 40$				$dt = \frac{1}{128}, \Delta t = \frac{dt}{32}, \text{TOL} = 10^{-12}, \text{maxit} = 20$						
dx	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
$\frac{1}{16}$	$1.9393 * 10^{-2}$	$7.8880 * 10^{-3}$	9.31	[25, 25]	$3.7402 * 10^{-2}$	$8.3962 * 10^{-3}$	4.68	[5, 5]	$8.0319 * 10^{-8}$	$6.6608 * 10^{-3}$	$2.75 * 10^{-2}$
$\frac{1}{32}$	$9.5747 * 10^{-3}$	$7.2625 * 10^{-3}$	41.1	[25, 25]	$2.6983 * 10^{-2}$	$7.3999 * 10^{-3}$	8.82	[5, 5]	$5.0100 * 10^{-9}$	$6.6607 * 10^{-3}$	$2.48 * 10^{-2}$
$\frac{1}{64}$	$4.8743 * 10^{-3}$	$6.9575 * 10^{-3}$	237	[25, 25]	$2.2044 * 10^{-2}$	$6.9299 * 10^{-3}$	27.3	[5, 5]	$3.1351 * 10^{-10}$	$6.6607 * 10^{-3}$	$4.23 * 10^{-2}$
$\frac{1}{128}$	$3.1003 * 10^{-3}$	$6.8079 * 10^{-3}$	1501	[25, 25]	$1.9558 * 10^{-2}$	$6.7029 * 10^{-3}$	517	[5, 5]	$1.9593 * 10^{-11}$	$6.6607 * 10^{-3}$	$6.87 * 10^{-2}$

Example 3.4.7. *We have*

$$X = Y = \mathbb{T}^2, \quad \rho_0(x_1, x_2) = (1 - \frac{1}{32} \sin(2\pi x_1))(1 - \frac{1}{32} \cos(2\pi x_2)), \quad \rho_1(y_1, y_2) = 1,$$

where $\mathbb{T}^2 = [0, 1]^2$ is the 2-torus. The optimal transport map is:

$$T(x_1, x_2) = \begin{pmatrix} x_1 + \frac{1}{64\pi} \cos(2\pi x_1) \\ x_2 - \frac{1}{64\pi} \sin(2\pi x_2) \end{pmatrix}.$$

Table 3.9 shows numerical results for this example. For this periodic, and separable, problem, the discretization of Section 3.2.2.3 outperformed the Regularized Benamou-Brenier method, and single shooting to find the initial velocity was adequate. Nevertheless, the explicit RK4 method still gives the best result among all three methods.

Table 3.9: Numerical Results for Example 3.4.7

	Regularized Benamou-Brenier				Multiple Shooting				Explicit RK-4		
	$dt = \frac{1}{21}, \beta = 10^{-5}, \alpha = 0.5, \text{TOL} = 10^{-8}, \text{maxit} = 40$				$dt = \frac{1}{128}, \Delta t = dt, \text{TOL} = 10^{-5}, \text{maxit} = 50$						
dx	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
$\frac{1}{16}$	$9.7683 * 10^{-4}$	$1.1674 * 10^{-5}$	8.03	[22, 17]	FAIL (Singular Jacobian)				$8.2517 * 10^{-8}$	$1.2368 * 10^{-5}$	$1.89 * 10^{-2}$
$\frac{1}{32}$	$4.3948 * 10^{-4}$	$1.2001 * 10^{-5}$	34.9	[24, 17]	FAIL (Singular Jacobian)				$5.1396 * 10^{-9}$	$1.2368 * 10^{-5}$	$3.86 * 10^{-3}$
$\frac{1}{64}$	$2.3508 * 10^{-4}$	$1.2181 * 10^{-5}$	207	[26, 17]	$4.5351 * 10^{-6}$	$1.2184 * 10^{-5}$	0.13	[10, 10]	$3.2095 * 10^{-10}$	$1.2368 * 10^{-5}$	$3.24 * 10^{-3}$
$\frac{1}{128}$	$1.2172 * 10^{-4}$	$1.2270 * 10^{-5}$	1342	[28, 17]	$1.1279 * 10^{-6}$	$1.2274 * 10^{-5}$	0.17	[4, 4]	$2.0054 * 10^{-11}$	$1.2368 * 10^{-5}$	$2.69 * 10^{-3}$
$\frac{1}{256}$	FAIL (Out of Time)				$7.1512 * 10^{-7}$	$1.2320 * 10^{-5}$	0.45	[3, 3]	$1.2537 * 10^{-12}$	$1.2368 * 10^{-5}$	$5.05 * 10^{-3}$

Example 3.4.8. *This is a problem used in [8] and in [80]. We have*

$$X = Y = [0, 1]^2, \quad \rho_0(x) = ce^{-10[(x_1-0.5)^2+(x_2-0.5)^2]}, \quad \rho_1(y) = \rho_0(y_1 + 0.5 \text{ [mod 1]}, y_2 + 0.5 \text{ [mod 1]}),$$

where c is the normalization constant.

As Table 3.10 shows, only the explicit RK4 method shows convergence with desired order. Both Benamou-Brenier algorithms converge with small error but order is not there and give different values for the Wasserstein distance. Figure 3.5 shows density evolution for this example, which was constructed through hybrid technique and matches that reported in [8].

Table 3.10: Numerical Results for Example 3.4.8

dx	Regularized Benamou-Brenier				Multiple Shooting				Explicit RK-4		
	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time
$\frac{1}{16}$	$4.2148 * 10^{-2}$	$3.5447 * 10^{-2}$	11.5	[28, 18]	$1.0370 * 10^{-1}$	$3.3534 * 10^{-2}$	8.12	[6, 6]	$9.2316 * 10^{-2}$	$2.8575 * 10^{-2}$	$5.48 * 10^{-2}$
$\frac{1}{32}$	$2.9934 * 10^{-2}$	$3.1972 * 10^{-2}$	59.1	[27, 27]	$8.6082 * 10^{-2}$	$3.0213 * 10^{-2}$	10.7	[6, 6]	$1.7530 * 10^{-2}$	$2.9157 * 10^{-2}$	$5.88 * 10^{-2}$
$\frac{1}{64}$	$2.1585 * 10^{-2}$	$3.0434 * 10^{-2}$	327	[28, 28]	$7.7826 * 10^{-2}$	$2.8763 * 10^{-2}$	35.8	[7, 7]	$1.9884 * 10^{-3}$	$2.9259 * 10^{-2}$	$1.11 * 10^{-1}$
$\frac{1}{128}$	$2.1062 * 10^{-2}$	$2.9719 * 10^{-2}$	2154	[28, 28]	$7.4088 * 10^{-2}$	$2.8092 * 10^{-2}$	607	[7, 7]	$1.5436 * 10^{-4}$	$2.9272 * 10^{-2}$	$2.15 * 10^{-1}$

3.4.3 Problems and Results for Non-Separable 2D Examples

Last, we consider two non-separable 2-d problems.

Example 3.4.9. *We have*

$$X = Y = [-0.5, 0.5]^2, \quad \rho_1(y) = 1,$$

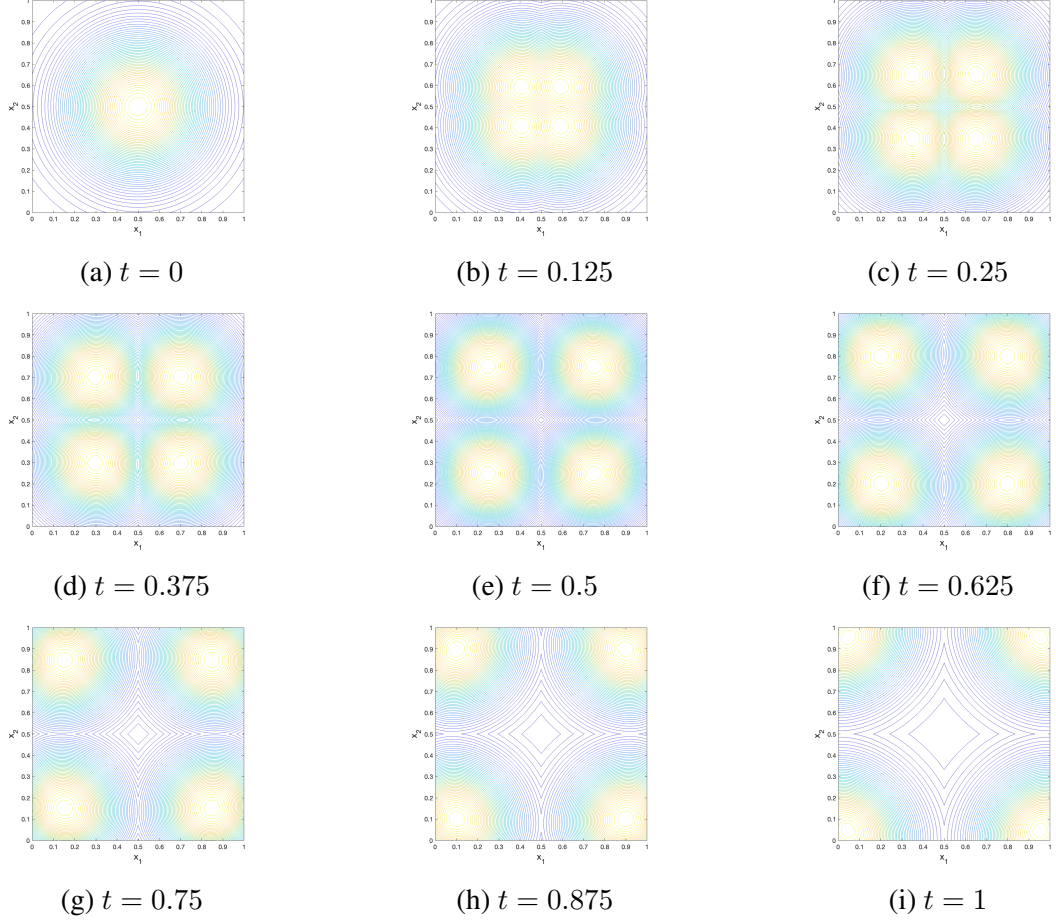


Figure 3.5: Evolution of probability density for Example 3.4.8, solved with the multiple shooting method, $dx = \frac{1}{64}$. Here, we first obtained the initial velocity $v(0, x)$ solving by multiple shooting two 1-d problems, and then integrated in time for ρ and v the system (3.5) using the time integrator from [29].

and

$$\begin{aligned} \rho_0(x) = & \left(a_1 + a_2 e^{0.5x_1^2} x_1 + 0.01\pi \sin(\pi x_1) \sin(\pi x_2) \right) \\ & \left(b_1 + b_2 e^{0.5x_2^2} x_2 + 0.01\pi \sin(\pi x_1) \sin(\pi x_2) \right) - 0.01^2 \pi^2 \cos^2(\pi x_1) \cos^2(\pi x_2), \\ a_0 = & -1; \quad a_1 = 1; \quad a_2 = -a_0 e^{-0.125}; \quad b_0 = -1; \quad b_1 = 1; \quad b_2 = -b_0 e^{-0.125}. \end{aligned}$$

The optimal transport map is:

$$T(x_1, x_2) = \begin{pmatrix} a_0 + a_1 x_1 + a_2 e^{0.5x_1^2} - 0.01 \cos(\pi x_1) \sin(\pi x_2) \\ b_0 + b_1 x_2 + b_2 e^{0.5x_2^2} - 0.01 \sin(\pi x_1) \cos(\pi x_2) \end{pmatrix}.$$

Table 3.11 shows numerical results for Example 3.4.9. The Monge-Ampere discretization of Section 3.2.2.1 shows linear convergence to the exact map, as expected. Multiple shooting method has decrease in error too but order is not linear. Regularized Benamou-Brenier method converges, but the error is large and not decreasing as dx is halved. In addition, increase in computation time is significant compared to the 1D problems. As a result, evaluations for smaller dx values did not succeed. Figure 3.6 illustrates density evolution for this example with $dx = \frac{1}{32}$, when solved with multiple shooting method.

Table 3.11: Numerical Results for Example 3.4.9

dx	Regularized Benamou-Brenier $dt = \frac{1}{21}, \beta = 10^{-3}, \alpha = 0.3, \text{TOL} = 10^{-6}, \text{maxit} = 40$				Multiple Shooting $dt = \frac{1}{21}, \Delta t = \frac{dt}{32}, \text{TOL} = 10^{-5}, \text{maxit} = 20$				Monge-Ampere (Identity) $\text{TOL} = 10^{-8}, \text{maxit} = 20$			
	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton	Map Error	$g_W(\rho_0, \rho_1)$	Time	Newton
$\frac{1}{8}$	0.12680	$2.4279 * 10^{-3}$	268	11	$5.4786 * 10^{-2}$	$1.5900 * 10^{-1}$	1.74	4	$5.1241 * 10^{-2}$	$1.5014 * 10^{-1}$	0.01	6
$\frac{1}{16}$	0.12823	$1.2956 * 10^{-3}$	4526	4	$3.4618 * 10^{-2}$	$1.3876 * 10^{-1}$	18.9	4	$2.3174 * 10^{-2}$	$1.3623 * 10^{-1}$	0.02	6
$\frac{1}{32}$	FAIL (Out of Time)				$2.4568 * 10^{-2}$	$1.2859 * 10^{-1}$	631	4	$1.1108 * 10^{-2}$	$1.3017 * 10^{-1}$	0.19	8
$\frac{1}{64}$	FAIL (Out of Time)				FAIL (Out of Memory)				$5.4616 * 10^{-3}$	$1.2731 * 10^{-1}$	2.78	9
$\frac{1}{128}$	FAIL (Out of Time)				FAIL (Out of Memory)				$2.7276 * 10^{-3}$	$1.2593 * 10^{-1}$	78.3	17

Example 3.4.10. Here we consider mapping a Gaussian to a two-bumps Gaussian. We have

$$X = Y = [0, 1]^2, \quad \rho_0(x_1, x_2) = c_0 e^{-5[(x_1-0.5)^2 + (x_2-0.5)^2]},$$

and

$$\rho_1(y_1, y_2) = c_1 [e^{-20[(y_1-0.25)^2 + (y_2-0.75)^2]} + e^{-20[(y_1-0.75)^2 + (y_2-0.25)^2]}],$$

where c_0, c_1 are normalization constants. Here, we do not have an expression for the optimal map.

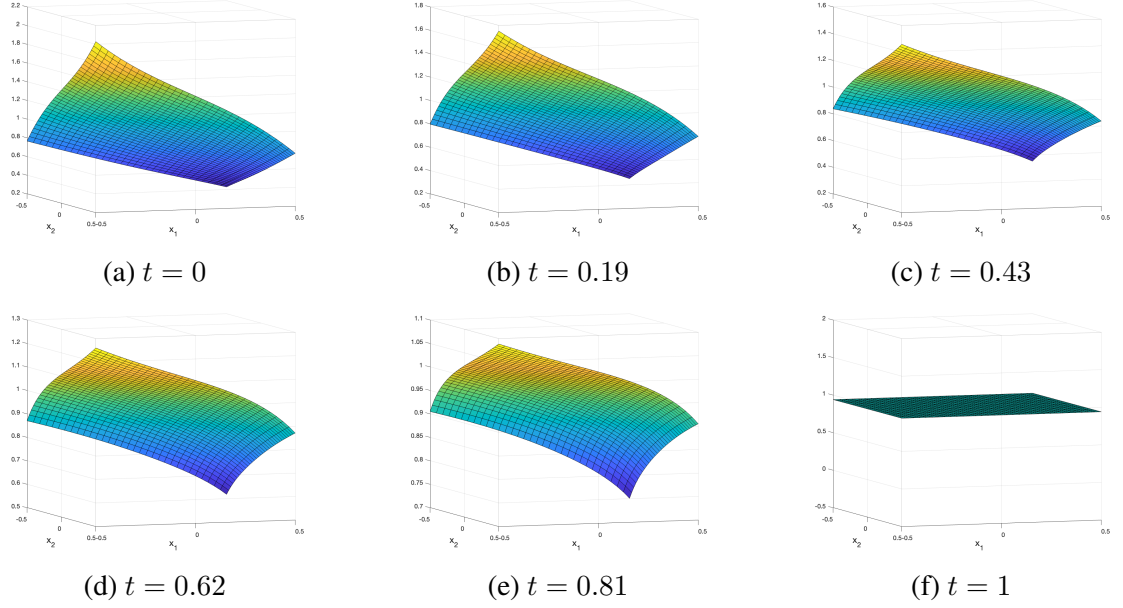
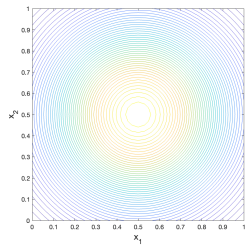


Figure 3.6: Evolution of probability density in Example 3.4.9 with $dx = \frac{1}{32}$, solved with multiple shooting method.

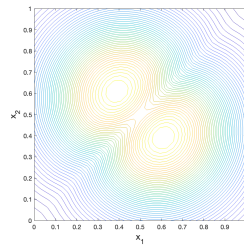
Table 3.12 gives the results for this 2D example, showing that the Regularized BB method is very costly to say the least. Figure 3.7 shows density evolution for this example when solved by multiple shooting method.

Table 3.12: Numerical Results for Example 3.4.10

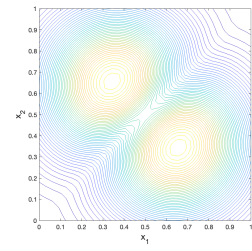
	Regularized Benamou-Brenier			Multiple Shooting			Monge-Ampere (Identity)		
	$dt = \frac{1}{21}, \beta = 10^{-3}, \alpha = 0.3, \text{TOL} = 10^{-6}, \text{maxit} = 40$			$dt = \frac{1}{21}, \Delta t = \frac{dt}{32}, \text{TOL} = 10^{-5}, \text{maxit} = 20$			$\text{TOL} = 10^{-8}, \text{maxit} = 20$		
dx	$g_W(\rho_0, \rho_1)$	Time	Newton	$g_W(\rho_0, \rho_1)$	Time	Newton	$g_W(\rho_0, \rho_1)$	Time	Newton
$\frac{1}{8}$	$2.4279 * 10^{-2}$	420	16	$1.8587 * 10^{-1}$	1.96	5	$8.5576 * 10^{-2}$	0.23	7
$\frac{1}{16}$	$3.6912 * 10^{-2}$	10956	9	$1.8737 * 10^{-1}$	24.6	6	$7.9610 * 10^{-2}$	0.50	6
$\frac{1}{32}$	FAIL (Out of Time)			$1.8717 * 10^{-1}$	928	7	$7.6449 * 10^{-2}$	1.91	6
$\frac{1}{64}$	FAIL (Out of Time)			FAIL (Out of Memory)			$7.4841 * 10^{-2}$	9.17	8
$\frac{1}{128}$	FAIL (Out of Time)			FAIL (Out of Memory)			$7.4034 * 10^{-2}$	81.9	12



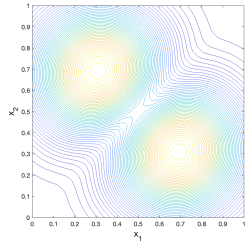
(a) $t = 0$



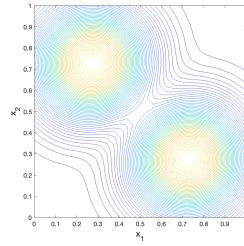
(b) $t = 0.24$



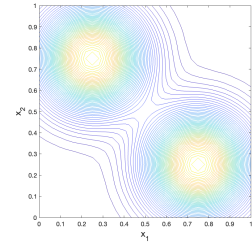
(c) $t = 0.43$



(d) $t = 0.62$



(e) $t = 0.86$



(f) $t = 1$

Figure 3.7: Evolution of probability density in Example 3.4.10 with $dx = \frac{1}{32}$, solved with multiple shooting method.

CHAPTER 4

SEMI-DISCRETE OT PROBLEMS

4.1 Formulation

Recall the general MK problem consists in determining the *optimal transport plan*, that is the joint probability density function $\pi \in \mathcal{P}(X \times Y)$, with marginals μ and ν , which realizes the inf of the functional $J(\pi)$ in (1.3). As it was mentioned in the introduction, one can obtain the dual formulation of the MK problem in a continuous setting as follows:

$$\begin{aligned} \sup_{\phi, \psi} D &:= \int_X \phi d\mu + \int_Y \psi d\nu, \\ \text{s. t. } \phi(x) + \psi(y) &\leq c(x, y), \quad \forall (x, y) \in X \times Y. \end{aligned} \tag{4.1}$$

For completeness, we will repeat results from [58] but for continuous setting, as it was done for the linear assignment problem (see Section 2.2).

Lemma 4.1.1 (Observation 4.3.2 in [58]). *Let $\psi^c(x) = \inf_{y \in Y} [c(x, y) - \psi(y)]$. If the pair (ϕ, ψ) is admissible in (4.1), then the pair (ψ^c, ψ) is admissible as well.*

Proof.

$$\psi^c(x) + \psi(y) = \inf_{y' \in Y} [c(x, y') - \psi(y')] + \psi(y) \leq c(x, y) - \psi(y) + \psi(y) = c(x, y). \quad \square$$

Lemma 4.1.2 (Observation 4.3.3 in [58]). *If the pair (ϕ, ψ) is admissible for (4.1), then one obtains a better pair in terms of the objective function value by replacing ϕ with ψ^c .*

Proof.

$$\left\{ \begin{array}{l} \psi^c(x) = \inf_{y \in Y} [c(x, y) - \psi(y)] \\ \phi(x) \leq c(x, y) - \psi(y), \quad \forall y \in Y \end{array} \right\} \implies \phi(x) \leq \psi^c(x). \quad \square$$

Using above lemmas, one can rewrite the dual formulation (4.1) as follows:

$$\sup_{\psi} D := \int_X \inf_{y \in Y} [c(x, y) - \psi(y)] d\mu + \int_Y \psi d\nu \quad (4.2)$$

Next let ν be a **discrete** probability measure on the set $Y = \{y_i\}_{i=1}^N$: $\nu(y) = \nu_i \delta(y_i)$, $i = 1, \dots, N$, $\nu_i > 0$, and $\sum_{i=1}^N \nu_i = 1$. Then (4.2) can be rewritten as:

$$\begin{aligned} D &= \int_X \inf_{y \in Y} [c(x, y) - \psi(y)] d\mu + \int_Y \psi d\nu, \\ &= \int_X \inf_{y_i \in Y} [c(x, y_i) - \psi_i] d\mu + \sum_{i=1}^N \psi_i \nu_i, \\ &= \sum_{i=1}^N \int_{A(y_i)} [c(x, y_i) - \psi_i] d\mu + \sum_{i=1}^N \psi_i \nu_i, \end{aligned}$$

where

$$A(y_i) = \{x \in X \mid c(x, y_i) - \psi_i \leq c(x, y_j) - \psi_j, \ j \neq i\}, \ i = 1, 2, \dots, N.$$

To be consistent with the standard semi-discrete OT problem notation let $X = \Omega$ and $w_i = \psi_i$. Then the minimization problem for the semi-discrete OT problem has the following form:

$$\min_{w \in \mathbb{R}^N} \Phi(w) := - \sum_{i=1}^N \int_{A_i} [c(x, y_i) - w_i] d\mu - \sum_{i=1}^N w_i \nu_i, \quad (4.3)$$

where

$$A_i = A(y_i) = \{x \in \Omega \mid c(x, y_i) - w_i \leq c(x, y_j) - w_j, \ j \neq i\}, \ i = 1, 2, \dots, N.$$

In literature, vector w is called a shift (or weight) vector and partitions A_i of the domain Ω are known as Laguerre cells of Ω .

Once an optimal vector of weights in (4.3) is determined, and for as long as the cells' boundaries have μ -measure 0, it is a simple observation that there is an optimal map and

that it is piecewise constant, in that it maps every $x \in A_i$ to the same y_i , up to sets of μ -measure 0.

Given the relevance in applications, there are several works concerned with numerical solution of semi-discrete OT problems, the chief differences between them being the cost functions considered. By far, the most widely studied case is for the cost function given by the 2-norm square: $c(x, y) = \|x - y\|_2^2$. In this case, the Laguerre tessellation is also known as “*Power Diagram*”, see [3]. The beauty of this case is in the simplicity of the cells’ structure: they are polytopes, their boundaries are given by (hyper) planes. Unsurprisingly, very elegant and efficient tools of computational geometry are used to compute power diagrams, with remarkable success and stunning display of the resulting subdivision. A likely incomplete list of computational works which tackle semi-discrete optimal transport in the case of the 2-norm squared includes the works [3], [83], [30], [82], [74, Chapter 5], [57], [63], [79], [31], [52], [18]. Conceivably, the reason for the algorithmic successes in this case of the 2-norm squared as cost is given by the intimate relation of power diagrams and Voronoi tessellation. In fact, a most useful algorithmic result in this case allows to lift a power diagram to be the same as a Voronoi tessellation in one higher space dimension, see [57], and computation of Voronoi tessellation has reached remarkable maturity in computational geometry: in the plane, the classical sweeping algorithm of Fortune [39] is still very popular, though there are of course several more modern presentations, e.g., [35], and there are also widely available codes for commonly used computational environments, like the Matlab functions `voronoi` and `voronoin`.

The situation for other cost functions is much less developed, and it is our present concern. To be sure, there are important theoretical results for more general costs than the 2-norm squared, for example see [44]. However, computational techniques do not appear to be very well developed, probably because of the lack of simple geometrical shapes of the Laguerre cells. Notable exceptions are the work [32], that considered general p -norm costs ($p \neq 1, \infty$), and the work [47] that considered the case of $p = 2$, the standard

Euclidean norm. As in [32], we will also restrict to p -norms as costs ($1 < p < \infty$), but we will follow a very different algorithmic path from [32]. (Comparison with the approach of [47] is done in the main body of this paper; in particular, see Section 4.3.2). The key ideas of the “boundary method” of [32] were to track/refine only the boundary-zone of the Laguerre cells, and not their interiors, by a grid-subdivision/refinement algorithm, and to improve upon the weights w through an original adaptation of the auction algorithm. In the present work, our key insight is that –although the shape of the Laguerre cells is not as simple as having boundary given by intersection of hyper-planes– the Laguerre cells are star-shaped. This is the most we can say for general cost functions of the type we consider. Moreover, we will adopt a Newton’s method approach to refine the weights, and very accurate manifold techniques to track the boundaries between cells. As a result, in the end our method appears to be much more accurate than any other method we know of, but the price we pay is that we need the density ρ associated to the measure μ (i.e., $d\mu = \rho(x)dx$) to be a sufficiently smooth density. That being the case, we develop a fully adaptive algorithm with error control, and we show that it performs extremely well both in terms of speed and accuracy on a variety of problems. Our main contributions are the following: (i) new algorithms for Newton’s method, exploiting star-shapedness of the Laguerre cells, (ii) unified treatment of the problem, (iii) comparison with other techniques, (iv) a full set of replicable results, with accuracy assessment well beyond the graphical display of results witnessed in the literature (how can one distinguish two close figures with the naked eye?), and (v) a rigorous numerical analysis, including the introduction of a proposal for a condition number of semi-discrete OT problems.

4.2 Theoretical Results

In this section, we give results about the geometrical and analytical (measure theoretical) properties of the partition in Laguerre cells we are after. Some of these results have appeared before, often for more specialized cost functions than those we consider, and

appropriate references will be given.

Although Laguerre tessellations, and optimal transport maps, can be –and are– defined in the entire ambient space $\mathbb{R}^n$, in practice we will restrict to working on a bounded set Ω , that is henceforth chosen to satisfy the following characterization.

Assumption 4.2.1 (Bounded domain Ω). *Ω will always refer to a compact convex domain of $\mathbb{R}^n$, of nonzero n -dimensional Lebesgue measure, and with C^k piecewise smooth boundary, $k \geq 1$.*

For completeness, recall that a C^k piecewise smooth boundary of Ω means that

$$\partial\Omega = \bigcup_{i=1}^N \bar{\Gamma}_i,$$

and each Γ_i is a C^k $(n - 1)$ -dimensional surface. Moreover, two of the $\bar{\Gamma}_i$'s may intersect each other along a C^k $(n - 2)$ -dimensional surface, etc..

Remarks 4.2.2.

- (a) *The value of k in Assumption 4.2.1 should be at least the same value of k as we have for the function F in Lemma 4.2.22.*
- (b) *In practice, Ω is often given by a (hyper)-cube, which surely satisfies Assumption 4.2.1 with piecewise C^∞ boundary. To witness, the standard unit square S will be our prototypical choice in 2-d, and its boundary is clearly piecewise smooth, being the union of four closed intervals, each intersecting two other intervals at a vertex.*

Although the results below hold regardless of the measures μ and ν , we will need to restrict to appropriate classes of cost functions. At first, we will consider cost functions that satisfy the following assumption, which is sufficient to prove geometrical properties of the Laguerre cells. Later, see Restriction 4.2.15, we will further restrict the class of cost functions considered, in order to obtain analytical results on the cells themselves.

Assumption 4.2.3 (Cost function). *The cost function $c : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}^+$ satisfies the requirements of a distance function:*

$$\text{positivity: } \begin{cases} c(x, y) \geq 0, \quad \forall x, y \in \mathbb{R}^n, \\ c(x, y) = 0 \iff x = y, \quad \forall x, y \in \mathbb{R}^n, \end{cases}$$

$$\text{symmetry: } c(x, y) = c(y, x), \quad \forall x, y \in \mathbb{R}^n,$$

$$\text{triangular inequality: } c(x, y) \leq c(x, z) + c(z, y), \quad \forall x, y, z \in \mathbb{R}^n,$$

and additionally we also assume that the cost function satisfies these two properties:

$$\text{homogeneity: } c(tx, ty) = |t|c(x, y), \quad \forall t \in \mathbb{R}, \forall x, y \in \mathbb{R}^n,$$

$$\text{shift invariance: } c(x + z, y + z) = c(x, y), \quad \forall x, y, z \in \mathbb{R}^n.$$

Remarks 4.2.4.

(a) *It is an interesting theoretical consequence of the last two properties above that (e.g., see [13, p.21]) there exists a vector norm $\|\cdot\|$ such that $c(x, 0) = \|x\|$, $\forall x \in \mathbb{R}^n$, and therefore our cost is a continuous function.*

(b) *Below, see Restriction 4.2.15, we will give very concrete examples of the cost functions we consider in practice. Presently, we remark that commonly adopted choices, like the Euclidean norm squared: $c(x, y) = \|x - y\|_2^2$, do not satisfy Assumption 4.2.3, and hence some of our results below (in particular, Lemma 4.2.10 and Theorem 4.2.12) do not apply to this case.*

4.2.1 Star-shapedness Results

In this Section, we define Laguerre cells and tessellation and show some important geometrical properties of Laguerre cells, that are relevant for our later algorithmic development to solve semi-discrete optimal transport problems. Our main result is Theorem 4.2.12, that appears to be new, at least in the given generality. The results in this section are derived

irrespective of the measure theoretic setting of optimal transport and only require that the cost function satisfy Assumption 4.2.3.

Definition 4.2.5 (Laguerre tessellation). *Given a cost function c satisfying Assumption 4.2.3, given a set of N distinct points $y_i \in \mathbb{R}^n$, $i = 1, 2, \dots, N$, $N \geq 2$, and given a shift vector $w \in \mathbb{R}^N$, the Laguerre tessellation of $\mathbb{R}^n$ associated to $c(\cdot, \cdot)$, to the y_i 's, and to w , is given by the set of N regions (Laguerre cells)*

$$L(y_i) = \{x \in \mathbb{R}^n \mid c(x, y_i) - w_i \leq c(x, y_j) - w_j, \quad \forall j = 1, \dots, N, j \neq i\}, \quad (4.4)$$

for $i = 1, 2, \dots, N$. The points y_i , $i = 1, 2, \dots, N$, are called target points. See Figure 4.1. Moreover, let Ω be as in Assumption 4.2.1 and let $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$. Then, the Laguerre tessellation of Ω is the set of Laguerre cells (see Figure 4.1)

$$A(y_i) = L(y_i) \cap \Omega = \{x \in \Omega \mid c(x, y_i) - w_i \leq c(x, y_j) - w_j, \quad j \neq i\}, \quad i = 1, 2, \dots, N. \quad (4.5)$$

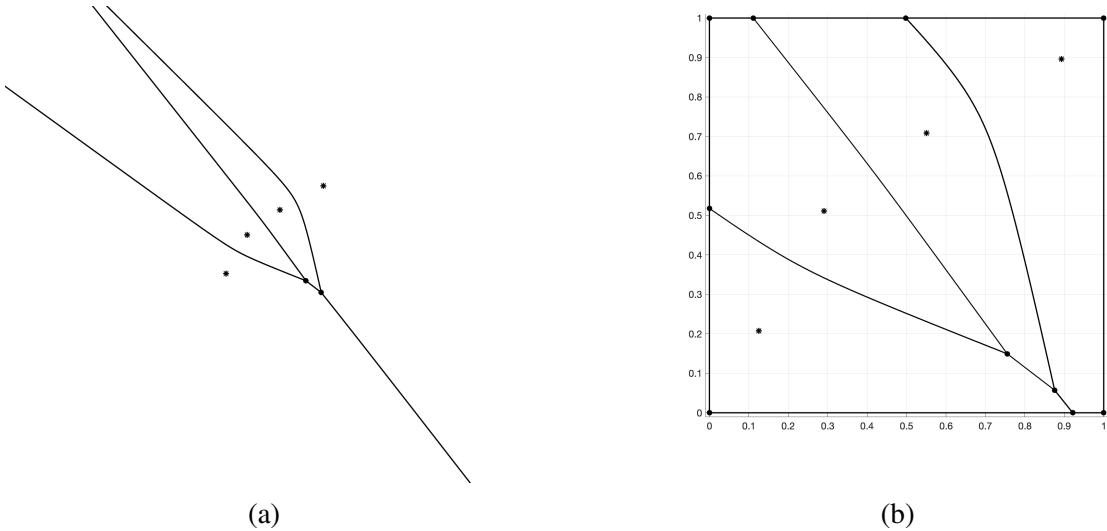


Figure 4.1: For the same target points, w , and cost: (A) Laguerre cells in $\mathbb{R}^2$ and (B) Restricted to the unit square.

Remark 4.2.6. *The special case of $w = 0$ gives the tessellation in terms of Voronoi cells. This is the standard proximity problem.*

Lemma 4.2.7 (Every $x \in \mathbb{R}^n$ is in some $L(y_i)$). *For any given $x \in \mathbb{R}^n$, there exist an index i such that $x \in L(y_i)$.*

Proof. This result follows from the fact that $\mathbb{R}^n = \bigcup_{i=1}^N L(y_i)$. □

Definition 4.2.8 (Active boundary and interior). *The active boundary, or simply boundary, between two Laguerre cells $L(y_i)$ and $L(y_j)$, $i \neq j$, such that $L(y_i) \cap L(y_j) \neq \emptyset$, is indicated with L_{ij} , and it is given by the set of all $x \in \mathbb{R}^n$ satisfying the relation*

$$c(x, y_i) - w_i = c(x, y_j) - w_j \implies w_{ij} = c(x, y_i) - c(x, y_j), \quad w_{ij} = w_i - w_j,$$

as well as the $(N - 2)$ inequalities

$$c(x, y_i) - w_i \leq c(x, y_k) - w_k, \quad k \neq i, j.$$

The boundary of a Laguerre cell $L(y_i)$, indicated with $\partial L(y_i)$, is given by

$$\partial L(y_i) = \bigcup_{\substack{j=1, j \neq i; \\ L(y_i) \cap L(y_j) \neq \emptyset}}^N L_{ij}.$$

Accordingly, we define the interior of $L(y_i)$, and indicate it with $L(y_i)^\circ$, to be the set

$$L(y_i)^\circ = L(y_i) \setminus \partial L(y_i).$$

Lemma 4.2.9 (Existence of boundary between cells: necessary condition). *Let the cost function c satisfy Assumption 4.2.3. The necessary condition for having an active boundary L_{ij} in $\mathbb{R}^n$ between $L(y_i)$ and $L(y_j)$ is that*

$$|w_i - w_j| \leq c(y_i, y_j) = c(y_j, y_i).$$

Proof. Let $w_{ij} = w_i - w_j$. By Definition 4.2.8, the values $x \in L_{ij}$ must satisfy the following relation:

$$c(x, y_i) - w_i = c(x, y_j) - w_j \implies w_{ij} = c(x, y_i) - c(x, y_j).$$

From the triangular inequality:

$$c(x, y_i) \leq c(x, y_j) + c(y_j, y_i) \implies w_{ij} \leq c(x, y_j) + c(y_j, y_i) - c(x, y_j) = c(y_j, y_i).$$

Similarly, with $w_{ji} = w_j - w_i = -w_{ij}$, one has

$$w_{ji} = c(x, y_j) - c(x, y_i) \leq c(x, y_i) + c(y_i, y_j) - c(x, y_i) = c(y_i, y_j)$$

and since $c(y_i, y_j) = c(y_j, y_i)$, we obtain

$$\left\{ \begin{array}{l} w_{ij} \leq c(y_j, y_i) \\ -w_{ij} \leq c(y_j, y_i) \end{array} \right\} \implies |w_{ij}| \leq c(y_j, y_i)$$

and the result follows. □

Lemma 4.2.10 ($y_i \in L(y_i)$). *Let the cost function c satisfy Assumption 4.2.3. If $L(y_i) \neq \emptyset$, each target point y_i is contained in its own Laguerre cell $L(y_i)$.*

Proof. Assume $y_i \notin L(y_i)$. Then, because of Lemma 4.2.7, $y_i \in L(y_j) \setminus \partial L(y_i)$, for some $j \neq i$.

Case 1. Let $L(y_i) \cap L(y_j) \neq \emptyset$. Then, by construction of Laguerre cells, it follows that:

$$y_i \notin L(y_i), y_i \in L(y_j) \implies c(y_i, y_i) - w_i > c(y_i, y_j) - w_j \implies -w_i + w_j > c(y_i, y_j).$$

It follows that $-w_{ij} > c(y_i, y_j)$. But this contradicts Lemma 4.2.9.

Case 2. Let $L(y_i) \cap L(y_j) = \emptyset$. Then, since $y_i \in L(y_j) \setminus \partial L(y_i)$:

$$c(y_i, y_j) - w_j < c(y_i, y_i) - w_i = -w_i \implies w_j - w_i > c(y_i, y_j).$$

Next, let $x_0 \in L(y_i) \setminus \partial L(y_j)$. Such point x_0 exists since $L(y_i) \neq \emptyset$. Then:

$$c(x_0, y_i) - w_i < c(x_0, y_j) - w_j \implies w_j - w_i < c(x_0, y_j) - c(x_0, y_i).$$

Combining the above two equations:

$$\left\{ \begin{array}{l} w_j - w_i > c(y_i, y_j) \\ w_j - w_i < c(x_0, y_j) - c(x_0, y_i) \end{array} \right\} \implies c(y_i, y_j) < c(x_0, y_j) - c(x_0, y_i).$$

Using the triangular inequality:

$$c(x_0, y_j) \leq c(x_0, y_i) + c(y_i, y_j) \implies c(y_i, y_j) < c(x_0, y_i) + c(y_i, y_j) - c(x_0, y_i) = c(y_i, y_j),$$

which is the sought contradiction. □

Lemma 4.2.11. *Let the cost function c satisfy Assumption 4.2.3. If $y_i \in \partial L(y_i)$, then $L(y_i)$ is degenerate, i.e. $L(y_i)^\circ = \emptyset$.*

Proof. W.l.o.g., we can assume that $y_i \in L_{ij}$, for some $j \neq i$. Then, it follows that:

$$c(y_i, y_i) - w_i = c(y_i, y_j) - w_j \implies w_j - w_i = c(y_i, y_j).$$

Next, assume by contradiction that there exist some point $x_0 \in L(y_i)^\circ$. Then x_0 needs to satisfy the following relation:

$$c(x_0, y_i) - w_i < c(x_0, y_j) - w_j \implies w_j - w_i < c(x_0, y_j) - c(x_0, y_i) \implies c(y_i, y_j) < c(x_0, y_j) - c(x_0, y_i).$$

But this contradicts the triangular inequality. □

Theorem 4.2.12 ($L(y_i)$ is star shaped). *Let the cost function c satisfy Assumption 4.2.3. Let $x \in L(y_i)$ and $S(t) = \left\{ t(x - y_i) + y_i, \ 0 \leq t \leq 1 \right\}$. Then, for all $t \in [0, 1]$, $S(t) \in L(y_i)$. As a consequence, each Laguerre cell $L(y_i)$ is star-shaped in $\mathbb{R}^n$ with respect to its own target point y_i .*

Proof. The case $t = 0$ follows from Lemma 4.2.10 and the case $t = 1$ holds since $x \in L(y_i)$. Next, by contradiction, assume that there exists $t_0 \in (0, 1)$ such that the point $x_0 = t_0(x - y_i) + y_i$ is not in $L(y_i)$. W.l.o.g., let $x_0 \in L(y_j) \setminus \partial L(y_i)$, for some $j \neq i$. Then, it follows that, since

$$\begin{aligned} x \in L(y_i) &\implies c(x, y_i) - w_i \leq c(x, y_j) - w_j, \\ x_0 = S(t_0) &\implies c(x, y_i) = c(x, x_0) + c(x_0, y_i), \end{aligned}$$

where the second equality comes about from the following

$$\begin{aligned} c(x, x_0) + c(x_0, y_i) &= c(x, y_i + t_0(x - y_i)) + c(y_i + t_0(x - y_i), y_i) = \\ c((1 - t_0)(x - y_i), 0) + c(t_0(x - y_i), 0) &= (1 - t_0)c(x - y_i, 0) + t_0c(x - y_i, 0) = c(x - y_i, 0) = c(x, y_i). \end{aligned}$$

As a consequence, we get

$$c(x, x_0) + c(x_0, y_i) - w_i \leq c(x, y_j) - w_j \implies c(x_0, y_i) - w_i \leq c(x, y_j) - c(x, x_0) - w_j.$$

From the triangular inequality, it follows that:

$$\begin{aligned} c(x_0, y_i) - w_i &\leq c(x, y_j) - c(x, x_0) - w_j \leq c(x, x_0) + c(x_0, y_j) - c(x, x_0) - w_j = c(x_0, y_j) - w_j, \\ &\implies c(x_0, y_i) - w_i \leq c(x_0, y_j) - w_j. \end{aligned} \tag{4.6}$$

In addition, since $x_0 \in L(y_j) \setminus \partial L(y_i)$, it holds that

$$x_0 \in L(y_j) \implies c(x_0, y_j) - w_j < c(x_0, y_i) - w_i \implies c(x_0, y_i) - w_i > c(x_0, y_j) - w_j.$$

But this inequality contradicts (4.6), and hence the entire segment $S(t) \in L(y_i)$.

The fact that $L(y_i)$ is star-shaped with respect to y_i follows at once by the fact that every point in $L(y_i)$ can be reached via a line segment from y_i entirely contained in $L(y_i)$. $\square$

Corollary 4.2.13. *Let Ω be as in Assumption 4.2.1, let $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$, and let the cost function c satisfy Assumption 4.2.3. Then each Laguerre cell $A(y_i)$ is star-shaped in Ω with respect to the point y_i .*

Proof. This result follows from Theorem 4.2.12 and the fact that the intersection of a convex set with a star-shaped set is a star-shaped set (e.g., see [17]). $\square$

Remark 4.2.14. *For Voronoi diagrams, star-shapedness is simple to infer; e.g., see [43] for results about even more general Voronoi diagrams. But, for Laguerre cells, we have found much fewer conclusive results, except for the standard Euclidean distance. Indeed, that $L(y_i)$ are star-shaped when the cost function is the 2-norm has been observed before, for example see [19, Example 2.12], [33, p.21]. In this case of $c(x, y) = \|x - y\|_2$, Laguerre tessellation are often called Apollonius diagrams or also additively weighted Voronoi diagrams, and in 2-d it has been observed many times that the boundaries between cells are arcs of hyperbolas (e.g., see [19], [33], [47], or [81]), although we are not aware of a similar result for cost functions being a p -norm, with $p \neq 2$.*

4.2.2 Smoothness Results

In this section we give analytical results on smoothness of the boundaries L_{ij} . To get these results, we further restrict the class of appropriate cost functions, as well as an appropriate measure μ . As said in the introduction, μ will always be assumed to be equivalent to Lebesgue measure and with bounded density ρ in Ω : $d\mu = \rho(x)dx$. These requests on μ allow to infer, under mild conditions on the cost function $c(x, y)$ (namely continuity, or even lower semi-continuity), that in (1.3) the “inf” is realized as a “min”, a fact noted several times (e.g., see [2, 28, 75]). However, we need that there is a unique partition (map), and to obtain this result we need to place further restrictions on the cost function. Indeed,

a fundamental result of Cuesta-Albertos and Tuero-Diaz [28, Corollary 4] states that there exists a unique optimal partition (map) **if** the cost function $c(x, y)$ gives μ -measure 0 to the cells boundaries. Because of this, the key concern will be to determine that the boundaries have indeed 0-measure, in particular under what conditions on the cost function this is possible.

Several authors have addressed the issue of selecting an appropriate cost, and proposed restrictions on the cost function in order to obtain a desired end result. For example, in [44] the authors restricted to what they called *admissible costs* (see below), in [19] they restricted to so-called *radial cost* functions, whereas in [52] the authors required the cost to satisfy conditions that they called (Twist)-(Reg)-(QC), and in [31] the authors required the cost to satisfy their conditions (Diff-2) and (Cont-2); more on all these cost functions is below. We should note that the restrictions placed on the cost in the works [19], [52], and [31], appear to have been motivated by the cost given by the 2-norm square, a case that violates our original Assumption 4.2.3 and that therefore is not of specific interest in the present work, since we may fail to have star-shaped Laguerre cells, a fact which plays a very important role in our algorithmic development. A different approach was taken in [32] and in [47]. Namely, in [32], Dieci and Walsh considered cost given by p -norms ($1 < p < \infty$) and in [47] Hartmann and Schuhmacher restricted to the 2-norm cost, $\|x - y\|_2$; these choices surely satisfies Assumption 4.2.3 and thus lead to star-shaped Laguerre cells, although this fact was not noticed/used in [32] or [47]. The choice we will make is just slightly more general than the choice made in [32], though more general than that in [47]: we will henceforth restrict to cost functions satisfying the following Restriction 4.2.15, which satisfy of course our previous Assumption 4.2.3.

Restriction 4.2.15. *Hereafter, the cost functions considered are given by positive, finite, combinations of p -norms, with $1 < p < \infty$. That is,*

$$c(x, y) = \sum_{l=1}^m \alpha_l \|x - y\|_{p_l}, \quad \text{where } \alpha_l > 0, \quad p_l \in (1, \infty). \quad (4.7)$$

The relation between our class of cost functions, and the choices made in previous works is elaborated on in Remark 4.2.16 below.

Remark 4.2.16. *The radial cost functions of [19] are functions of the form $c(x, y) = g(\|x - y\|_2)$, where g is an increasing function, e.g., $\|x - y\|_2^2$; our cost functions do not fit in this class. Conditions (Reg) of [52] can be generally satisfied by our cost functions, in light of Lemma 4.2.21; however, conditions (Twist) and (QC) of [52] are more difficult to verify, and it may possibly not hold for our cost functions. Finally, the authors of [31] require sufficiently regular cost functions (which we can require, see Lemma 4.2.21), satisfying (in their notation) conditions (Diff-2-a), (Diff-2-b), (Diff-2-c) and (Cont-2); it is not difficult to verify that, with our cost functions, conditions (Diff-2-b)-(Diff-2-c)-(Cont-2) can be enforced, but condition (Diff-2-a) is more problematic. This condition asks for the existence of $\varepsilon > 0$ such that $\|\nabla_x c(x, y_i) - \nabla_x c(x, y_j)\| > \varepsilon$ for all x in L_{ij} , and thus the authors of [31] ask for this condition to hold in $\mathbb{R}^n$, and not just in a bounded set Ω . For completeness, in Lemma 4.2.25 below we show that this condition holds for us for even values of p , with $c(x, y) = \|x - y\|_p$, when we look at it on a bounded convex set Ω , which is what we do in practice in our numerical computations. Finally, our choice of cost function always gives an admissible cost according to the definition of [44], as we prove below.*

Definition 4.2.17 (Admissible cost; [44]). *Let $\mathcal{L}$ be Lebesgue measure, let Ω be as in Assumption 4.2.1, and let y_1, y_2 be two distinct target points in Ω° . A cost function satisfying Assumption 4.2.3 is called admissible if there exist values $w_a \leq w_b$ such that:*

- $\mathcal{L}(A(y_1))$ is continuously increasing from 0 to $\mathcal{L}(\Omega)$ as $w_{12} = w_1 - w_2$ increases from w_a to w_b , where $A(y_1) = \{x \in \Omega \mid c(x, y_1) - w_1 \leq c(x, y_2) - w_2\}$;
- likewise, $\mathcal{L}(A(y_2))$ is continuously decreasing from $\mathcal{L}(\Omega)$ to 0 as $w_{12} = w_1 - w_2$ increases from w_a to w_b , where $A(y_2) = \{x \in \Omega \mid c(x, y_2) - w_2 \leq c(x, y_1) - w_1\}$.

Lemma 4.2.18. *All p -norms, $1 < p < \infty$, are admissible cost functions as in Definition 4.2.17, that is $c(x, y) = \|x - y\|_p$, for $1 < p < \infty$, is an admissible cost function. Moreover,*

a positive combination of admissible cost functions is an admissible cost function, and in particular a cost function satisfying Restriction 4.2.15 is admissible according to Definition 4.2.17.

Proof. For $1 < p < \infty$, it is enough to observe that when $c(x, y) = \|x - y\|_p$, choosing $w_a = -c(y_1, y_2) = -\|y_1 - y_2\|_p$ and $w_b = c(y_1, y_2) = \|y_1 - y_2\|_p$ will work in Definition 4.2.17.

Next, take a cost function $c(x, y) = \sum_{k=1}^m \alpha_k c_k(x, y)$ where $\alpha_k > 0$ and $c_k(\cdot, \cdot)$ are admissible cost functions. Let $w_a^{(k)}$ and $w_b^{(k)}$ be values satisfying the requirements for admissibility of the $c_k(\cdot, \cdot)$ functions in Definition 4.2.17. Then, in Definition 4.2.17, we can take $w_a = \sum_{k=1}^m \alpha_k w_a^{(k)}$ and $w_b = \sum_{k=1}^m \alpha_k w_b^{(k)}$ to obtain that $c(\cdot, \cdot)$ is admissible. $\square$

An interesting and useful result for our cost functions is the following.

Lemma 4.2.19. *Let Ω be as in Assumption 4.2.1 and let $y_1, y_2 \in \Omega^\circ$ be two distinct target points. Let $w_{12} = w_1 - w_2$ satisfy $|w_{12}| \leq c(y_1, y_2)$ (see Lemma 4.2.9). If $c(\cdot, \cdot)$ is an admissible cost function as in Restriction 4.2.15, then $A(y_1) \cap A(y_2) \neq \emptyset$, where*

$$\begin{aligned} A(y_1) &= \{x \in \Omega \mid c(x, y_1) - w_1 \leq c(x, y_2) - w_2\}, \\ A(y_2) &= \{x \in \Omega \mid c(x, y_2) - w_2 \leq c(x, y_1) - w_1\}. \end{aligned}$$

Proof. By direct computation, it is straightforward to show that the point $x_0 = y_1 + \frac{k+1}{2}(y_2 - y_1)$, $k = \frac{w_{12}}{c(y_1, y_2)} \in [-1, 1]$, is contained in the boundary set $A(y_1) \cap A(y_2)$, where

$$A(y_1) \cap A(y_2) = \{x \in \Omega \mid c(x, y_1) - w_1 = c(x, y_2) - w_2\}.$$

Hence the result follows. $\square$

Next, in order to obtain μ -measure 0 for the boundaries between Laguerre cells, we will use some results of differential geometric flavor, and for these we need some preliminary results.

Fact 4.2.20. *Let $a \in \mathbb{R}$. For $s < 1$ and $r \geq 1$:*

$$\lim_{a \rightarrow 0} \frac{a^r}{|a|^s} = 0.$$

Proof. This follows from the fact that $\frac{a}{|a|^s} = |a|^{1-s} \text{sign}(a) \rightarrow 0$ since $\text{sign}(a)$ is bounded and $s < 1$. □

Lemma 4.2.21. *Consider the function $f(x) = \|x\|_p$, $1 < p < \infty$, for x in an open subset B of $\mathbb{R}^n$ not containing the origin. Then, f is a $\mathcal{C}^k$ function in B , where*

$$k = \begin{cases} \infty, & \text{if } p \text{ even} \\ m, & \text{if } m < p \leq m+1, m \in \mathbb{Z}^+. \end{cases}$$

Proof. For all x , the function $f(x)$ can be rewritten as:

$$f(x) = [|x_1|^p + |x_2|^p + \cdots + |x_d|^p]^{1/p}.$$

Then, it is enough to check smoothness of the single term $|x_j|^p$, $j \in [1, d]$, since $x \neq 0$. Note that $|x_j|^p = x_j^p$ when p is an even integer, and so the result follows for p even. Otherwise, using Fact 4.2.20 it is simple to show that the function $|x_j|^p$ is t times continuously differentiable where $t < p \leq t+1$. □

Let $F(x)$ be the function corresponding to the boundary set between two Laguerre cells $L(y_1)$ and $L(y_2)$ in $\mathbb{R}^n$:

$$F(x) = c(x, y_1) - c(x, y_2) \implies L(y_1) \cap L(y_2) = \{x \in \mathbb{R}^n \mid F(x) = w_{12}\}. \quad (4.8)$$

Lemma 4.2.22. *Let $c(\cdot, \cdot)$ be an admissible cost function as in Restriction 4.2.15, $y_1, y_2 \in \mathbb{R}^n$ be two distinct target points, and $w_{12} = w_1 - w_2$ satisfy $|w_{12}| < c(y_1, y_2)$ (see Lemma 4.2.9). Assume that w_{12} is a regular value for the function $F(x)$, that is that $\nabla F(x_0) \neq 0$,*

$\forall x_0 \in L(y_1) \cap L(y_2)$. Then, the boundary set $\{x : F(x) = w_{12}\}$ is a $\mathcal{C}^k$ submanifold of dimension $(n - 1)$ in $\mathbb{R}^n$.

Proof. Lemma 4.2.19 gives the existence of at least one point $x_0 : F(x_0) = w_{12}$, where F is given in (4.8). By our Restriction 4.2.15, the function $F(x)$ is a $\mathcal{C}^k$ function, with $k \geq 1$, in an open neighborhood of L_{ij} . Since $\nabla F(x_0) \neq 0$ is assumed, then the smooth manifold theorem tells us that $S = F^{-1}(w_{12}) \subset \mathbb{R}^n$ is a $(n - 1)$ -dimensional $\mathcal{C}^k$ manifold embedded in $\mathbb{R}^n$. $\square$

Remark 4.2.23. We notice that the request that w_{12} is a regular value for F is not a severe restriction, since the Morse-Sard Theorem guarantees that almost all values w_{12} are regular values for F . Furthermore, we show in Claim 4.2.24 that $\nabla F(x_0) \neq 0$, $\forall x_0 \in L(y_1) \cap L(y_2)$, whenever the cost function is a p -norm with p an even number. As a consequence, for cost functions given by positive combination of p -norms with p even, there is no need to assume that $\nabla F(x_0) \neq 0$, since it is always satisfied.

4.2.2.1 Even p -norms

Claim 4.2.24. Let the cost function be a p -norm, where p is even: $c(x, y) = \|x - y\|_p$, $p = 2k$, $k = 1, 2, \dots$. If $|w| < \|y_1 - y_2\|_p$, then $\nabla F(x_0) \neq 0$, $\forall x_0$ such that $F(x_0) = w$, where

$$F(x) = \|x - y_1\|_p - \|x - y_2\|_p.$$

Proof. Assume, by contradiction, that there exists a point x_0 where both $F(x_0) = w$ and

$\nabla F(x_0) = 0$. An explicit computation gives

$$\nabla F(x) = \frac{1}{\|x - y_1\|_p^{p-1}} \begin{pmatrix} (x - y_1)_1^{p-1} \\ \vdots \\ (x - y_1)_d^{p-1} \end{pmatrix} - \frac{1}{\|x - y_2\|_p^{p-1}} \begin{pmatrix} (x - y_2)_1^{p-1} \\ \vdots \\ (x - y_2)_d^{p-1} \end{pmatrix} \quad \text{and so}$$

$$\nabla F(x_0) = 0 \implies \frac{(x_0 - y_1)_j^{p-1}}{\|x_0 - y_1\|_p^{p-1}} = \frac{(x_0 - y_2)_j^{p-1}}{\|x_0 - y_2\|_p^{p-1}} \implies \frac{(x_0 - y_1)_j}{\|x_0 - y_1\|_p} = \frac{(x_0 - y_2)_j}{\|x_0 - y_2\|_p},$$

for $j = 1, \dots, d$. Since $F(x_0) = 0$, then $\|x_0 - y_1\|_p = w + \|x_0 - y_2\|_p$ and so

$$\frac{(x_0 - y_1)_j}{w + \|x_0 - y_2\|_p} = \frac{(x_0 - y_2)_j}{\|x_0 - y_2\|_p} \implies w(x_0 - y_2)_j = \|x_0 - y_2\|_p (y_2 - y_1)_j,$$

for $j = 1, \dots, d$. Taking the p -norm of the vectors on both sides of the last relation gives

$$|w| \cdot \|x_0 - y_2\|_p = \|x_0 - y_2\|_p \|y_2 - y_1\|_p \quad \text{or} \quad |w| = \|y_2 - y_1\|_p,$$

which is a contradiction and the proof follows. $\square$

Lemma 4.2.25. *Let $\Omega \subset \mathbb{R}^n$ satisfy Assumption 4.2.1, $y_i \in \Omega^\circ$ and $i = 1, 2, \dots, N$, be N distinct target points, and let the shift values be feasible: $|w_{ij}| < c(y_i, y_j)$, $\forall i, j = 1, 2, \dots, N$, $i \neq j$. Consider the cost function given by a p -norm with p even. Then, there $\exists \varepsilon > 0$ such that $\forall i, j = 1, 2, \dots, N$, $i \neq j$*

$$\|\nabla_x c(x, y_i) - \nabla_x c(x, y_j)\| \geq \varepsilon, \quad \forall x \in L_{ij} \cap \Omega,$$

where $\|\cdot\|$ is the standard Euclidean distance $\|\cdot\|_2$.

Proof. Firstly, we are going to show that exists a constant, which depends on p , call it $\varepsilon_{ij}(p) > 0$, such that

$$\|\nabla_x \|x - y_i\|_p - \nabla_x \|x - y_j\|_p\| \geq \varepsilon_{ij}(p), \quad \forall x \in L_{ij} \cap \Omega.$$

By direct computation, as in Claim 4.2.24, we have

$$\|\nabla_x \|x - y_i\|_p - \nabla_x \|x - y_j\|_p\|_p = \left\| \begin{pmatrix} a_1^{p-1} \\ \vdots \\ a_d^{p-1} \end{pmatrix} - \begin{pmatrix} b_1^{p-1} \\ \vdots \\ b_d^{p-1} \end{pmatrix} \right\|_p \quad \text{where}$$

$$a_j = \frac{(x - y_1)_j}{\|x - y_1\|_p}, \quad b_j = \frac{(x - y_2)_j}{\|x - y_2\|_p}, \quad j = 1, \dots, d.$$

Now, abusing of notation, we will write a^{p-1} for the Hadamard product of the vector a with itself $(p-1)$ -times, and thus $a^{p-1} - b^{p-1} = (a - b) \circ (a^{p-2} + a^{p-3} \circ b + \dots + b^{p-2})$, where “ $\circ$ ” is the Hadamard product¹. Next, using Claim 4.2.26 we have

$$\left\| \frac{(x - y_i)^{p-1}}{\|x - y_i\|_p^{p-1}} - \frac{(x - y_j)^{p-1}}{\|x - y_j\|_p^{p-1}} \right\|_p \geq \Delta \left\| \frac{(x - y_i)}{\|x - y_i\|_p} - \frac{(x - y_j)}{\|x - y_j\|_p} \right\|_p.$$

Using that $x \in L_{ij} \cap \Omega \implies \|x - y_i\|_p = \|x - y_j\|_p + w_{ij}$:

$$\begin{aligned} \frac{\| \|x - y_j\|_p(x - y_i) - \|x - y_i\|_p(x - y_j) \|_p}{\|x - y_i\|_p \|x - y_j\|_p} &= \frac{\| \|x - y_j\|_p(x - y_i) - (\|x - y_j\|_p + w_{ij})(x - y_j) \|_p}{\|x - y_i\|_p \|x - y_j\|_p}, \\ \implies \frac{\| \|x - y_j\|_p(y_j - y_i) - w_{ij}(x - y_j) \|_p}{\|x - y_i\|_p \|x - y_j\|_p} &\geq \frac{\| \|x - y_j\|_p \|y_j - y_i\|_p - |w_{ij}| \|x - y_j\|_p \|_p}{\|x - y_i\|_p \|x - y_j\|_p}, \\ \implies \|\nabla_x \|x - y_i\|_p - \nabla_x \|x - y_j\|_p\|_p &\geq \Delta \frac{\| \|y_j - y_i\|_p - |w_{ij}| \|_p}{\|x - y_i\|_p} \end{aligned}$$

Using that $x \in \Omega \implies \|x - y_i\|_p \leq C(y_i, \Omega, p) \text{diam}(\Omega) \implies \frac{1}{\|x - y_i\|_p} \geq \frac{1}{C(y_i, \Omega, p) \text{diam}(\Omega)}$,

where $C(y_i, \Omega, p) > 0$ is a constant depending on y_i, Ω and p , and so:

$$\|\nabla_x \|x - y_i\|_p - \nabla_x \|x - y_j\|_p\|_p \geq \Delta \frac{\| \|y_j - y_i\|_p - |w_{ij}| \|_p}{C(y_i, \Omega, p) \text{diam}(\Omega)} \equiv \varepsilon_{ij}(p) > 0.$$

Next, using equivalency of p -norm and 2-norm, there is a constant which depends on p , call

$${}^1 a \circ b = \begin{pmatrix} a_1 b_1 \\ \vdots \\ a_d b_d \end{pmatrix}$$

it $\alpha(p)$, such that $\|\cdot\|_2 \geq \alpha(p)\|\cdot\|_p$, $\alpha(p) > 0$, and so we get

$$\|\nabla_x\|x - y_i\|_p - \nabla_x\|x - y_j\|_p\| \geq \alpha(p)\|\nabla_x\|x - y_i\|_p - \nabla_x\|x - y_j\|_p\|_p = \alpha(p)\varepsilon_{ij}(p) \equiv \varepsilon_{ij}.$$

Finally, it follows that $\forall i, j = 1, 2, \dots, N, i \neq j$,

$$\|\nabla_x\|x - y_i\|_p - \nabla_x\|x - y_j\|_p\| \geq \varepsilon, \quad (4.9)$$

where $\varepsilon = \min_{i,j} \varepsilon_{ij} > 0$. □

Claim 4.2.26. *With the notation from Lemma 4.2.25, there exists $\Delta > 0$ such that*

$$\|a^{p-2} + a^{p-3} \circ b + \dots + b^{p-2}\|_p \geq \Delta > 0$$

where p is an even number and

$$a, b \in \mathbb{R}^n, \quad a_l = \frac{(x - y_i)_j}{\|x - y_i\|_p}, \quad b_l = \frac{(x - y_j)_j}{\|x - y_j\|_p}, \quad l = 1, \dots, d.$$

Proof.

$$\|a^{p-2} + a^{p-3} \circ b + \dots + b^{p-2}\|_p \geq |a_k^{p-2} + a_k^{p-3}b_k + \dots + b_k^{p-2}|,$$

for an index k such that $|a_k| = \|a\|_\infty$. Next, using Claim 4.2.27 we have

$$|a_k| = \frac{(|x - y_i|)_k}{\|x - y_i\|_p} \geq \frac{\delta_a}{C(y_i, \Omega, p)\text{diam}(\Omega)} > 0.$$

Next we are going to consider cases:

- $a_k < 0$ and $b_k \leq 0$:

$$|a_k^{p-2} + a_k^{p-3}b_k + \dots + b_k^{p-2}| \geq |a_k^{p-2}| \geq \frac{\delta_a^{p-2}}{[C(y_i, \Omega, p)\text{diam}(\Omega)]^{p-2}}.$$

- $a_k > 0$ and $b_k \geq 0$:

$$|a_k^{p-2} + a_k^{p-3}b_k + \dots + b_k^{p-2}| \geq |a_k^{p-2}| \geq \frac{\delta_a^{p-2}}{[C(y_i, \Omega, p)\text{diam}(\Omega)]^{p-2}}.$$

- $a_k < 0$ and $b_k > 0$:

$$\begin{aligned} |a_k^{p-2} + a_k^{p-3}b_k + \dots + b_k^{p-2}| &\geq |a_k^{p-2} + a_k^{p-3}b_k + \dots + a_k b_k^{p-3}| = |a_k| |a_k^{p-3} + a_k^{p-4}b_k + \dots + b_k^{p-3}| \geq \\ &\geq |a_k| |a_k^{p-3} + a_k^{p-4}b_k + \dots + a_k b_k^{p-4}| = |a_k|^2 |a_k^{p-4} + a_k^{p-5}b_k + \dots + b_k^{p-4}| \geq \dots \geq |a_k|^{p-2} \implies \\ &\implies |a_k^{p-2} + a_k^{p-3}b_k + \dots + b_k^{p-2}| \geq |a_k|^{p-2} \geq \frac{\delta_a^{p-2}}{[C(y_i, \Omega, p)\text{diam}(\Omega)]^{p-2}}. \end{aligned}$$

- $a_k > 0$ and $b_k < 0$: using Claim 4.2.27 we have

$$|b_k| = \frac{(|x - y_j|)_k}{\|x - y_j\|_p} \geq \frac{\delta_b}{C(y_j, \Omega, p)\text{diam}(\Omega)} > 0.$$

Then it follows that

$$\begin{aligned} |b_k^{p-2} + b_k^{p-3}a_k + \dots + a_k^{p-2}| &\geq |b_k^{p-2} + b_k^{p-3}a_k + \dots + b_k a_k^{p-3}| = |b_k| |b_k^{p-3} + b_k^{p-4}a_k + \dots + a_k^{p-3}| \geq \\ &\geq |b_k| |b_k^{p-3} + b_k^{p-4}a_k + \dots + b_k a_k^{p-4}| = |b_k|^2 |b_k^{p-4} + b_k^{p-5}a_k + \dots + a_k^{p-4}| \geq \dots \geq |b_k|^{p-2} \implies \\ &\implies |a_k^{p-2} + a_k^{p-3}b_k + \dots + b_k^{p-2}| \geq |b_k|^{p-2} \geq \frac{\delta_b^{p-2}}{[C(y_j, \Omega, p)\text{diam}(\Omega)]^{p-2}}. \end{aligned}$$

As a result, it follows that

$$\|a^{p-2} + a^{p-3}b + \dots + b^{p-2}\|_p \geq \Delta$$

where

$$\Delta = \begin{cases} \frac{\delta_b^{p-2}}{[C(y_j, \Omega, p)\text{diam}(\Omega)]^{p-2}}, & \text{if } a_k > 0 \text{ and } b_k < 0, \\ \frac{\delta_a^{p-2}}{[C(y_i, \Omega, p)\text{diam}(\Omega)]^{p-2}}, & \text{otherwise.} \end{cases}$$

Thus, the result follows. $\square$

Claim 4.2.27. *With the notation from Lemma 4.2.25 and Claim 4.2.26, there exists $\delta_a > 0$*

and $\delta_b > 0$ such that

$$\|a\|_\infty \geq \frac{\delta_a}{C(y_i, \Omega, p)\text{diam}(\Omega)} > 0, \quad \|b\|_\infty \geq \frac{\delta_b}{C(y_j, \Omega, p)\text{diam}(\Omega)} > 0$$

where p is an even number and

$$a, b \in \mathbb{R}^n, \quad a_l = \frac{(x - y_i)_l}{\|x - y_i\|_p}, \quad b_l = \frac{(x - y_j)_l}{\|x - y_j\|_p}, \quad l = 1, \dots, d.$$

Proof. Firstly, using assumption from Lemma 4.2.25, let $w_{ij} = \xi - \|y_i - y_j\|_p$, $0 < \xi < \|y_i - y_j\|_p$. Then

$$\begin{aligned} x \in e_{ij} \cap \Omega &\implies \|x - y_i\|_p = \|x - y_j\|_p + w_{ij} = \|x - y_j\|_p + \xi - \|y_i - y_j\|_p \\ \|x - y_i\|_p &\geq \xi - \|x - y_i\|_p \implies \|x - y_i\|_p \geq \frac{\xi}{2} > 0 \end{aligned}$$

Next, using equivalency of p -norm and ∞ -norm: $\|\cdot\|_\infty \geq \gamma(p)\|\cdot\|_p$, $\gamma(p) > 0$, we get

$$\|x - y_i\|_\infty \geq \gamma(p)\frac{\xi}{2} \implies \|a\|_\infty = \frac{\|x - y_i\|_\infty}{\|x - y_i\|_p} \geq \frac{\delta_a}{C(y_i, \Omega, p)\text{diam}(\Omega)} > 0$$

where $\delta_a = \gamma(p)\frac{\xi}{2}$. Similarly, it follows that

$$\begin{aligned} x \in e_{ij} \cap \Omega &\implies \|x - y_j\|_p = \|x - y_i\|_p - w_{ij} = \|x - y_i\|_p - \xi + \|y_i - y_j\|_p \geq \|y_i - y_j\|_p - \xi > 0 \\ \|x - y_j\|_\infty &\geq \gamma(p)(\|y_i - y_j\|_p - \xi) \implies \|b\|_\infty = \frac{\|x - y_j\|_\infty}{\|x - y_j\|_p} \geq \frac{\delta_b}{C(y_j, \Omega, p)\text{diam}(\Omega)} > 0 \end{aligned}$$

where $\delta_b = \gamma(p)(\|y_i - y_j\|_p - \xi)$. □

Remark 4.2.28. In 2-d, Lemma 4.2.22 says that the set $F^{-1}(w_{12})$ is a smooth curve. Cfr. with Remark 4.2.14 for $c(x, y) = \|x - y\|_2$.

Theorem 4.2.29. Let $y_i \in \mathbb{R}^n$, $i = 1, 2, \dots, N$, be a set of N distinct target points, and $c(\cdot, \cdot)$ be an admissible cost function as in Restriction 4.2.15. In addition, assume that $y_i \notin \partial L(y_i)$, $i = 1, 2, \dots, N$. Then, the boundary of each Laguerre cell $L(y_i)$ consists of

at most $(N - 1)$ sections of $\mathcal{C}^k$ $(n - 1)$ -dimensional manifolds of $\mathbb{R}^n$.

Let Ω be as in Assumption 4.2.1, and let $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$, be a set of N distinct target points, such that $y_i \notin \partial A(y_i)$, $i = 1, 2, \dots, N$. Then, the boundary of each Laguerre cell $A(y_i)$ consists of sections of $\mathcal{C}^k$ $(n - 1)$ -dimensional manifolds of $\mathbb{R}^n$.

Proof. The first statement follows from Lemma 4.2.22 and the form of $\partial L(y_i)$:

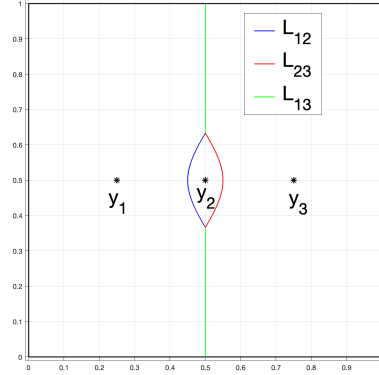
$$\partial L(y_i) = \bigcup_{\substack{j=1, j \neq i: \\ L(y_i) \cap L(y_j) \neq \emptyset}}^N L_{ij}.$$

The second statement follows from the fact that

$$\partial A(y_i) = \partial(L(y_i) \cap \Omega), \quad i = 1, 2, \dots, N,$$

from Lemma 4.2.22, and from the assumed piecewise smoothness of $\partial\Omega$. □

Remark 4.2.30. In general, the union of several pieces of the same smooth $(n - 1)$ -dimensional manifold can be part of $A(y_i) \cap A(y_j)$. See the Figure on the right. It is guaranteed to be a single smooth section when there are exactly two target points.



Lemma 4.2.31. Let $\mathcal{L}$ be Lebesgue measure. Let $\Omega \subset \mathbb{R}^n$ be as in Assumption 4.2.1, and let $y_1, y_2 \in \Omega^\circ$ be two distinct target points. Let $c(\cdot, \cdot)$ be a cost function satisfying Restriction 4.2.15. Then, $\mathcal{L}(A(y_1) \cap A(y_2)) = 0$ holds, where

$$A(y_1) = \{x \in \Omega \mid c(x, y_1) - w_1 \leq c(x, y_2) - w_2\},$$

$$A(y_2) = \{x \in \Omega \mid c(x, y_2) - w_2 \leq c(x, y_1) - w_1\}.$$

Proof. From Lemma 4.2.22, $\mathcal{L}(L_{ij}) = 0$, and thus the result follows. $\square$

Corollary 4.2.32. *Let $\mathcal{L}$ be Lebesgue measure. Let $\Omega \subset \mathbb{R}^n$ be as in Assumption 4.2.1, and let $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$, be N distinct target points. If $c(\cdot, \cdot)$ is an admissible cost function, then $\mathcal{L}(\partial A(y_i)) = 0$, $i = 1, 2, \dots, N$.*

Proof. This result follows from Lemma 4.2.31, and the facts that $\partial L(y_i) = \bigcup_{\substack{j=1, j \neq i: \\ L(y_i) \cap L(y_j) \neq \emptyset}}^N L_{ij}$, and that $A(y_i) = L(y_i) \cap \Omega$. $\square$

Example 4.2.33. *We show next why the 1-norm and the ∞ -norm must be excluded from consideration in our restriction on allowed costs. The reason is that they violate Lemma 4.2.31 and lead to lack of uniqueness, in general. This fact is actually well understood, and already true for Voronoi diagrams, but we give the following counterexamples because they are simple and can be worked out by hand.*

- Counterexample for 1-norm. Let $\Omega = [0, 3]^2$, $y_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $y_2 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$, $w_1 = w_2 = 0$. The boundary between the two Laguerre cells with respect to the 1-norm will satisfy the following equation:

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in A(y_1) \cap A(y_2) \implies \|x - y_1\|_1 = \|x - y_2\|_1$$

$$\implies |x_1 - 1| + |x_2 - 2| = |x_1 - 2| + |x_2 - 1|.$$

But, to illustrate, any point in the square $0 \leq x_1 \leq 1$ and $0 \leq x_2 \leq 1$ will satisfy the above relation: $(1 - x_1) + (2 - x_2) = (2 - x_1) + (1 - x_2)$. As a result, $\mu(A(y_1) \cap A(y_2)) \neq 0$. See Figure 4.2: any point in the shaded regions is both in $A(y_1)$ and $A(y_2)$. In particular, we cannot satisfy $\mu(A(y_1)) + \mu(A(y_2)) = 1$.

- Counterexample for ∞ -norm. Let $\Omega = [0, 3]^2$, $y_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $y_2 = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$, $w_1 = w_2 =$

0. The boundary between the two Laguerre cells with respect to ∞ -norm will satisfy the following equation:

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in A(y_1) \cap A(y_2) \implies \|x - y_1\|_\infty = \|x - y_2\|_\infty,$$

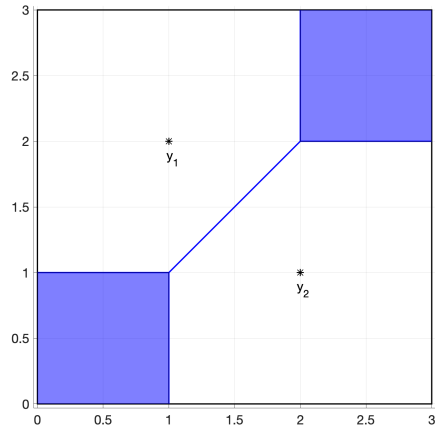
$$\max\{|x_1 - 1|, |x_2 - 2|\} = \max\{|x_1 - 3|, |x_2 - 2|\}.$$

Now, take a point x in the square $1 \leq x_1 \leq 2$ and $0 \leq x_2 \leq 1$. Then:

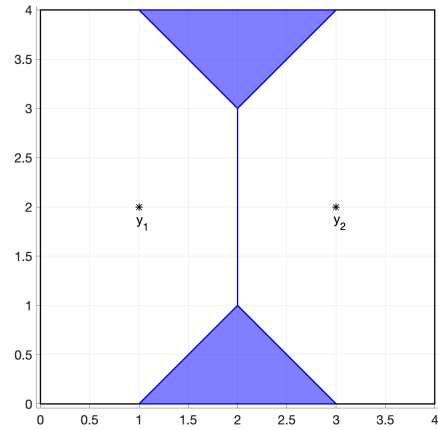
$$\max\{(x_1 - 1), (2 - x_2)\} = \max\{(3 - x_1), (2 - x_2)\} \implies$$

$$\implies (2 - x_2) = \max\{(3 - x_1), (2 - x_2)\}.$$

But, to illustrate, any point in the region $x_1 - x_2 > 1$ will satisfy this relation and thus $\mu(A(y_1) \cap A(y_2)) \neq 0$. See Figure 4.2: any point in the shaded regions is both in $A(y_1)$ and $A(y_2)$. In particular, we cannot satisfy $\mu(A(y_1)) + \mu(A(y_2)) = 1$.



(a) $c(x, y) = \|x - y\|_1$



(b) $c(x, y) = \|x - y\|_\infty$

Figure 4.2: Counterexamples to uniqueness $w = 0$ The shaded regions are in both $A(y_1)$ and $A(y_2)$.

Finally, the following elementary result is useful to justify some steps of our algorithm

(see Section 4.3.1.1).

Corollary 4.2.34. *Let $\Omega \subset \mathbb{R}^n$ be as in Assumption 4.2.1, let c satisfy Restriction 4.2.15, and let $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$, be N distinct target points. Then, any open line segment from a point on $\partial A(y_i)$ to the point y_i is contained in the interior of $A(y_i)$.*

Proof. This result follows from Corollary 4.2.13, piecewise smoothness of $\partial A(y_i)$ and the fact that $\mathcal{L}(\partial A(y_i)) = 0$. □

4.2.3 Minimization Problem Results

The results in this section are needed to justify use of our numerical method for finding the OT partition. With the exception of Theorem 4.2.36, and possibly point (2) of Theorem 4.2.40, the results below are known in the literature on semi-discrete optimal transport.

Theorem 4.2.35. *Let $\Omega \subset \mathbb{R}^n$ be as in Assumption 4.2.1, μ be a probability measure with support on Ω , equivalent to Lebesgue measure, let $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$, be a set of N distinct target points, and $c(\cdot, \cdot)$ be an admissible cost function as in Restriction 4.2.15. Given a discrete measure ν supported at the y_i 's, that is $\nu(x) = \nu_i \delta(y_i)$, $i = 1, \dots, N$, $\nu_i > 0$ and $\sum_{i=1}^N \nu_i = 1$, then there exists a shift vector w (unique up to adding the same constant to all its entries), and a partition of Ω : $\{A_i\}$, such that the partition is unique (up to sets of μ -measure 0), and such that $\mu(A(y_i)) = \nu_i$ is satisfied for all $i = 1, 2, \dots, N$.*

Proof. Since our cost function is an admissible cost (Lemma 4.2.18), this result is in [44, Theorem 1]. □

A very important fact, relative to the solution w of Theorem 4.2.35, is the following *feasibility result*, which strengthens Lemma 4.2.9 and also has crucial algorithmic implications.

Theorem 4.2.36 (Feasibility of w). *With the notation of Theorem 4.2.35, the vector w must satisfy the following relation*

$$|w_i - w_j| < c(y_i, y_j), \quad i \neq j. \quad (4.10)$$

Proof. From Lemma 4.2.11, we have that if

$$w_j - w_i = c(y_i, y_j) \Rightarrow L^\circ(y_i) = \emptyset.$$

But then, $A^\circ(y_i) = \emptyset$ and $\mu(A(y_i)) = 0$ and we cannot satisfy $\mu(A(y_i)) = \nu_i$ with $\nu_i > 0$. □

Theorem 4.2.37. *Let μ be a bounded probability measure on $\Omega \subset \mathbb{R}^n$, with Ω satisfying Assumption 4.2.1, let $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$, be a set of N distinct target points, let $c(\cdot, \cdot)$ be an admissible cost function satisfying Restriction 4.2.15, and let ν be a discrete measure supported at the y_i 's, that is $\nu(x) = \nu_i \delta(y_i)$, $i = 1, \dots, N$, $\nu_i > 0$ and $\sum_{i=1}^N \nu_i = 1$. Then, the function $\Phi(w)$*

$$\Phi(w) = - \sum_i^N \int_{A(y_i)} [c(x, y_i) - w_i] d\mu - \sum_i^N w_i \nu_i, \quad (4.11)$$

is convex, $\mathcal{C}^1$ -smooth, and has gradient

$$\nabla \Phi(w) = \{\mu(A(y_i)) - \nu_i\}_{i=1:N}. \quad (4.12)$$

Moreover, a minimizer of Φ gives the optimal transport map (partition), up to sets of μ -measure 0.

Proof. With our Restriction 4.2.15, the result follows from from [31, Proposition 1]. (See also [52, Theorem 1.1 and Corollary 1.2].) □

Remarks 4.2.38.

- (1) *In agreement with the uniqueness statement of Theorem 4.2.35, it is plainly apparent that in Theorem 4.2.37 neither $\Phi(w)$ nor $\nabla\Phi(w)$ change if a constant value is added to all entries of w (the same constant, of course). For our problem, this is the only lack of uniqueness we have.*
- (2) *It is worth emphasizing that the dependence on w in the gradient $\nabla\Phi$ is through the regions $A(y_i)$, recall (4.5).*

Theorem 4.2.37 is the basis of minimization approaches to find the OT partition; e.g., see [47] for the 2-norm cost. Our numerical method is based on seeking a root of the gradient in (4.12), and we will use Newton's method to this end. Of course, use of Newton's method requires the Jacobian of $\nabla\Phi$ (the Hessian of Φ), and we will use the formulas for the Hessian given in [31], [52], [74]. Indeed, in spite of slightly different notations, the expressions of the Hessian given in these works are actually the same. One of the merits of our work will be to show that our implementation of Newton's method, whereby we exploit the geometrical features of Laguerre's cells and their star-shapedness, is very efficient and –in principle– it leads to arbitrarily accurate results, at least in $\mathbb{R}^2$ and for sufficiently smooth density function ρ .

Theorem 4.2.39. *The Hessian matrix of $\Phi(w)$ is given by:*

$$\begin{aligned} \frac{\partial^2\Phi}{\partial w_i\partial w_j} &= \int_{A_{ij}=A(y_i)\cap A(y_j)} \frac{-\rho(x)}{\|\nabla_x c(x, y_i) - \nabla_x c(x, y_j)\|} d\sigma, \quad j \neq i; \quad i, j = 1, 2, \dots, N, \\ \frac{\partial^2\Phi}{\partial w_i^2} &= - \sum_{j \neq i} \frac{\partial^2\Phi}{\partial w_i\partial w_j}, \quad i = 1, 2, \dots, N, \end{aligned} \tag{4.13}$$

where $d\sigma$ is the $(n-1)$ -dimensional infinitesimal surface element.

Proof. See [31, Theorem 1]. □

Theorem 4.2.40. *Let H be the Hessian matrix of $\Phi(w)$ from Theorem 4.2.39 associated to a feasible w . Then, H has the following properties:*

(1) H is symmetric positive semi-definite, and the vector of all ones is in the nullspace of H ;

(2) H has rank $(N - 1)$. In particular, 0 is a simple eigenvalue.

Proof. The fact that H is symmetric positive semi-definite follows immediately from the structure of the Hessian matrix in (4.13) and the fact that the function $\Phi(w)$ is convex.

Likewise, the fact that $H \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} = 0$ is a simple verification.

Next, let $\lambda_1(H) = 0 \leq \lambda_2(H) \leq \dots \leq \lambda_N(H)$ be the eigenvalues of H . To show statement (2), we need to argue that $\lambda_2(H) > 0$. Consider the undirected graph Γ whose vertices are the target points y_i , and where there is an edge between vertices y_i and y_j (because of our assumptions on μ) if $A(y_i) \cap A(y_j) \neq \emptyset$. Then, this graph is strongly connected by construction (there is a path from any vertex to any other vertex moving along the graph edges), since w is feasible. By construction of H , H is a *generalized Laplacian* associated to this graph (see [45, Section 13.9] for the concept of generalized Laplacian). Then, we can argue like in [45, Lemma 13.9.1] to infer that $\lambda_1(H)$ is a simple eigenvalue. Indeed, take the matrix $H - \eta I$ for some value of η so that the diagonal entries of $H - \eta I$ are non-positive and consider the matrix $-(H - \eta I)$ which has nonnegative entries and it is irreducible, since the graph associated to it is strongly connected (because Γ was). Then, by the Perron-Frobenius theorem, the largest eigenvalue of $-(H - \eta I)$ is simple, and it is its spectral radius and therefore it is the positive value η , and we can take the vector of all 1's as the associated Perron eigenvector. Therefore, 0 is a simple eigenvalue of H and the rank of the Hessian matrix is $(N - 1)$. $\square$

4.3 Laguerre cell computation in 2D

Here we describe the numerical algorithms we implemented. Since all of the computations we will present are in $\mathbb{R}^2$, we will delay discussion of appropriate techniques in $\mathbb{R}^3$ to a

future work, and our algorithmic description below is done relative to 2-d only.

The fundamental task is to compute $\nabla\Phi$ in (4.12) and then form the Hessian in (4.13). In essence, this requires being able to compute the $\mu(A_i)$'s, the boundaries A_{ij} 's, and the line integrals in (4.13). Below, we will describe fast and precise algorithm for these tasks, and for ease of exposition, we are going to make two simplifications in our presentation: (i) the cost function is a p -norm (for $1 < p < \infty$), $c(x, y) = \|x - y\|_p$, and (ii) the domain Ω is the square $[a, b] \times [a, b]$. It is fairly straightforward to adapt the description of our algorithms to the case of cost given by a positive combination of p -norms, and for several choices of Ω . Our computations in Chapter 4.5 will exemplify these two points; see Sections 4.5.3 and 4.5.6.

First of all, we note a main advantage of our technique: it is fully parallelizable. To witness, the computation of $\mu(A(y_i))$ is independent of the computation of $\mu(A(y_j))$, $\forall j \neq i$. Hence, we can formulate the task as follows. Let $Y = y_i$, for some i , then:

$$\text{Compute } \mu(A(Y)) = \int_{A(Y)} d\mu,$$

$$\text{where } A(Y) = \{x \in \Omega \mid \|x - Y\|_p - W \leq \|x - y_j\|_p - w_j, \forall j : y_j \neq Y\}.$$

The second feature of our algorithm is that it exploits star-shapedness result from Section 4.2.1. Thus:

$$A(Y) = \{x \in \Omega \mid \|x - Y\|_p - W \leq \|x - y_j\|_p - w_j, \forall j : y_j \neq Y\}, \quad \text{or}$$

$$A(Y) = \{x = Y + z \in \Omega \mid \|z\|_p - W \leq \|z + Y - y_j\|_p - w_j, \forall j : y_j \neq Y\}.$$

Since $A(Y)$ is star shaped with respect to Y , we can parametrize in a unique way the values on a ray from Y in terms of distance from Y along the direction determined by an angle θ ;

in other words, in the above we can write $z = D(\theta) \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix}$, and thus write for $A(Y)$:

$$A(Y) = \{(\theta, D(\theta)), \theta \in [0, 2\pi] \mid \|D(\theta) \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix}\|_p - W \leq \|D(\theta) \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} + Y - y_j\|_p - w_j, \forall j : y_j \neq Y\}. \quad (4.14)$$

Clearly, the boundary of the Laguerre cell, $\partial A(Y)$, is also parametrizable in terms of the angle θ . Namely, reserving the notation $r(\theta)$ for the points on the boundary, we have

$$x \in \partial A(Y) \iff x = Y + r(\theta) \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \text{ for some unique } \theta \in [0, 2\pi). \quad (4.15)$$

When determining $\partial A(Y)$, in addition to the value of r corresponding to each angle θ , we will monitor also an index k , which indicates the neighboring cell or the portion of the physical boundary for that value of the angle θ :

$$\begin{aligned} x = Y + r(\theta) \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \in A(Y) \cap A(y_j) &\implies k = j \\ x = Y + r(\theta) \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \in A(Y) \cap \partial\Omega &\implies k = \begin{cases} -1, & \text{when } x \text{ is on the bottom side of } \Omega \\ -2, & \text{when } x \text{ is on the right side of } \Omega \\ -3, & \text{when } x \text{ is on the top side of } \Omega \\ -4, & \text{when } x \text{ is on the left side of } \Omega. \end{cases} \end{aligned} \quad (4.16)$$

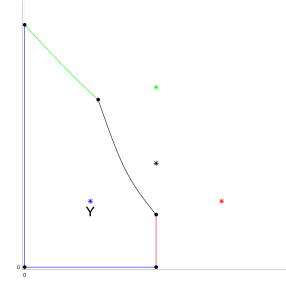
Below, we give a high level description of the algorithms we used to compute $\partial A(Y)$, $\mu(A(Y))$, and so forth, and then at the end we give a descriptions of the algorithms we used in the form of easily replicable pseudo-codes. Although we implemented our algorithms in `Matlab`, by providing these pseudo-codes in a language-independent manner, it will be possible for others to implement the techniques in the environment of their choosing.

4.3.1 Boundary tracing technique

Here we describe how we find the boundary of $A(Y)$. Our approach below is very different from that used in [32]. Namely, in [32], for a given set of weights, the boundary of $A(Y)$ was found by a subdivision algorithm, whereby the boundary itself was covered by (smaller and smaller) squares up to a desired resolution. As explained in [32], this approach was inherently limited to a resolution of the boundary of $\mathcal{O}(\sqrt{\text{eps}})$ accuracy, but did not need to distinguish a-priori the different smooth arcs making up the boundary, which we instead need to do, see below. At the same time, our present technique is able to find the boundary to arbitrary accuracy (in principle).

Remark 4.3.1. *We have to track*

the boundary of $A(Y)$, by making sure we also identify where changes can occur. In essence, the basic task is encoded in the figure on the right, where we identify the smooth arcs and the breakpoints making up $\partial A(Y)$.



4.3.1.1 Shooting Step

The goal of this step is to find a point on $\partial A(Y)$, for a given value θ_0 . That is, we will identify the value r_0 , and the index k in (4.16), such that

$$x = Y + r_0 \begin{pmatrix} \cos(\theta_0) \\ \sin(\theta_0) \end{pmatrix} \in \partial A(Y) .$$

The idea is simple: we move along the ray in the direction of the angle θ_0 until we hit the boundary. Algorithm 1, *Shooting*, shows implementation details of this technique. Algorithm 2, *BisectR*, finds the root of the boundary function $F(r, \theta_0)$ given in (4.17) below

using the bisection method on an interval $[r_a, r_b]$ until $\frac{r_b - r_a}{2}$ is sufficiently small. It is assumed/enforced that $F(r_a, \theta_0) < 0$ and $F(r_b, \theta_0) > 0$. Finally, Algorithm 3, *Bound*, identifies the value on $\partial\Omega$ intersected by the ray from Y in the direction of the angle θ_0 .

Although with the shooting technique of Section 4.3.1.1 we can find, for any value θ , the value of r on $\partial A(Y)$, and the index k , it is more efficient to use a curve continuation technique when the index k is not changing (see Section 4.3.1.3 on this last issue). Our technique is a classical predictor-corrector approach, with tangent predictor and Newton corrector, described next.

4.3.1.2 Predictor-Corrector Method

Here, we exploit the fact that the value of r on the boundary of $\partial A(Y)$ is a smooth function of θ , and a solution of $F(r(\theta), \theta) = 0$ for the function $F(r, \theta)$ below:

$$F(r, \theta) = \left\| r \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \right\|_p - \left\| r \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} + Y - y_j \right\|_p + w_j - W. \quad (4.17)$$

Therefore, a simple application of the implicit function theorem gives that

$$r_\theta = -\frac{F_\theta}{F_r},$$

and this allows for an immediate computation of a tangent predictor, followed by a Newton corrector.

Namely, given θ_0 , $r(\theta_0)$, and an angular increment Δ , we compute $r(\theta + \Delta)$ by tangent

predictor and Newton corrector using F_r and F_θ below:

$$\begin{aligned}
F_r(r, \theta) &= \frac{1}{p} \left\| r \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \right\|_p^{1-p} \left[\frac{\partial}{\partial r} |r \cos(\theta)|^p + \frac{\partial}{\partial r} |r \sin(\theta)|^p \right] - \\
&\quad - \frac{1}{p} \left\| Y - y_j + r \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \right\|_p^{1-p} \left[\frac{\partial}{\partial r} |Y_1 - y_{j,1} + r \cos(\theta)|^p + \frac{\partial}{\partial r} |Y_2 - y_{j,2} + r \sin(\theta)|^p \right], \\
F_\theta(r, \theta) &= \frac{1}{p} \left\| r \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \right\|_p^{1-p} \left[\frac{\partial}{\partial \theta} |r \cos(\theta)|^p + \frac{\partial}{\partial \theta} |r \sin(\theta)|^p \right] - \\
&\quad - \frac{1}{p} \left\| Y - y_j + r \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix} \right\|_p^{1-p} \left[\frac{\partial}{\partial \theta} |Y_1 - y_{j,1} + r \cos(\theta)|^p + \frac{\partial}{\partial \theta} |Y_2 - y_{j,2} + r \sin(\theta)|^p \right].
\end{aligned}$$

Algorithm 4, *PredCorr*, gives implementation details of the PC method. In addition to the value of $r_1 = r(\theta_0 + \Delta)$, this algorithm outputs also a new value for the angular increment Δ , which reflects how close the approximation given by the tangent predictor was to the final value r_1 .

Remark 4.3.2. *We performed extensive experiments also with the trivial, the secant, and the parabolic predictor. But, in our experiments, the tangent predictor outperformed these alternatives, not only in accuracy, but also in overall computational time.*

Through use of the techniques of Sections 4.3.1.1 and 4.3.1.2, we compute smooth arcs of $\partial A(Y)$. To guarantee that these are indeed smooth arcs, we need to locate points where there is a change.

4.3.1.3 Breakpoint Location

During the curve continuation process, we always monitor the index k in (4.16). When this index changes in the interval $[\theta_0, \theta_0 + \Delta]$, this means there is a *breakpoint*, i.e. there is a value $\theta \in [\theta_0, \theta_0 + \Delta]$ such that boundary is changing from one branch to another. There are three things that can happen: (i) the boundary changes from being portion of a curve to a portion of another curve (i.e., the neighboring cell is changing from $A(y_j)$ to another cell $A(y_i)$); (ii) the boundary is changing from being a portion of the curve between two/more

cells to being a portion of the boundary of Ω , or vice versa; and (iii) the boundary changes from one smooth branch of $\partial\Omega$ to another branch of $\partial\Omega$ (presently, this means that two sides of $\partial\Omega$ are meeting at a corner). The goal is to identify the breakpoint in these three cases.

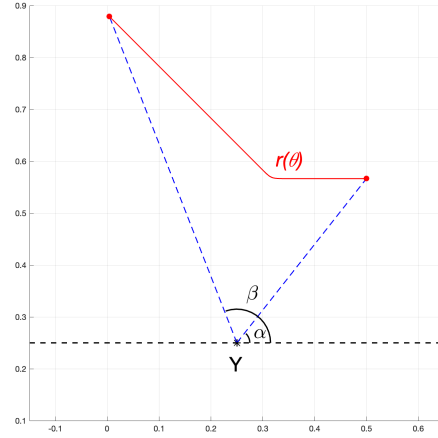
In the first situation, we use bisection method on the interval $[\theta_0, \theta_0 + \Delta]$ (see Algorithm 5, *BisectTheta*). In the second situation, we use a standard root finding technique. In fact, since we can express r in terms of θ , we used the MATLAB function `fzero` to find the root, in the interval $[\theta_0, \theta_0 + \Delta]$, of the function $F(r(\theta), \theta)$ in (4.17); see Algorithm 6, *CurveBound*. Finally, for the last situation, when the breakpoint is at the intersection of two branches of $\partial\Omega$, using the relation between indices it is easy to identify exactly the intersection point of the two branches (in our case, the corner of the square). See Algorithm 7, *BoundBound*.

4.3.2 2-d integral computation

The other main task is to compute the integral, $\mu(A(Y))$. This step is common to all method that want to minimize the functional $\Phi(w)$ in (4.11) (see also (4.12)), whether or not by using Newton's method. The approach taken in [52] and [31], where Newton's method is used for $c(x, y) = \|x - y\|_2^2$, is not detailed in their works, but it is most likely based on triangulation of the cell $A(Y)$ in order to take advantage of the polygonal shape of the latter. In [47], instead, where a quasi-Newton approach is used for $c(x, y) = \|x - y\|_2$, the authors adopt a quadrature based on subdivision of the given domain (the square Ω) in a uniform way and approximate all integrals on this uniform grid. No adaptivity, error estimation, nor error control, is given in [47]. In our algorithm, instead, we will exploit the parametrization in θ expressed in (4.14), in particular, points inside $A(Y)$ along a ray from Y are parametrized by the distance from Y and the angular direction θ of the ray. The main advantages of our method are that it will provide error estimates, work in a fully adaptive fashion, and be much more efficient and accurate than competing techniques. The

main drawback is the need to have a sufficiently smooth density ρ , $\mathcal{C}^4$ to be precise, since we use a composite Simpson's rule. Conceivably, there may be situation when this is a stringent requirement and alternatives will need to be adopted, either by using a quadrature rule requiring less smoothness², or by accepting answers which are accurate to just a few digits.

Consider a smooth arc of the boundary $\partial A(Y)$, comprised between two breakpoints, say at the angles α and β , and let $r(\theta)$ be the smooth function expressing the boundary curve. Then, we have that the integral giving $\mu(A(Y))$ restricted to the values of x falling in the sector determined by $(\alpha, r(\alpha))$ and $(\beta, r(\beta))$, and center Y , is given by (see the figure on the right):



$$\int_{\alpha}^{\beta} \int_0^{r(\theta)} d\mu = \int_{\alpha}^{\beta} \int_0^{r(\theta)} \rho(x) dx = \int_{\alpha}^{\beta} \int_0^{r(\theta)} \rho(D, \theta) D dD d\theta .$$

As a consequence, for $\mu(A(Y))$, we have

$$\begin{aligned} \mu(A(Y)) &= \int_{A(Y)} d\mu = \int_{A(Y)} \rho(x) dx = \sum_{l=1}^{N_l} \int_{\theta_{l,0}}^{\theta_{l,1}} \int_0^{r(\theta)} \rho(D, \theta) D dD d\theta \\ &= \sum_{l=1}^{N_l} \int_{\theta_{l,0}}^{\theta_{l,1}} F(\theta) d\theta , \quad \text{with} \quad F(\theta) = \int_0^{r(\theta)} \rho(D, \theta) D dD, \end{aligned} \tag{4.18}$$

where N_l is the number of breakpoints and $\theta_{l,0}$ and $\theta_{l,1}$ are the angular values corresponding

²We also implemented a version of composite trapezoidal rule, but for smooth ρ it was much less efficient than Simpson's.

to consecutive (in θ) breakpoints. As a result, the integration is always performed along smooth sections of the boundary $\partial A(Y)$ and it involves 1-d integrals (in nested fashion). Because of this fact, to evaluate each integral we implemented an “adaptive composite Simpson’s rule ” with error control, see Algorithm 8, *AdaptSimpson*, and Algorithm 9, *CompSimpson*. Recall that, on an interval $\theta \in [\alpha, \beta]$, where $r(\theta)$ is smooth, the basic composite Simpson’s rule with n subdivisions reads

$$\int_{\alpha}^{\beta} F(\theta) d\theta = \frac{h}{3} \left[F(t_0) + 4 \sum_{i=1}^{n/2} F(t_{2i-1}) + 2 \sum_{i=1}^{n/2-1} F(t_{2i}) + F(t_n) \right] + E(F) ,$$

where $h = \frac{\beta-\alpha}{n}$ and $t_i = \alpha + ih$. When $n = 2k$ for some $k \in \mathbb{N}$, the error term $E(F)$ has the form:

$$E(F) = -\frac{h^4}{180}(\beta - \alpha)F^{(4)}(\xi) ,$$

justifying why we are requiring ρ to be $\mathcal{C}^4$. In the nested form of (4.18), the function $F(\theta)$ is of course approximated by adaptive composite Simpson’s rule as well.

Remark 4.3.3. *Adaptivity with error control turned out to be essential in order to resolve a lot of problems, since the boundary of the Laguerre cells often have large, localized, curvature values, and local refinement turned out to be appropriate. Of course, the caveat to use an accurate integral approximation, as we did, is that we are asking that the density ρ is $\mathcal{C}^4$. See Example 4.5.2-(D) for a case where it is needed to smooth out a given non-smooth density.*

4.4 Newton’s Method and Conditioning of the problem

In the context of semi-discrete optimal transport problem, both [31] and [52] give implementation and testing of Newton’s method when the cost function is the 2-norm square; of course, in this case the effort to compute Laguerre cells is greatly simplified since they happen to be polygons. We are not aware of any effort for the cost functions we considered in

this work, in particular for cost given by a p -norm, $1 < p < \infty$. Indeed, even in the important case of the 2-norm, Hartmann and Schumacher in [47] used a quasi-Newton minimization technique (LBFGMS) and, referring to the expression of the Hessian in (4.13), they remarked “*It remains to be seen, however, if the advantage from using the Hessian rather than performing a quasi Newton method outweighs the considerably higher computational cost due to computing the above integrals numerically ...*”. The main computational goal of ours in this work is to show that Newton’s method can be implemented very efficiently, and we will see in Section 4.5 that is quite a bit more effective than quasi-Newton approaches. In fact, computation of the Hessian itself using (4.13) turns out to be inexpensive (often much less expensive than computing the gradient), ultimately because it requires 1-d integrals to be approximated, rather than 2-d integrals; this is particularly apparent in the increase in cost when approximating the Hessian by divided differences rather than by using the analytic expression (4.13), though of course the divided difference approximations lead to just as accurate end results. To witness, see Table 4.1 below, which refers to Examples (E1)-(E4) of Section 4.5; in Table 4.1, “Forward Difference” refers to the approximate Hessian whose columns are obtained as standard difference quotient: $\frac{\nabla\Phi(w+e_i h)-\nabla\Phi(w)}{h}$, $h = \sqrt{\text{eps}}$. “Centered Difference” refers to the approximate Hessian where each column is obtained as centered difference quotient: $\frac{\nabla\Phi(w+e_i h)-\nabla\Phi(w-e_i h)}{2h}$, $h = (\text{eps})^{1/3}$. When using these divided difference approximation, we explicitly enforce symmetry and the row-sum condition in (4.13).

As usual, an implementation of Newton’s method requires three main ingredients: forming the Hessian, solving for the updates and iterate, and of course providing a good initial guess. We look at these issues next.

4.4.1 The Hessian matrix computation

To compute the Hessian, we exploit the parametrization in terms of the angular variable θ . Namely, looking at the expression of $\frac{\partial^2\Phi}{\partial w_i \partial w_j}$ in (4.13), we note that x is restricted to the

Table 4.1: Iterations refer to Newton’s steps, Difference is the ∞ -norm of the difference between analytic and divided differences Hessians.

	Analytic (4.13)		Forward Difference			Centered Difference		
Example	Iterations	Time	Iterations	Time	Difference	Iterations	Time	Difference
(E1)	2	6.2872	2	9.5086	$2.5724 * 10^{-9}$	2	13.268	$1.1513 * 10^{-10}$
(E2)	6	38.559	6	66.805	$5.3836 * 10^{-8}$	6	96.908	$2.5479 * 10^{-8}$
(E3)	2	8.8570	2	17.115	$2.6139 * 10^{-8}$	2	26.294	$2.4851 * 10^{-10}$
(E4)	3	23.908	3	61.79	$1.2420 * 10^{-7}$	3	109.24	$1.4503 * 10^{-7}$

boundary A_{ij} between $A(y_i)$ and $A(y_j)$, and thus can be given as

$$x(r(\theta), \theta) = y_i + r(\theta) \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \end{pmatrix}$$

for an appropriate range of θ -values, and where $r(\theta)$ refers to the associated part of the boundary curve, that is the distance from y_i along the boundary curve. Thus, the term $\frac{\partial^2 \Phi}{\partial w_i \partial w_j}$ can be rewritten as an arc-length integral:

$$\frac{\partial^2 \Phi}{\partial w_i \partial w_j} = \sum_{\text{smooth arcs}} \int_{\theta_0}^{\theta_1} \frac{-\rho(r(\theta), \theta)}{\|\nabla_x c(x(r(\theta), \theta), y_i) - \nabla_x c(x(r(\theta), \theta), y_j)\|} \sqrt{r^2(\theta) + (r'(\theta))^2} d\theta \quad (4.19)$$

$j \neq i; \quad i, j = 1, 2, \dots, N,$

where $[\theta_0, \theta_1]$ is an interval corresponding to a smooth portion of the boundary $A(y_i) \cap A(y_j)$ and the summation is done along all smooth arcs making up A_{ij} (recall the Figures in Remarks 4.2.30 and 4.3.1). Note that, just like in Section 4.3.1.2, $r'(\theta)$ can be obtained using the Implicit Function theorem on the boundary equation $F(r(\theta), \theta) = 0$, where F is given in (4.17), that is: $r_\theta = -\frac{F_\theta}{F_r}$. The 1-d integrals in (4.19) are again computed by the composite adaptive Simpson’s rule.

Of course, due to the symmetric structure of the Hessian matrix, and the special form of its diagonal (see (4.13)), only the strictly upper-triangular part of H needs to be computed.

Finally, the Hessian will generally be a very sparse matrix (which is especially relevant for large number of target points), since if $A(y_i) \cap A(y_j) = \emptyset$, then $\frac{\partial^2 \Phi}{\partial w_i \partial w_j} = 0$.

4.4.2 Taking a Newton step: solving a system with the Hessian matrix

Assume we have a (guess) w^0 , satisfying the feasibility condition (4.2.36) and also such that its component add up to 0.

Now, a typical Newton step would read

Given w^0 , form $H(w^0)$ solve for s $H(w^0)s = -\nabla \Phi(w^0)$, update $w^1 = w^0 + s$.

However, there are at least two obvious issues that prevent implementing this step as naively as above.

For one thing, $H(w^0)$ is singular, see Theorem 4.2.40. As said before, this is a reflection of the fact that every feasible vector w can be shifted by a multiple of e without changing the associated Laguerre cells $A(y_i)$'s. Singularity of the Hessian has of course been recognized by those working with Newton's method, though dealt with in different ways; for example, in [52] the authors use the pseudo-inverse of H , and in [80] an approach is adopted by holding at 0 one component of w and eliminating the row and column of that index from H . However, we favor an approach that exploits that H has a 1-dimensional nullspace spanned by the vector $e = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} \in \mathbb{R}^N$. So, we select w^0 so that its components add up to 0, and we will keep this property during Newton iteration.

To achieve the above, our approach is to seek the update $s \in \mathbb{R}^N$ by “solving” $H(w^0)s = \nabla \Phi(w^0)$ restricting the solution for s in the (orthogonal) complement to e . In formulas, we form once and for all the orthogonal matrix U below (which is independent of w)

$$U = \begin{bmatrix} q & Q \end{bmatrix} \text{ with } q = \frac{1}{\sqrt{N}}e, \quad Q = \{I_N - uu^T\}_{\{2,3,\dots,N\}} \in \mathbb{R}^{N \times N-1},$$

$$\text{and where } u = \frac{1}{\sqrt{\sqrt{N}-1}}\left(\frac{1}{\sqrt{N}}e - e_1\right) \in \mathbb{R}^N. \quad (4.20)$$

Then, letting $H^0 := H(w^0)$, and $b^0 = -\nabla\Phi(w^0)$, we rewrite the problem as

$$H^0 s = b^0 \rightarrow (U^T H^0 U)(U^T s) = U^T b^0 \rightarrow \begin{bmatrix} 0 & 0 \\ 0 & Q^T H^0 Q \end{bmatrix} U^T s = \begin{bmatrix} q^T b \\ Q^T b \end{bmatrix},$$

since $U^T H(w^0) = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}$ and $H^0 = (H^0)^T$. Therefore, we must have $q^T b = 0$ and thus

$$\begin{bmatrix} 0 & 0 \\ 0 & Q^T H^0 Q \end{bmatrix} \begin{bmatrix} q^T s \\ Q^T s \end{bmatrix} = \begin{bmatrix} 0 \\ -Q^T \nabla\Phi(w^0) \end{bmatrix}$$

from which multiplying both sides by the invertible matrix $\begin{bmatrix} 1 & 0 \\ 0 & (Q^T H^0 Q)^{-1} \end{bmatrix}$, we obtain

$$\begin{bmatrix} q^T \\ Q \end{bmatrix} s = \begin{bmatrix} 0 \\ -[Q^T H^0 Q]^{-1} Q^T \nabla\Phi(w^0) \end{bmatrix} \text{ and eventually our update } s \text{ is given by}$$

$$s = -(H^0)^I \nabla\Phi(w^0), \text{ where } (H^0)^I = Q[Q^T H^0 Q]^{-1} Q^T. \quad (4.21)$$

Since $q^T s = 0$, then $e^T s = 0$, and so the components of $w^1 = w^0 + s$ indeed will add up to 0.

4.4.2.1 Conditioning of semi-discrete OT problems

The second concern is that the updated shift values needs to be feasible, in other words, see Theorem 4.2.36, they must satisfy the condition

$$\min_{i,j} c(y_i, y_j) - |w_i^1 - w_j^1| > 0, \quad \forall i, j = [1, N], i \neq j. \quad (4.22)$$

If this is satisfied, then the step is accepted, otherwise we reduce by a factor of $1/2$ the update s until the criterion (4.22) is satisfied (similar to damped Newton method, see below)

$$s \leftarrow s/2, \quad w^1 \leftarrow w^0 + s.$$

Lemma 4.4.1. *With feasible w^0 , the above updating strategy will eventually lead to a feasible w^1 .*

Proof. We rewrite the feasibility requirement in matrix form as the component-wise inequality $|Aw| < b$, where the matrix $A \in \mathbb{R}^{N(N-1)/2 \times N}$ and the vector $b \in \mathbb{R}^{N(N-1)/2}$ are given by

$$A = \begin{bmatrix} 1 & -1 & 0 & 0 & \cdots & 0 \\ 1 & 0 & -1 & 0 & \cdots & 0 \\ 1 & 0 & 0 & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & 0 & 0 & 0 & \cdots & -1 \\ 0 & 1 & -1 & 0 & \cdots & 0 \\ 0 & 1 & 0 & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix}, \quad b = \begin{pmatrix} c(y_1, y_2) \\ c(y_1, y_3) \\ c(y_1, y_4) \\ \vdots \\ c(y_1, y_N) \\ c(y_2, y_3) \\ c(y_2, y_4) \\ \vdots \end{pmatrix}$$

According to the strategy above, consider $w^1 = w^0 + \frac{1}{2^k}s$, where $s = -H(w^0)^I \nabla \Phi(w^0)$, and recall that w^0 is feasible (hence, it satisfies (4.22)). We claim that w^1 is guaranteed to be feasible for $k = \max \{0, \lceil \max(\log_2 |As| - \log_2 |\varepsilon|) \rceil\}$, where $\varepsilon = b - |Aw^0|$. In fact:

$$|A(w^0 + \frac{1}{2^k}s)| = |Aw^0 + \frac{1}{2^k}As| \leq |Aw^0| + \frac{1}{2^k}|As| = b - \varepsilon + \frac{1}{2^k}|As| \quad \text{and so}$$

$$\frac{1}{2^k}|As| - \varepsilon < 0 \iff |As| < 2^k \varepsilon \iff \log_2 |As| < k + \log_2 \varepsilon \iff k > \max(\log_2 |As| - \log_2 |\varepsilon|)$$

as claimed. □

Remark 4.4.2. *Of course, it is possible that w^0 , albeit feasible, is far from the solution of the nonlinear system; in this case, it is possible that the neighborhood of feasibility of w^0 is small and that to obtain w^1 by the above updating strategy will lead to very tiny steps, and ultimately numerical failure. If this is the case, there are effectively two possibilities: (i) we should give an improved initial guess w^0 , or (ii) the problem is inherently ill-conditioned, in that the neighborhood of feasibility of the exact solution w is just too small, and in*

finite precision one cannot obtain arbitrary accuracy. Below, we elaborate on both of these aspects.

Motivated by 4.22 and Remark 4.4.2, we introduce the *feasibility coefficient* relative to a given feasible w . Namely, we let it be the quantity:

$$\kappa(w) = \min_{i,j} \left[1 - \frac{|w_{ij}|}{c(y_i, y_j)} \right], \quad (4.23)$$

and of course $\kappa(w)$ has to be positive. In our computations, we always monitor this quantity $\kappa(w)$ (during and at the end of the Newton iterations), and we have consistently observed that smaller values of $\kappa(w)$ correspond to more difficult problems; e.g., see Example 4.5.4. We also note that $\kappa(w)$ is a scaling-invariant quantity (i.e., it is not changed by uniformly dilating or restricting the domain Ω , since w rescales in the same way as the cost), and it only depends on the problem itself, that is the location of the target points, the cost function, and the value of w , which ties together the continuous and discrete probability densities. Thus, our proposal is:

Regard $1/\kappa$ as condition number of semi-discrete optimal transport problems.

4.4.3 Getting an initial guess

We have implemented and experimented with several different strategies to obtain a good and feasible initial guesses for solving $\nabla\Phi(w) = 0$.

(i) *Trivial initial guess.* This is the simplest strategy, we take $w^0 = 0$ as an initial guess.

This corresponds to a classical Voronoi diagram (proximity problem), and we call V_i the resulting value of $\mu(A(y_i))$ obtained when using $w = 0$. In spite of its simplicity, this approach often worked remarkably well, especially in conjunction with damped Newton's method (see below).

(ii) *Grid Based.* This technique is based on the work [63], and it is quite similar to the well known Lloyd algorithm. First, consider a grid of meshsize $h = (b - a)/N_h$

to discretize Ω , so that there are $N_h^2 = \frac{(b-a)^2}{h^2}$ squares on the grid. Compute the coordinates of the N_h^2 centers of these squares. For given shift values w , approximate $\mu(A(y_i))$ as

$$\mu(A(y_i)) \approx \sum_{l=1}^{M_i} \rho(x_l) h^2 ,$$

where $x_l, l = 1, 2, \dots, M_i$, are the square centers which are contained in the Laguerre cell $A(y_i)$. Then, using this approximation we can adjust the values $\mu(A(y_i))$ to satisfy $\mu(A(y_i)) = \nu_i$ up to some tolerance by changing shift values. For example, if the approximated $\mu(A(y_i))$ value is greater (smaller) than the required ν_i value, then we force a decrease (increase) in the value of $\mu(A(y_i))$ by increasing (decreasing) $w_k, \forall k \neq i$; we do this by the same increment for all $k \neq i$. Note that the adjustment of $\mu(A(y_i))$ is done one index at a time. In practice, we choose the tolerance to be the area of a square, $|\mu(A(y_i)) - \nu_i| < h^2$, and an initial increment to be $\sqrt{h}$. Depending on the problem, the desired tolerance might not be reachable for all $\mu(A(y_i))$ values. Then, one can either increase the maximum number of allowed iterations, or decrease the mesh size h . See Algorithm 11, *GridInitialGuess*, and Algorithm 12, *GridMeasure*, for implementation details.

The next two approaches are classical homotopy techniques, which we implemented as commonly done when solving nonlinear systems (e.g., see [51]).

(iii) *Trivial Homotopy*. We want to solve $g_1(w(\alpha)) = 0$, where the components of g_1 are given by

$$(g_1(w))_i = \mu(A(y_i)) - [(1 - \alpha)\mu(V_i) + \alpha\nu_i], \quad i = 1, \dots, N, \quad 0 \leq \alpha \leq 1 .$$

At $\alpha = 0, w = 0$ is a solution, which corresponds to the Voronoi diagram $\{V_i\}$ (see (i)), and at $\alpha = 1$ we obtain the problem we are trying to solve. We observe

that the problem can be viewed, for each α , as finding w such that

$$\mu(A(y_i)) = \hat{\nu}_i \equiv \alpha \nu_i + (1 - \alpha) \mu(V_i), \quad i = 1, \dots, N,$$

that is we have a semi-discrete problem where the target density is given by $\sum_i \delta(y_i) \hat{\nu}_i$, which –by virtue of Theorem 4.2.35– is solvable giving a solution $w(\alpha)$ (unique up to adding a constant to all entries)³. We use Newton’s method to solve from $\alpha = 0$ to 1 choosing the steps adaptively beginning with $\Delta^0 = 1/10$ (or $1/100$) and updating it based on the number of Newton iterations k required for convergence at the previous step:

$$\Delta^i = 2^{\frac{4-k}{3}} \Delta^{i-1}, \quad \alpha^i = \alpha^{i-1} + \Delta^i.$$

We note that the Hessian resulting from using Newton’s method is always the one in (4.13), hence it is always singular along the homotopy path, with null vector given by e .

(iv) *Non-Singular Homotopy*. Here, we solve $g_2(w(\alpha)) = 0$, where the components of g_2 are

$$(g_2(w(\alpha)))_i = (1 - \alpha)w + \alpha(\mu(A(y_i)) - \nu_i), \quad i = 1, \dots, N, \quad 0 \leq \alpha \leq 1.$$

At $\alpha = 0$, we have the solution $w = 0$, and for $\alpha > 0$, we can look at the problem as finding w such that $\sum w_i = 0$ and

$$\mu(A(y_i)) = \hat{\nu}_i \equiv \nu_i - \frac{1 - \alpha}{\alpha} w_i, \quad i = 1, \dots, N, \quad 0 < \alpha \leq 1;$$

however, although $\sum_i \hat{\nu}_i = 1$, we now have no guarantee that $\hat{\nu}_i > 0$, and

³This homotopy path is the counterpart to what was used in [65], where the homotopy was done to move from an “easy” source density to the desired source density.

in fact we occasionally had difficulties with this homotopy choice precisely because of this fact. Otherwise, for as long as $\hat{\nu}_i > 0$, we can use Newton’s method to solve for $w(\alpha)$ from 0 to 1, and a main advantage now is that the Hessian (Jacobian of g_2 with respect to w) is non-singular for $\alpha \in [0, 1)$, since it is $(1 - \alpha)I + \alpha H(w(\alpha))$ and $H(\alpha)$ is positive semi-definite from Theorem 4.13.

On the same four problems (E1)-(E4) of Section 4.5, in Table 4.2 we show relative performance of the above strategies to provide an initial guess w^0 . For the homotopy techniques, “Nsteps” refer to the number of homotopy steps, and “Iterations” to the total number of Newton’s iterations. Clearly, the “Grid Based” approach worked much better, and for this reason all results we report on in Section 4.5 were obtained by using the “Grid Based” approach to provide the initial guess w^0 .

Table 4.2: Different strategies to give an initial guess for Newton’s method

	Grid Based $h = 0.05$		Trivial Homotopy		Non-Singular Homotopy	
Example	Iterations	Time	Nsteps, Iterations	Time	Nsteps, Iterations	Time
(E1)	2	7.6242	6, 16	50.767	7, 22	66.755
(E2)	7	40.951	7, 24	126.39	8, 31	148.63
(E3)	2	9.0191	5, 11	44.867	6, 18	68.335
(E4)	3	22.317	6, 19	154.22	7, 25	186.74

A standard modification of Newton’s method is the well known “Damped Newton’s Approach”. This can be viewed as a mean by which we improve on a feasible initial guess w^0 , e.g., $w^0 = 0$, to bring it closer to the solution of $\nabla \Phi(w) = 0$. As a general technique, damped Newton’s is routinely used in solving nonlinear systems, and was also used in [52] for solving semi-discrete optimal transport problems for the 2-norm square cost. Below, we simply give its justification for our case of singular Hessian, particularly given the way we find the Newton’s update.

4.4.3.1 Damped Newton Method

When one has to solve a nonlinear system $F(w) = 0$, and the Jacobian at w^0 is invertible, then it is well known that the Newton direction is a direction of descent for the function $g(w) := \|F(w)\|_2^2 = F(w)^T F(w)$, in the sense that $g(w^0 + \alpha s) < g(w^0)$ for some $0 < \alpha \leq 1$ and s is the solution of $DF(w^0)s = -F(w^0)$. Below, we show that also in our case the Newton direction from (4.21) is an appropriate direction of descent in the subspace complementary to the null-space of H .

Lemma 4.4.3. *If $\nabla\Phi(w) \neq 0$, then the direction s given by (4.21) is a direction of descent for the functional given by*

$$g(w) = (Q^T \nabla\Phi(w))^T (Q^T \nabla\Phi(w)) , \quad (4.24)$$

where $U = \begin{bmatrix} q & Q \end{bmatrix}$ is the orthogonal matrix defined in (4.20). That is, there exists $0 < \alpha \leq 1$ such that $g(w^0 + \alpha s) < g(w^0)$.

Proof. Expand:

$$\begin{aligned} g(w + \alpha s) &= g(w) + \alpha (\nabla g(w))^T s + \mathcal{O}(\alpha^2) \\ &= g(w) + 2\alpha (\nabla\Phi(w))^T Q Q^T H(w) s + \mathcal{O}(\alpha^2) . \end{aligned}$$

Then, using (4.21), we have

$$g(w + \alpha s) - g(w) = -2\alpha (\nabla\Phi(w))^T Q (Q^T H(w) Q) (Q^T H(w) Q)^{-1} Q^T \nabla\Phi(w) + \mathcal{O}(\alpha^2) = -2\alpha g(w) + \mathcal{O}(\alpha^2).$$

Therefore, the claim follows if $g(w) > 0$. Next, we show that $g(w) = 0$ only if $\nabla\Phi(w) = 0$ and the result will follow. In fact:

$$g(w) = 0 \iff Q^T \nabla\Phi(w) = 0 \iff \nabla\Phi(w) = ce , \quad c \in \mathbb{R} ,$$

since the kernel of Q^T is 1-dimensional and spanned by e . Now, since $(\nabla\Phi(w))_i = \mu(A_i) - \nu_i$, $i = 1, \dots, N$, then $\sum_{i=1}^N (\nabla\Phi(w))_i = 0$ and therefore we must have $c = 0$. $\square$

Lemma 4.4.3 justifies use of a full damped Newton's method to solve the problem $\nabla\Phi(w) = 0$. And, in the end, using damped Newton's method revealed to be the best method (among those we implemented) to perform the full iteration in order to solve $\nabla\Phi(w) = 0$. Namely, we always attempt to take the full Newton step, and damp it –if needed– to force a decrease in the functional $g(w)$ in (4.24). Details of our implementation are provided in Algorithm 13, *DampedNewton*. That said, most of the time we found no need to damp the Newton step, see Section 4.5.

4.5 Computational examples

Here we present results of our algorithms. We pay special attention to give Examples that can be replicated by other approaches, and give quantitative results, rather than just pictures.

All of our computations were performed with `Matlab`. The key quantities we monitor are the **Error**, which is $\|\nabla\Phi(w)\|_\infty$ at convergence, the number of Newton **Iterations** and further the number of **Damped** steps if required, the execution **Time**, and the **Feasibility Coefficient** κ as needed. The key quantities by which we control accuracy are two tolerance values, one to control accuracy of the computation of $\mu(A(y_i))$ (and, therefore of the gradient $\nabla\Phi$), the other to measure convergence of the Newton's process, as assessed by either $\|\nabla\Phi\|_\infty$ or the ∞ -norm of the Newton's updates. The default values are 10^{-12} for the approximation of $\mu(A(y_i))$ and 10^{-8} for Newton's convergence.

The next four examples have been used to produce Tables 4.1 and 4.2 as well as Table 4.3 below. They are fairly easy problems, mostly used for testing purposes. For all of them, the domain is $\Omega = [0, 1] \times [0, 1]$, the cost is the 2-norm, $c(x, y) = \|x - y\|_2$, and ρ, ν , and the locations y_i 's are given below. See Figure below for their solution.

$$\rho(x) = 1, \nu = \frac{1}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, y = \left\{ \begin{pmatrix} 0.125 \\ 0.125 \end{pmatrix}, \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \right\}, \quad (E1)$$

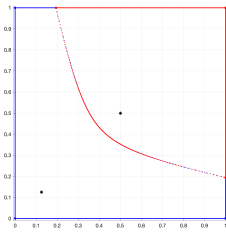
$$\rho(x) = 4x_1x_2, \nu = \frac{1}{4} \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix}, \quad (E2)$$

$$y = \left\{ \begin{pmatrix} 0.25 \\ 0.25 \end{pmatrix}, \begin{pmatrix} 0.75 \\ 0.25 \end{pmatrix}, \begin{pmatrix} 0.5 \\ 0.25(1 + \sqrt{3}) \end{pmatrix}, \begin{pmatrix} 0.5 \\ 0.25(1 + \frac{\sqrt{3}}{3}) \end{pmatrix} \right\}.$$

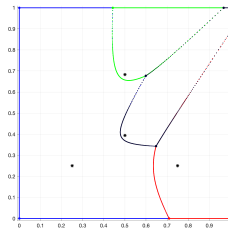
$$\rho(x) = 1, \nu = \frac{1}{5} \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix}, \quad (E3)$$

$$y = \frac{1}{4096} \left\{ \begin{pmatrix} 646 \\ 3491 \end{pmatrix}, \begin{pmatrix} 3480 \\ 3686 \end{pmatrix}, \begin{pmatrix} 1364 \\ 2737 \end{pmatrix}, \begin{pmatrix} 609 \\ 857 \end{pmatrix}, \begin{pmatrix} 2967 \\ 509 \end{pmatrix} \right\}.$$

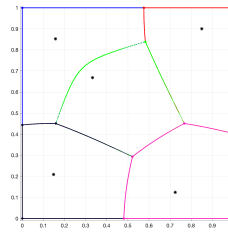
$$\rho(x) = 1, \nu = \frac{1}{8} \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix}, y = 8 \text{ random uniform points using seed } \text{rng}(2). \quad (E4)$$



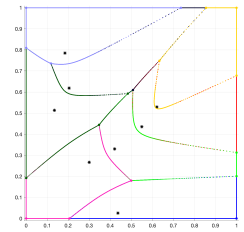
(a) Example (E1)



(b) Example (E2)



(c) Example (E3)



(d) Example (E4)

Example 4.5.1. In Table 4.3, we compare performance of our method with two other techniques, the “boundary method” of [32] and the minimization approach via use of a quasi-Newton’s method as in [47]. In the former case, the results are those obtained by the C++-code kindly provided by J.D. Walsh and the **Time** in this case refers to that of the C++-code, neglecting the final time required to measure the **Error** since the final areas computation is done by our approach. In the case of the BFGS quasi-Newton method we used the tried and true Matlab routine `fminunc`. First of all, with the boundary method we could not reach the required accuracy of $\text{TOL} = 10^{-8}$ and thus had to lower that to 10^{-5} , and even so we had to decrease considerably the minimum allowed grid-size. Secondly, with BFGS we could never obtain answers more accurate than a couple of digits and the code `fminunc` always returned a message that was not able to achieve further reduction in the error.

Table 4.3: Different Methods

	Our method			Boundary method of [32]			Quasi-Newton (BFGS) using <code>fminunc</code>		
Example	Error	Iterations	Time	Error	Grid size	Time	Error	Iterations	Time
(E1)	$4.1959 * 10^{-11}$	2	7.9786	$7.0657 * 10^{-6}$	2^{-15}	1.01	$1.0365 * 10^{-2}$	1	645.81
(E2)	$1.5987 * 10^{-9}$	6	37.807	$7.8626 * 10^{-6}$	2^{-18}	34.5	$5.0087 * 10^{-2}$	7	591.15
(E3)	$7.2028 * 10^{-6}$	1	6.2608	$5.3557 * 10^{-6}$	2^{-16}	6.06	$1.3930 * 10^{-3}$	3	500.25
(E4)	$3.3980 * 10^{-7}$	2	16.170	$8.3461 * 10^{-7}$	2^{-20}	772	$7.6671 * 10^{-3}$	1	430.38

Example 4.5.2 (Example with Different Source Densities). Here we keep $\Omega = [0, 1] \times [0, 1]$, the cost is $c(x, y) = \|x - y\|_2$, ν is uniform, the target points are located at $\left\{ \begin{pmatrix} 0.25 \\ 0.25 \end{pmatrix}, \begin{pmatrix} 0.5 \\ 0.75 \end{pmatrix}, \begin{pmatrix} 0.75 \\ 0.25 \end{pmatrix}, \begin{pmatrix} 0.5 \\ 0.3 \end{pmatrix} \right\}$, and we take four different source densities to

illustrate the impact of ρ on the overall error and execution time. The source densities are

$$\begin{aligned}\rho_1(x) &= 1, \text{ uniform}, \quad \rho_2(x) = 4x_1x_2, \text{ non-uniform}, \\ \rho_3(x) &= \gamma e^{-10(x_1-0.5)^2-10(x_2-0.5)^2}, \text{ Gaussian}, \\ \rho_4(x) &= \begin{cases} \frac{1}{2} & 0 \leq x_1 \leq 0.3, \forall x_2 \\ g(x) & 0.3 < x_1 < 0.7, \forall x_2 \text{ non smooth, smoothed out,} \\ \frac{3}{2} & 0.7 \leq x_1 \leq 1, \forall x_2 \end{cases}\end{aligned}$$

where γ is the normalization constant and

$$g(x_1, x_2) = \frac{1}{2} + \frac{(500x_1(4x_1(175(x_1 - 3)x_1 + 594) - 1203) + 115173)(10x_1 - 3)^5}{131072}.$$

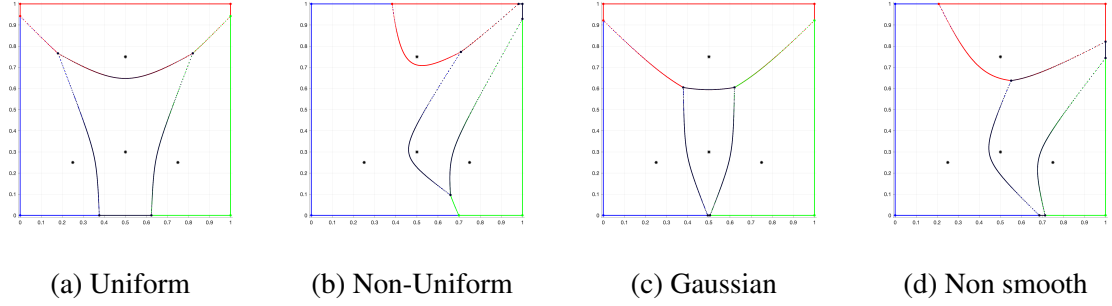
We notice that $g(x)$ is obtained as a C^4 interpolant on a non-smooth density given by $1/2$ before $x_1 = 1/2$ and by $3/2$ past it. For all problems, the initial guess is obtained with grid size $h = 0.05$.

Table 4.4: Example with Different Source Densities

Source Density	Error	Iterations	Damping	Time	Feasibility Coefficient κ
Uniform	$3.7612 * 10^{-9}$	2	0	12.806	0.45594
Non-Uniform	$3.4750 * 10^{-14}$	7	2	46.448	0.13112
Gaussian	$7.3751 * 10^{-11}$	4	0	53.691	0.66334
Non smooth	$4.9682 * 10^{-14}$	6	0	119.99	0.34405

Clearly, the non-smooth case is the most computationally expensive due to the very large values of the derivatives of ρ .

Example 4.5.3 (Different p -norms). Here we want to assess the impact of different p -norms for the cost, in particular we are interested in assessing the impact of using odd value of p (which gives reduced smoothness with respect to the even values of p) and of



“approaching” the ∞ -norm and the 1-norm. Although with costs given by the ∞ -norm and the 1-norm the problem is not well posed (see Example 4.2.33), we will see that as we approach these values the algorithm selects clearly defined Laguerre tessellations, and we conjecture that this is a typical scenario and deserves further study. (Incidentally, also for Example 4.2.33, the approach just outlined select a well defined tessellation).

We fix $\Omega = [0, 1] \times [0, 1]$, $y = \left\{ \begin{pmatrix} 0.25 \\ 0.25 \end{pmatrix}, \begin{pmatrix} 0.5 \\ 0.75 \end{pmatrix}, \begin{pmatrix} 0.75 \\ 0.25 \end{pmatrix} \right\}$, take ρ and ν uniform, and we consider the following cost functions: Cost function:

1. $c(x, y) = \|x - y\|_3$; $c(x, y) = \frac{1}{2}(\|x - y\|_2 + \|x - y\|_4)$; $c(x, y) = \|x - y\|_3 + \|x - y\|_5 + \|x - y\|_7$;
2. toward the ∞ -norm: $c(x, y) = \|x - y\|_{2^k}$, $k = 1, 2, 3, 4, 5$;
3. toward the 1-norm: $c(x, y) = \|x - y\|_{1+2^{-k}}$, $k = 1, 2, 3, 4, 5$.

All results obtained with initial guess from the grid based approach with $h = 0.05$.

Table 4.5: Example with Different p -Norm Cost Functions

Cost Function	Error	Iterations	Damping	Time	Feasibility Coefficient κ
$\ \cdot\ _3$	$1.9444 * 10^{-10}$	2	0	8.6622	0.74508
$\frac{1}{2}(\ \cdot\ _2 + \ \cdot\ _4)$	$1.0988 * 10^{-12}$	2	0	10.756	0.74652
$\ \cdot\ _3 + \ \cdot\ _5 + \ \cdot\ _7$	$8.1481 * 10^{-10}$	2	0	10.742	0.74023

No appreciable difference is seen with the above three different costs, as confirmed by Figure 4.5.

Table 4.6: Approaching the ∞ -norm. Cost is $\|x - y\|_{2^k}$

	Error	Iterations	Damping	Time	Feasibility Coefficient κ
$k = 1$	$1.5999 * 10^{-10}$	2	0	6.539	0.74940
$k = 2$	$7.9542 * 10^{-10}$	2	0	7.6051	0.74083
$k = 3$	$4.1729 * 10^{-9}$	2	0	7.8891	0.73576
$k = 4$	$7.0427 * 10^{-9}$	2	0	8.1167	0.73452
$k = 5$	$6.1689 * 10^{-11}$	3	0	320.97	0.73414

For larger values of p , computation of the integrals becomes expensive because of large derivative values. Figure 4.6 clearly shows the decrease in smoothness.

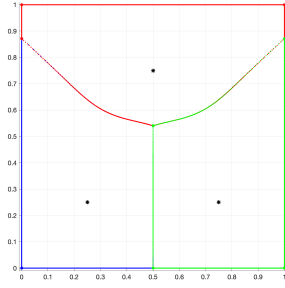
Table 4.7: Approaching the 1-norm. Cost is $\|x - y\|_{1+2^{-k}}$

	Error	Iterations	Damping	Time	Feasibility Coefficient κ
$k = 1$	$5.1627 * 10^{-9}$	2	0	46.634	0.74426
$k = 2$	$1.6436 * 10^{-11}$	3	0	214.18	0.73291
$k = 3$	$3.0620 * 10^{-9}$	2	0	165.3	0.7261
$k = 4$	$3.4457 * 10^{-10}$	2	0	249.13	0.72406
$k = 5$	$2.6122 * 10^{-11}$	2	0	283.11	0.72312

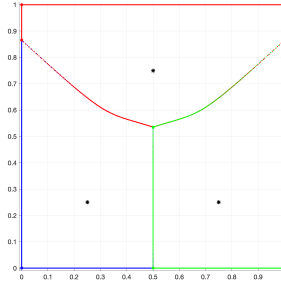
Also for costs approaching 1-norm, there is a clear increase in the computation time, again due to the decreased smoothness of the boundary. Figure 4.7 shows this clearly.

It is very interesting that the value of κ is effectively the same across all of the above experiments, and does not betray ill-conditioning of the problem. This is reflected in the fact that only 2 (or occasionally 3) Newton iterations are required for all of the above costs, and that the value of the Error is small. The difficulties here are entirely due to the decrease in smoothness of the boundary.

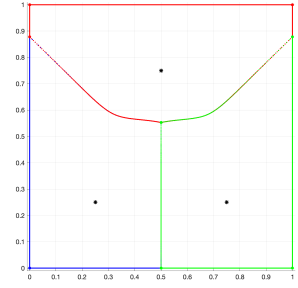
Example 4.5.4 (Impact of Feasibility). This is a difficult problem for all methods we tested,



(a) $\|\cdot\|_3$

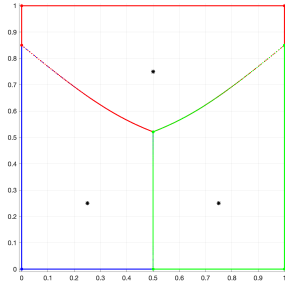


(b) $\frac{1}{2}(\|\cdot\|_2 + \|\cdot\|_4)$

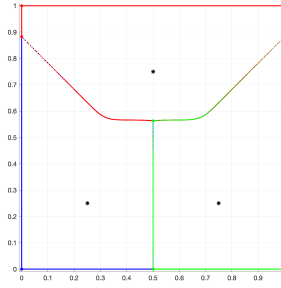


(c) $\|\cdot\|_3 + \|\cdot\|_5 + \|\cdot\|_7$

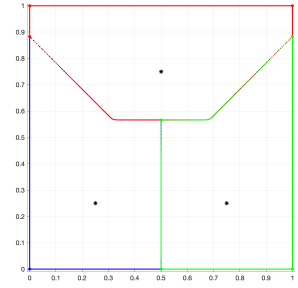
Figure 4.5: Different p -norm costs



(a) $\|\cdot\|_2$



(b) $\|\cdot\|_8$



(c) $\|\cdot\|_{32}$

Figure 4.6: p -norm costs for growing p

in that it becomes arbitrarily ill-conditioned. We have

$$\Omega = [0, 1] \times [0, 1], \quad c(x, y) = \|x - y\|_2, \quad y = \left\{ \begin{pmatrix} 0.25 \\ 0.5 \end{pmatrix}, \begin{pmatrix} 0.75 \\ 0.5 \end{pmatrix} \right\}, \quad \rho(x) = 1,$$

and a non-uniform target density $\nu = (\frac{1}{2^k}, 1 - \frac{1}{2^k})$, $k = [1, \dots, 10]$. Results in Table 4.8 are all obtained with initial guess from the grid-based approach with $h = 0.1$. The tessellation is shown in Figure 4.8.

The increase in time is entirely due to the difficulty in computing $\mu(A_i)$'s. We also note that the initial guess for $k \geq 7$ is just not good, as reflected in the increase in Newton's iterations and the need for damped Newton for $k = 10$.

Example 4.5.5 (Interplay of location, μ and ν). Here we highlight the role played by con-

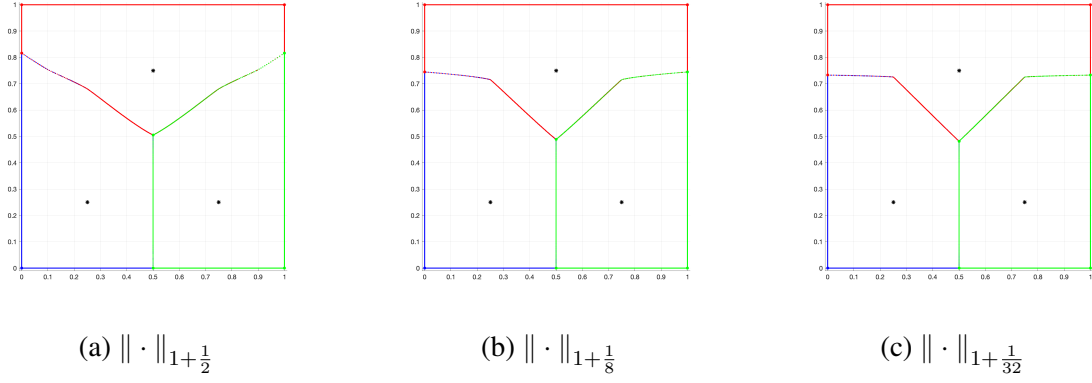


Figure 4.7: p -norm costs for p decreasing toward 1

Table 4.8: Impact of κ on Newton iterations and time

Target Density	Error	Iterations	Damping	Time	Feasibility Coefficient κ
$(\frac{1}{2}, 1 - \frac{1}{2})$	$3.3307 * 10^{-14}$	0	0	2.2765	1
$(\frac{1}{2^2}, 1 - \frac{1}{2^2})$	$4.6629 * 10^{-14}$	3	0	11.363	$4.0243 * 10^{-1}$
$(\frac{1}{2^3}, 1 - \frac{1}{2^3})$	$5.6483 * 10^{-14}$	2	0	9.6461	$2.0029 * 10^{-1}$
$(\frac{1}{2^4}, 1 - \frac{1}{2^4})$	$2.9511 * 10^{-10}$	2	0	10.27	$7.9527 * 10^{-2}$
$(\frac{1}{2^5}, 1 - \frac{1}{2^5})$	$4.1078 * 10^{-14}$	4	0	17.766	$2.4611 * 10^{-2}$
$(\frac{1}{2^6}, 1 - \frac{1}{2^6})$	$9.0354 * 10^{-10}$	3	0	20.834	$6.6039 * 10^{-3}$
$(\frac{1}{2^7}, 1 - \frac{1}{2^7})$	$2.7367 * 10^{-11}$	9	0	59.625	$1.6834 * 10^{-3}$
$(\frac{1}{2^8}, 1 - \frac{1}{2^8})$	$2.7842 * 10^{-12}$	10	0	120.6	$4.2294 * 10^{-4}$
$(\frac{1}{2^9}, 1 - \frac{1}{2^9})$	$3.9706 * 10^{-12}$	10	0	201.91	$1.0587 * 10^{-4}$
$(\frac{1}{2^{10}}, 1 - \frac{1}{2^{10}})$	$1.2863 * 10^{-10}$	12	4	1759	$2.6475 * 10^{-5}$

sistency of the location of the target points with respect to the continuous and discrete densities. We fix

$$\Omega = [0, 1] \times [0, 1], \quad c(x, y) = \|x - y\|_2, \quad y = \left\{ \begin{pmatrix} 0.8 \\ 0.8 \end{pmatrix}, \begin{pmatrix} 0.8 \\ 0.9 \end{pmatrix}, \begin{pmatrix} 0.9 \\ 0.9 \end{pmatrix}, \begin{pmatrix} 0.9 \\ 0.8 \end{pmatrix} \right\},$$

and select three different densities

(A) Uniform source and uniform target. Here, the location of the target points is not consistent with either ρ or ν .

(B) Uniform source ($\rho(x) = 1$) and non-uniform target ($\nu = (0.75, 0.1, 0.05, 0.1)^T$).

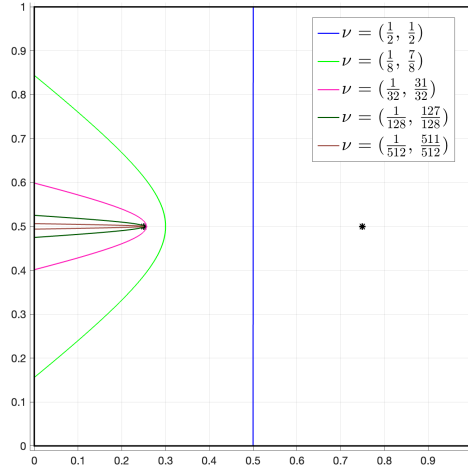


Figure 4.8: Impact of small κ on the tessellation

Here, the location of the target points is consistent with ν but not with μ .

(C) Non-Uniform source ($\rho(x) = 16x_1^3x_2^3$) and uniform target ($\nu_i = \frac{1}{4}, \forall i$). Here, the location of the target points is consistent with μ but not with ν .

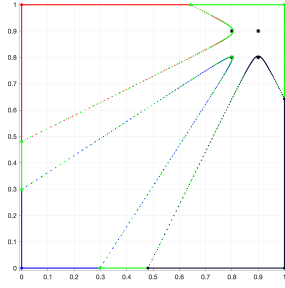
For all cases, the initial guess was obtained with the grid-based approach with $h = 0.05$.

Table 4.9: Interplay between Source and Target Densities with Location of Points

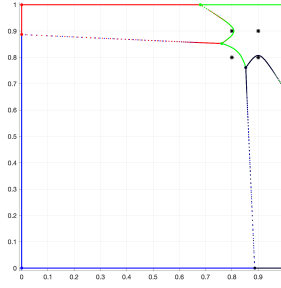
Densities	Error	Iterations	Damping	Time	Feasibility Coefficient κ
Uniform to Uniform	$3.2677 * 10^{-9}$	2	0	89.095	0.02198
Uniform to Non-Uniform	$1.2056 * 10^{-12}$	3	0	45.295	0.14509
Non-Uniform to Uniform	$8.3026 * 10^{-9}$	3	0	144.78	0.86597

The small value of κ in case (A) reflects the inconsistent location of the target points with respect to μ and ν . The high execution time for case (C), instead is entirely due to the cost of computing $\mu(A_i)$'s by our algorithm because of the change of variable we do for (x_1, x_2) , namely $\rho(x_1, x_2) = \rho(Y_1 + D \cos(\theta), Y_2 + D \sin(\theta))$.

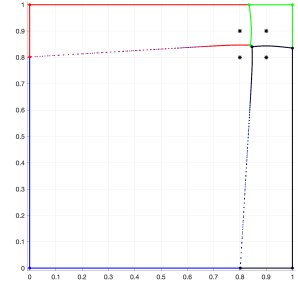
Example 4.5.6 (Different Domains). Here we show that our algorithms can be easily adapted to work with non-square domains, with no noticeable difference in performance.



(a) Uniform Source and Target Densities



(b) Uniform Source and Non-Uniform Target Densities



(c) Non-Uniform Source and Uniform Target Densities

Figure 4.9: Interplay between Source and Target Densities with Location of Points

For y we take 10 random points in domain Ω_i , obtained by retaining the first 10 random points uniformly distributed in the square circumscribed to Ω_i that fall inside Ω_i . Further, we take $c(x, y) = \|x - y\|_2$, ν is uniform, and so is μ : $\rho(x) = \frac{1}{|\Omega_i|}$, $\forall i$. As domains we considered:

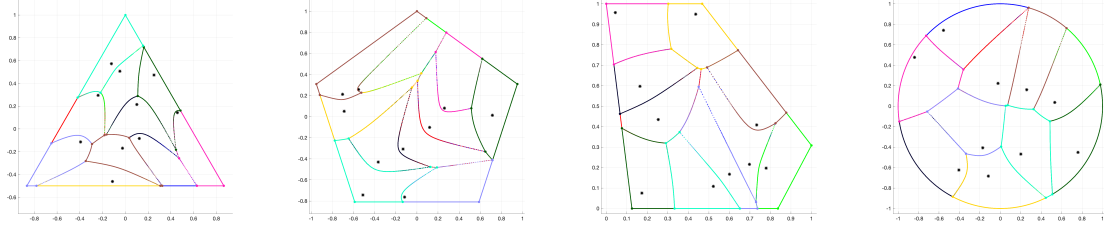
- (A) Ω_1 is the equilateral triangle centered at the origin with circumradius of 1;
- (B) Ω_2 is the regular pentagon centered at the origin with circumradius equal to 1;
- (C) Ω_3 is an irregular pentagon of centroid at (0.44239, 0.45993) and circumradius equal to 0.69813;
- (D) Ω_4 is the circle centered at the origin with radius equal to 1;

As initial guesses we take $w^0 = 0$.

Table 4.10: Different domains and uniform source density

Domain	Error	Iterations	Damping	Time	Feasibility Coefficient κ
Equilateral Triangle	$4.2979 * 10^{-11}$	6	0	97.79	0.07999
Regular Pentagon	$2.7436 * 10^{-10}$	6	0	130.1	0.11848
Irregular Pentagon	$5.2235 * 10^{-11}$	5	0	59.33	0.25617
Circle	$2.0969 * 10^{-11}$	4	0	22.251	0.51896

Interestingly, the most difficult problems are those with the most regular domains, equilateral triangle and regular pentagon. However, all cases are solved fairly easily. See Figure 4.10.



(a) Equilateral Triangle (b) Regular Pentagon (c) Irregular Pentagon (d) Circle

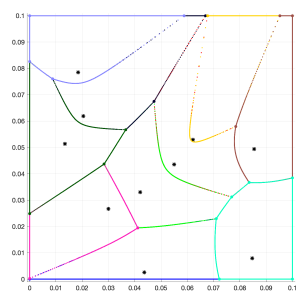
Figure 4.10: Examples with Different domains and uniform source density

Example 4.5.7 (Impact of Rescaling). *The purpose here is to show that the quantity κ is invariant under scaling of the data (domain and target points). For y , we take 10 random points uniformly distributed in the unit square using `rng(2)`, $c(x, y) = \|x - y\|_2$ and ρ and ν are uniform. As domains we take $\Omega_1 = (0, \frac{1}{10}) \times (0, \frac{1}{10})$, $\Omega_2 = [0, 1] \times [0, 1]$, $\Omega_3 = (0, 2) \times (0, 2)$, $\Omega_4 = (0, 4) \times (0, 4)$, and $\Omega_5 = (0, 8) \times (0, 8)$. Initial guess is grid based with $h = 0.05$.*

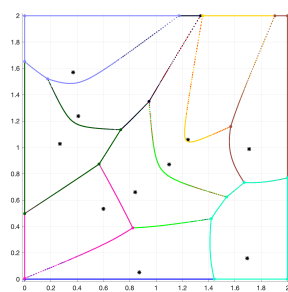
Table 4.11: Impact of rescaling

Domain	Error	Measure Error	Iterations	Damping	Time	Feasibility Coefficient κ
$(0, \frac{1}{10}) \times (0, \frac{1}{10})$	$7.7205 * 10^{-10}$	$7.9847 * 10^{-13}$	5	1	44.831	0.11236
$[0, 1] \times [0, 1]$	$8.2115 * 10^{-14}$	$7.9869 * 10^{-13}$	3	0	23.674	0.11236
$(0, 2) \times (0, 2)$	$4.8583 * 10^{-13}$	$4.8017 * 10^{-12}$	3	0	20.158	0.11236
$(0, 4) \times (0, 4)$	$2.2514 * 10^{-12}$	$1.2774 * 10^{-11}$	3	0	23.925	0.11236
$(0, 8) \times (0, 8)$	$2.3217 * 10^{-11}$	$7.5335 * 10^{-11}$	3	0	89.299	0.11236

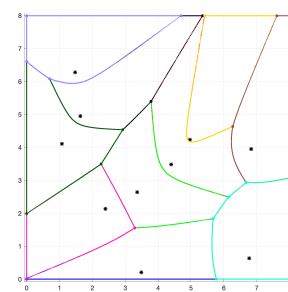
All cases are of the same level of difficulty, see Table 4.11, the increase in expense for the larger square is exclusively due to the requirement of a small absolute error for the areas.



(a) $\Omega_1 = (0, \frac{1}{10}) \times (0, \frac{1}{10})$



(b) $\Omega_2 = (0, 2) \times (0, 2)$



(c) $\Omega_4 = (0, 8) \times (0, 8)$

Figure 4.11: Effect of Rescaling

4.6 Semi-discrete OT problems in $\mathbb{R}^n$ ($n \geq 3$)

In this section we will consider problem in $\mathbb{R}^n$ with $n \geq 3$. Let $\Omega \subset \mathbb{R}^n$ be a domain satisfying Assumption 3.2.1 and $y_i \in \Omega^\circ$, $i = 1, 2, \dots, N$, be N distinct target points. Then the theoretical results from Section 4.2 still hold, in particular, given the cost function satisfying Restriction 4.2.15, the Laguerre cells are star-shaped, the boundaries between cells have zero Lebesgue measure, and the minimization function $\Phi(w)$ in (4.11) is convex and $\mathcal{C}^1$ -smooth. As a result, one can parametrize a Laguerre cell using $(n - 1)$ angular coordinates. However, the boundary tracing and the integration over the boundaries become significantly more expensive compared with the 2-d case to reach the same level of accuracy due to the lack of efficient methods in higher dimensions. Thus, for higher dimensions, we propose to use the Gradient Descent technique to solve the problem coupled with the Monte Carlo method to approximate the values of the the integrals.

We will use the general Gradient Descent to update the shift vector w :

$$w^{(k)} = w^{(k-1)} - \frac{1}{2^s} \nabla \Phi(w^{(k-1)}),$$

where $s \geq 0$ is the smallest integer such that $\Phi(w^{(k)}) < \Phi(w^{(k-1)})$. Details of the implementation are provided in Algorithm 14, *GradientDescent*. Next, recall the standard Monte Carlo method: given M random points $x_i \in \Omega \subset \mathbb{R}^n$, $i = 1, 2, \dots, M$, where K of those points are in the region $A \subset \Omega$, the integration over A with respect to the function $F(x)$ can be approximated as follows:

$$I = \int_A F(x) dx \approx |A| \frac{1}{K} \sum_{k=1}^K F(x_k) \approx \frac{K}{M} \frac{1}{K} \sum_{k=1}^K F(x_k) = \frac{1}{M} \sum_{k=1}^K F(x_k),$$

where $x_k \in A$, $k = 1, 2, \dots, K$. Then, the gradient vector $\nabla \Phi(w)$ in (4.12) is approxi-

mated by:

$$\begin{aligned}\mu(A(y_i)) &= \int_{A(y_i)} \rho(x) dx \approx \frac{1}{M} \sum_{k=1}^{K_i} \rho(x_k) = I_{y_i} \implies \\ \implies \{\nabla \Phi(w)\}_i &= \mu(A(y_i)) - \nu_i \approx \frac{1}{\sum_{j=1}^N I_{y_j}} I_{y_i} - \nu_i,\end{aligned}$$

where $x_k \in A(y_i) \iff i = \arg \min_j c(x_k, y_j) - w_j$, $k = 1, 2, \dots, K_i$, and $\sum_{i=1}^N K_i = M$. Note that we need to normalize I_{y_i} values since the condition $\int_{\Omega} d\mu = \sum_{i=1}^N \nu_i = 1$ needs to be satisfied. Similarly, one can approximate the minimization function value $\Phi(w)$ from (4.11):

$$\begin{aligned}\int_{A(y_i)} [c(x, y_i) - w_i] d\mu &= \int_{A(y_i)} [c(x, y_i) - w_i] \rho(x) dx \approx \frac{1}{M} \sum_{k=1}^{K_i} [c(x_k, y_i) - w_i] \rho(x_k) = J_{y_i}, \\ \Phi(w) &= - \sum_i^N \int_{A(y_i)} [c(x, y_i) - w_i] d\mu - \sum_i^N w_i \nu_i \approx - \sum_i^N J_{y_i} - \sum_i^N w_i \nu_i.\end{aligned}$$

4.6.1 Computational examples in 3-d

Let $n = 3$ and fix tolerance $\text{TOL} = 10^{-4}$. Recall that the order of convergence for Monte Carlo methods is $O(M^{-\frac{1}{2}})$ (e.g., see [23]). As a result, the number of sample points is fixed to be $M = 1.2 \times 10^8$. Then we can generate M random points x_k in $\Omega = [0, 1]^3$ and N random target points y_i in $\Omega^\circ = (0, 1)^3$ using seed `rng(1)`. In addition, the target densities will be either uniform ($\nu_i = \frac{1}{N}$, $\forall i$), or randomly generated values with posterior normalization (i.e. $\sum_i \nu_i = 1$ is enforced). Finally, we will test the Gradient Descent method with four source densities:

$$\begin{aligned}\rho_1(x) &= 1, \quad \rho_2(x) = 8x_1x_2x_3, \quad \rho_3(x) = \frac{1}{3}(x_1 + 2x_2 + 3x_3), \\ \rho_4(x) &= ce^{-(x_1-0.25)^2 - (x_2-0.5)^2 - (x_3-0.75)^2},\end{aligned}$$

where c is the normalization constant. In all our experiments we will start with the initial guess $w^{(0)} = 0$. Similar to 2-d problems, we report the **Error**, which is $\|\nabla \Phi(w)\|_\infty$ at convergence, the number of Gradient Descent **Iterations** and further the number of **Damped** steps if required, and the execution **Time**. Tables 4.12, 4.13, 4.14, and 4.15 demonstrate the results for these problems. As it can be seen from the reported error values, the Gradient Descent method has converged to the desired tolerance in all examples. In addition, Figure 4.12 shows the Laguerre cells at the convergence for the problem with uniform density with 2 and 4 target points.

Table 4.12: Problem with $\rho_1(x) = 1$

	Uniform Target Densities				Non-Uniform Target Densities			
N	Iterations	Damping	Error	Time	Iterations	Damping	Error	Time
2	4	4	$9.0983 * 10^{-5}$	276.87	33	1	$9.6278 * 10^{-5}$	1375
4	17	16	$9.4292 * 10^{-5}$	1152.7	17	17	$4.6346 * 10^{-5}$	1186.2
7	51	94	$9.1093 * 10^{-5}$	4902.7	70	27	$8.6454 * 10^{-5}$	4269.4
10	25	9	$8.7442 * 10^{-5}$	1532.4	47	36	$7.2668 * 10^{-5}$	3344.8

Table 4.13: Problem with $\rho_2(x) = 8x_1x_2x_3$

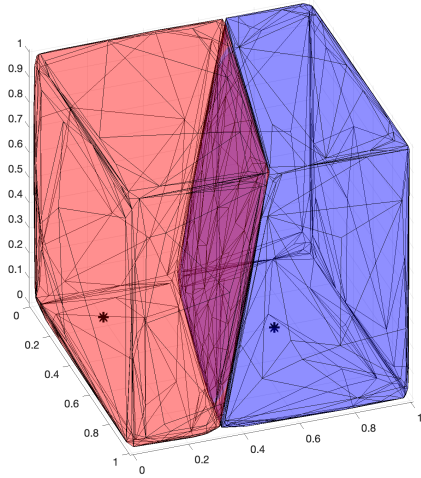
	Uniform Target Densities				Non-Uniform Target Densities			
N	Iterations	Damping	Error	Time	Iterations	Damping	Error	Time
2	7	13	$1.8391 * 10^{-5}$	609.37	6	6	$7.861 * 10^{-5}$	401.2
4	29	54	$9.2661 * 10^{-5}$	2632.5	40	77	$7.5526 * 10^{-5}$	3625.9
7	69	189	$7.2617 * 10^{-5}$	8814.1	82	46	$9.2285 * 10^{-5}$	5086.9
10	58	106	$9.8097 * 10^{-5}$	5890.2	71	162	$9.2499 * 10^{-5}$	8228.3

Table 4.14: Problem with $\rho_3(x) = \frac{1}{3}(x_1 + 2x_2 + 3x_3)$

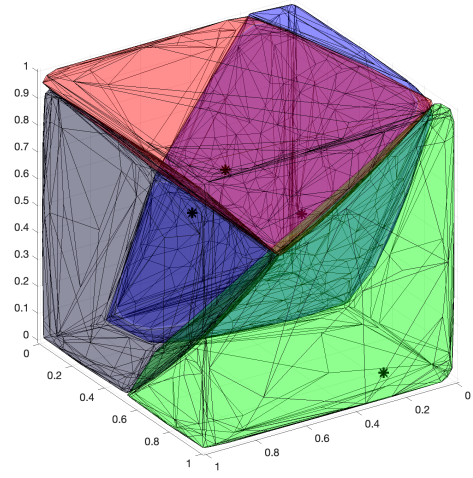
	Uniform Target Densities				Non-Uniform Target Densities			
N	Iterations	Damping	Error	Time	Iterations	Damping	Error	Time
2	9	9	$4.4604 * 10^{-5}$	601.07	24	1	$9.2972 * 10^{-5}$	1026.8
4	10	11	$2.8853 * 10^{-5}$	727.31	14	13	$6.7456 * 10^{-5}$	969.47
7	32	56	$9.8181 * 10^{-5}$	3006.5	56	15	$8.5037 * 10^{-5}$	3058.1
10	30	17	$7.5772 * 10^{-5}$	2016.2	55	73	$8.2109 * 10^{-5}$	4857.6

Table 4.15: Problem with $\rho_4(x) = ce^{-(x_1-0.25)^2-(x_2-0.5)^2-(x_3-0.75)^2}$

	Uniform Target Densities				Non-Uniform Target Densities			
N	Iterations	Damping	Error	Time	Iterations	Damping	Error	Time
2	5	5	$4.6182 * 10^{-5}$	356.74	34	1	$8.9955 * 10^{-5}$	1468.1
4	14	13	$6.4647 * 10^{-5}$	975.47	29	28	$2.5772 * 10^{-5}$	2044.6
7	39	70	$7.7648 * 10^{-5}$	3838.4	66	23	$8.7482 * 10^{-5}$	3814.9
10	27	13	$8.7765 * 10^{-5}$	1807.1	50	45	$5.0213 * 10^{-5}$	3915



(a) $N = 2$



(b) $N = 4$

Figure 4.12: Problem with uniform source density

4.7 Application on the congressional districts of Georgia

4.7.1 Problem statement

Defining boundaries between regions, states, districts, or other entities is inherently complex due to the various interests involved. However, there are instances where this complexity may be excessive. Here, we will examine a specific case related to the congressional districts in Georgia. The fundamental principles by which the congressional districts are drawn are: (i) equal number of people living in each district, (ii) connectedness of the district, and (iii) compactness of the district. We claim that the algorithm of Section 4 with a minor modification can be used to draw the boundaries between the congressional districts, such that these principles are satisfied.

Let $B_j, j = 1, 2, \dots, 7433$, be the block groups of the state of Georgia. Our task is to find 14 congressional districts $D_i, i = 1, 2, \dots, 14$ made up of the block groups and satisfying the fundamental principles mentioned above. The block groups are the smallest units of the problem, thus, each block group is assigned to a unique congressional district. In addition, the following data for each block group is available: the population P_j , the polygonal boundary, and the center point x_j . The population data was obtained from the *Policy Map* website and the geographical information for each block group was obtained from the *Census* website. Finally, the domain Ω representing the state of Georgia is approximated using an irregular polygon with 5 sides.

4.7.2 Algorithm and results

4.7.2.1 Step 1. Finding the Laguerre cells

To reduce the complexity of the problem we begin by splitting the domain Ω into 7 regions $A_i, i = 1, 2, \dots, 7$, with respect to the 2-norm cost function and the uniform source density, $\rho(x) = \frac{1}{|\Omega|}$. The region A_i is the Laguerre cell where the location of the target point y_i is found in the process. Ideally, each of these 7 regions will have equal population. To enforce

this constraint we proceed as follows.

We say that the block group B_j is in the cell A_i if its center x_j of B_j is contained in A_i . In addition, we define the district D_i as the union of the block groups contained in the Laguerre cell A_i . As a result, one can compute the actual population of the district D_i by adding the population of all block groups in it: $p_i = \sum_{j:x_j \in A_i} P_j$, where P_j is the population of the block group B_j . Then, our first goal is to find the Laguerre cells A_i , $i = 1, 2, \dots, 7$, such that the validation error $e_i = \frac{p_i - \bar{p}}{\bar{p}}$, $i = 1, 2, \dots, 7$, is smallest, where $\bar{p} = \frac{1}{7} \sum_j P_j$. This condition enforces the equal population restriction. By construction, the Laguerre cells are star-shaped with respect to the target point, which implies that they are connected. Lastly, to satisfy the compactness condition, we move the target points to the center of the Laguerre cell, whenever there is no decrease in the value of the validation error.

To start the process we need to set up the locations for the target points. During experiments we observed that the initial locations of the target points have significant effect on the results in Step 1. Based on these computations we concluded that one is looking for the target points which resemble the population density of Georgia but are not clustered in any one region. Many choices of initial 7 points are possible and one obtains different valid maps with basically equal population. Next, we initialize the target measures ν_i , $i = 1, 2, \dots, 7$, with respect to the Voronoi partition: let $w = 0$ and compute $\nu_i = \mu(A(y_i)) = \frac{|A_{y_i}|}{|\Omega|}$. Then, we compute the validation error e at each iteration and adjust $\nu^1 = \nu^0 - \frac{1}{7^k} e$, where k is some positive integer. In our experiments we started with $k = 2$ and doubled its value every 50 iterations. In addition, if there is no improvement in the validation error, the target points y_i are moved to the center of their corresponding Laguerre cells⁴. We refer to Algorithm 15, *DistrictsStep1*, for more details on the implementation.

For our computational experiments we have used the following two choices for the

⁴We have used the MATLAB function `centroid` for this task.

initial locations of the target points (see Figure 4.13a):

$$y = \left\{ \begin{pmatrix} 0.2530 \\ 0.7421 \end{pmatrix}, \begin{pmatrix} 0.3994 \\ 0.4994 \end{pmatrix}, \begin{pmatrix} 0.9301 \\ 0.3124 \end{pmatrix}, \begin{pmatrix} 0.4597 \\ 0.7597 \end{pmatrix}, \begin{pmatrix} 0.2568 \\ 0.8026 \end{pmatrix}, \begin{pmatrix} 0.0692 \\ 0.8332 \end{pmatrix}, \begin{pmatrix} 0.1723 \\ 0.7102 \end{pmatrix} \right\}, \quad (C1)$$

$$y = \left\{ \begin{pmatrix} 0.3317 \\ 0.6020 \end{pmatrix}, \begin{pmatrix} 0.5621 \\ 0.6891 \end{pmatrix}, \begin{pmatrix} 0.1691 \\ 0.7229 \end{pmatrix}, \begin{pmatrix} 0.8507 \\ 0.3168 \end{pmatrix}, \begin{pmatrix} 0.2125 \\ 0.3050 \end{pmatrix}, \begin{pmatrix} 0.2338 \\ 0.8966 \end{pmatrix}, \begin{pmatrix} 0.5670 \\ 0.2342 \end{pmatrix} \right\}. \quad (C2)$$

In Example C1 the locations of target points correspond to the locations of the following cities in Georgia: Macon, Savannah, Athens, Brookhaven, Milton, Rome, and Douglasville. Figure 4.13b shows the positions of the target points at the end of Step 1.

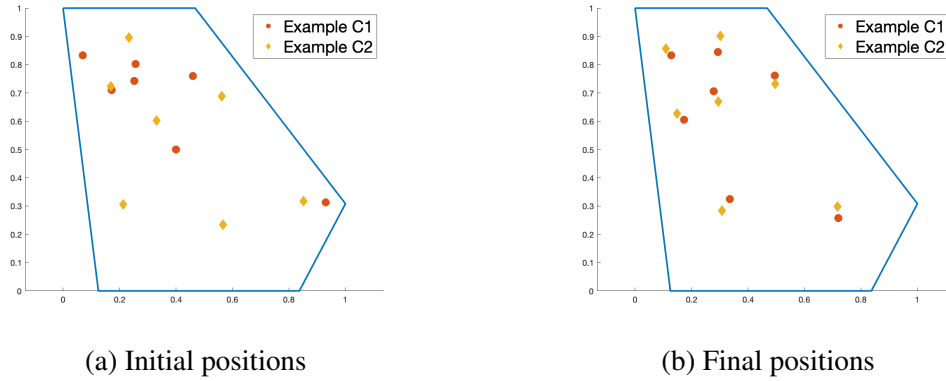


Figure 4.13: Locations of the target points

Table 4.16 provides the population values for districts after Step 1, where deviation in percentage is e_i and deviation in number of people is $p_i - \bar{p}$. Figure 4.14 displays the Laguerre cells A_i and the districts D_i after Step 1 is completed. To be clear, the districts are identified by color, while the Laguerre cells are identified by their boundaries (in black).

4.7.2.2 Step 2. Adjust Laguerre cells: Merge/Purge neighboring block groups

In this step, the goal is to improve the results of Step 1 by adjusting the block groups which are on the boundary between two or more Laguerre cells. If the district D_i has less than expected population, i.e. less than $\bar{p}$, then one can reassign the block groups B_j , which are on the boundary between A_i and A_k , for some k , as long as the population of

Table 4.16: Population after Step 1

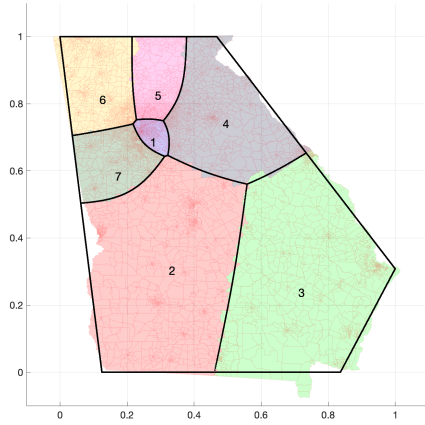
	Example (C1)		Example (C2)	
District	Population	Deviation	Population	Deviation
1	1532743	2470.4 (0.16 %)	1530532	259.4 (0.01 %)
2	1545353	15080 (0.98 %)	1529679	593.5 (−0.03 %)
3	1514960	−15313 (−1.00 %)	1530006	266.5 (−0.01 %)
4	1534661	4388.4 (0.28 %)	1512737	17535 (−1.14 %)
5	1529010	−1262.6 (−0.08 %)	1527905	2367.5 (−0.15 %)
6	1528424	−1848.6 (−0.12 %)	1530258	−14.5 (−0.01 %)
7	1526757	−3515.6 (−0.22 %)	1550791	20518 (1.34 %)

the district D_k is larger than $\bar{p}$. See Algorithm 16, *DistrictsStep2*, for the implementation details of this step. Note that at the end of Step 2, it is necessary to check and, if required, manually reassign block groups that do not comply with the connectedness restriction, while maintaining the same population values.

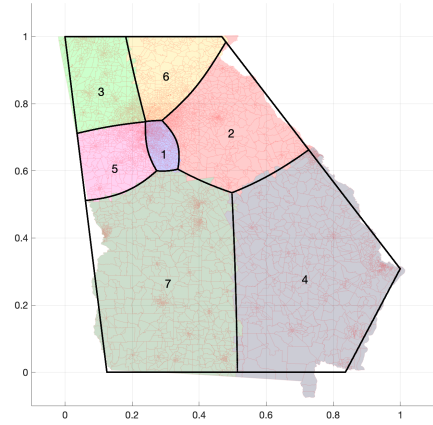
As it can be seen from Table 4.17, after this Step 2 the population of each district is much more uniform and the max-deviation has improved significantly and it is below 100 people. Figure 4.15 shows the districts after the adjustments of Step 2.

Table 4.17: Population after Step 2

	Example (C1)		Example (C2)	
District	Population	Deviation	Population	Deviation
1	1530326	53.429 ($3.49 * 10^{-3}$ %)	1530214	−58.5714 ($−3.82 * 10^{-3}$ %)
2	1530265	−7.5714 ($−4.94 * 10^{-4}$ %)	1530277	4.4286 ($2.89 * 10^{-4}$ %)
3	1530238	−34.571 ($−2.25 * 10^{-3}$ %)	1530362	89.4286 ($5.84 * 10^{-3}$ %)
4	1530280	7.4286 ($4.85 * 10^{-4}$ %)	1530272	−0.5714 ($−3.73 * 10^{-5}$ %)
5	1530275	2.4286 ($1.58 * 10^{-4}$ %)	1530254	−18.5714 ($−1.21 * 10^{-3}$ %)
6	1530218	−54.571 ($−3.56 * 10^{-3}$ %)	1530315	42.4286 ($2.77 * 10^{-3}$ %)
7	1530306	33.429 ($2.18 * 10^{-3}$ %)	1530214	−58.5714 ($−3.82 * 10^{-3}$ %)



(a) Example (C1)



(b) Example (C2)

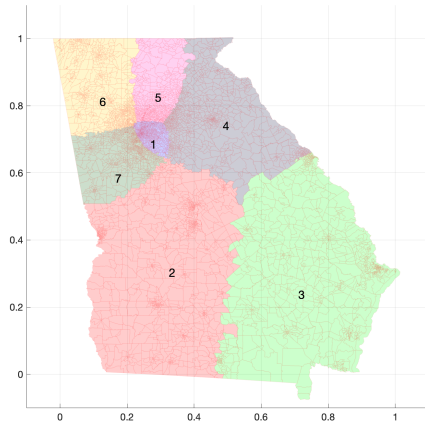
Figure 4.14: Laguerre cells and districts after Step 1

4.7.2.3 Step 3. Split the districts

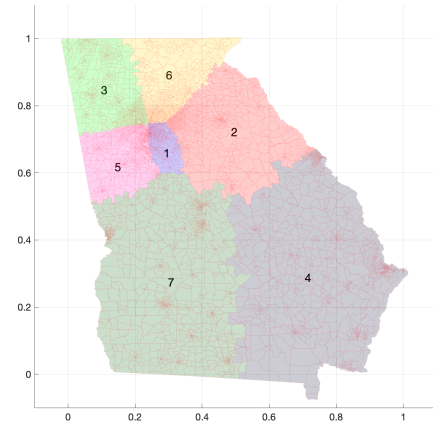
This is the last step of our process. We split each district from Step 2 into 2 sub-districts in order to get to 14 congressional districts in total. Recall that one still needs to enforce the equal population, connectedness, and compactness of the sub-districts. As a result, we are going to pick two target points y_a and y_b within the district D_i such that y_a and y_b are the center points for the block groups in D_i with maximum distance between them: $(a, b) = \arg \max_{k, l: B_k, B_l \in D_i} \|x_k - x_l\|_2$. This choice of points helps to satisfy connectedness and compactness conditions of the new districts. Next, to get the equal population of each sub-district, we use a linear assignment algorithm⁵ to map each block group in D_i to a unique sub-district, such that the population of the new split districts are equal. Refer to Algorithm 17, *DistrictsStep3*, for details on the implementation of Step 3. Just like in Step 2, it is important to review and potentially reassign block groups that do not meet the connectedness requirement after completing Step 3, all while keeping the population values.

From Table 4.18, one can see that we were able to keep the population deviation well

⁵We used MATLAB function `intlinprog`



(a) Example (C1)



(b) Example (C2)

Figure 4.15: Districts after Step 2

below 100 people after the splitting process. Figures 4.16 and 4.17 illustrate the 14 congressional districts after Step 3, where the initial district 1 has been split into new districts 1 and 14, the initial district 2 has been split into sub-districts 2 and 13, and so on.

Finally, in Figure 4.18, we present the potential distribution of districts based on the results of the 2020 presidential election. In this visualization, a red district indicates that the majority of voters in that area supported the Republican candidate, while a blue district represents Democratic support.

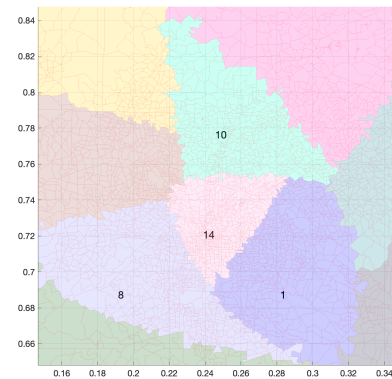
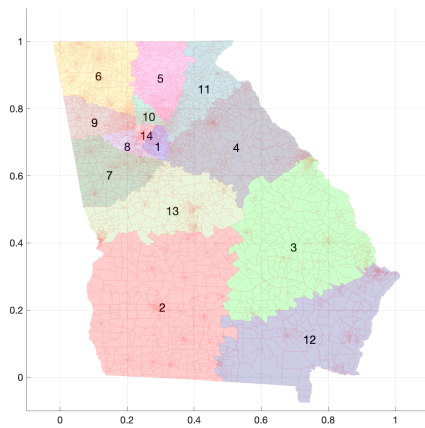


Figure 4.16: Districts after Step 3 for Example (C1)

Table 4.18: Population after Step 3

	Example (C1)		Example (C2)	
District	Population	Deviation	Population	Deviation
1	765166	29.714 ($3.88 * 10^{-3} \%$)	765094	-42.286 ($-5.52 * 10^{-3} \%$)
2	765128	-8.2857 ($-1.08 * 10^{-3} \%$)	765197	60.714 ($7.93 * 10^{-3} \%$)
3	765083	-53.286 ($-6.96 * 10^{-3} \%$)	765173	36.714 ($4.79 * 10^{-3} \%$)
4	765151	14.714 ($1.92 * 10^{-3} \%$)	765156	19.714 ($2.57 * 10^{-3} \%$)
5	765167	30.714 ($4.01 * 10^{-3} \%$)	765126	-10.286 ($-1.34 * 10^{-3} \%$)
6	765092	-44.286 ($-5.78 * 10^{-3} \%$)	765158	21.714 ($2.83 * 10^{-3} \%$)
7	765170	33.714 ($4.40 * 10^{-3} \%$)	765092	-44.286 ($-5.78 * 10^{-3} \%$)
8	765136	-0.28571 ($-3.73 * 10^{-5} \%$)	765122	-14.286 ($-1.86 * 10^{-3} \%$)
9	765126	-10.286 ($-1.34 * 10^{-3} \%$)	765157	20.714 ($2.70 * 10^{-3} \%$)
10	765108	-28.286 ($-3.69 * 10^{-3} \%$)	765128	-8.2857 ($-1.08 * 10^{-3} \%$)
11	765129	-7.2857 ($-9.52 * 10^{-4} \%$)	765116	-20.286 ($-2.65 * 10^{-3} \%$)
12	765155	18.714 ($2.44 * 10^{-3} \%$)	765189	52.714 ($6.88 * 10^{-3} \%$)
13	765137	0.71429 ($9.33 * 10^{-5} \%$)	765080	-56.286 ($-7.35 * 10^{-3} \%$)
14	765160	23.714 ($3.09 * 10^{-3} \%$)	765120	-16.286 ($-2.12 * 10^{-3} \%$)

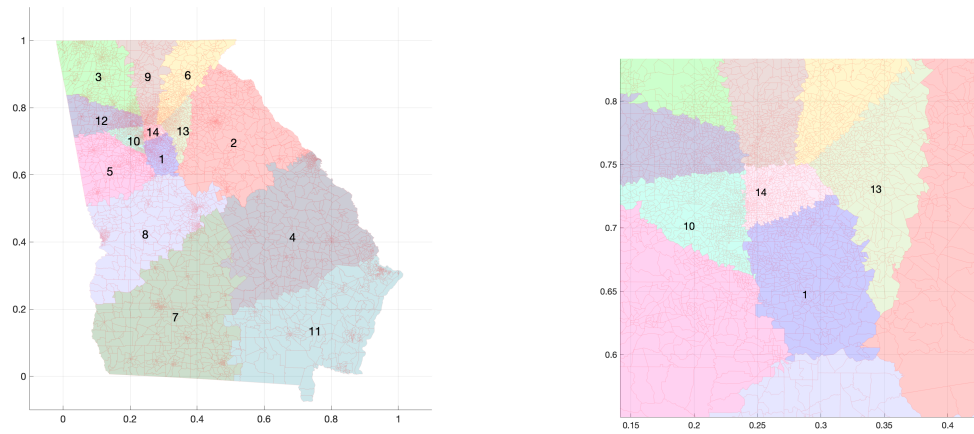
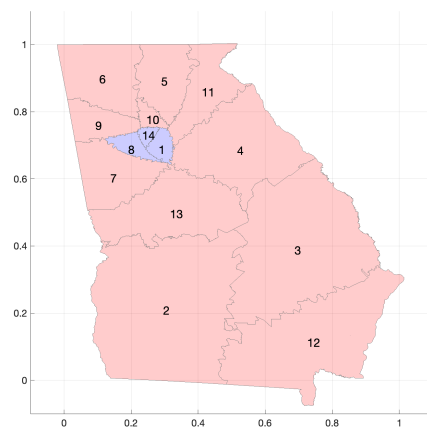
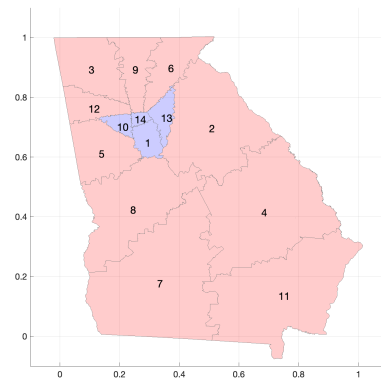


Figure 4.17: Districts after Step 3 for Example (C2)



(a) Example (C1)



(b) Example (C2)

Figure 4.18: Districts based on the 2020 presidential election results

CHAPTER 5

CONCLUSION

In this manuscript, we have discussed the current numerical algorithms for solving optimal transport (OT) problems in various contexts. Firstly, we examined discrete OT problems and presented the relevant techniques for solving linear programming and assignment problems. Furthermore, we offered a solution to the linear assignment problem involving a circular obstacle in a domain characterized by a permeability coefficient of k .

Next, we provided a numerical study of three different methods to solve the continuous OT problem with quadratic cost, posed in 1-d or 2-d. We gave implementation details and numerical comparison of three approaches based on the Monge-Ampère PDE and the fluid-dynamics (Benamou-Brenier) formulation of the problem. Further, a novel theoretical contribution of our work has been the introduction of separable structure, which is shared by many OT problems. Exploiting this viewpoint proved very valuable in obtaining much more efficient and accurate techniques. Our results show that if one is interested in the optimal map (the Monge map), then direct solution of the Monge-Ampère equation is the most efficient way to proceed, especially when we have, and exploit, separable structure. For those situation where the time evolution of the density is needed (as it is the case when we want to find the geodesic trajectory), then the two formulations of [59] and [29] offer some advantages with respect to each other: (i) in general, the formulation of [29] is less costly than that of [59], and outperforms it for periodic problems, but (ii) on a truncated domain the formulation of [59] performs better, though it remains somewhat expensive. In the end, although one can find justification for failure of any of the techniques we considered, in our opinion this defeats the purpose of a well designed numerical method for solving the OT problem: one should assume that it delivers the desired result and not wander why it does not.

In the last chapter, we presented a novel implementation of Newton’s method for solving semi-discrete optimal transport problems for cost functions given by a (positive combination of) p -norm(s), $1 < p < \infty$, and several choices of continuous and discrete densities. To date, there appeared to have been no implementation of Newton’s method for these problems, and we succeeded in making some progress by proving that the Laguerre cells providing the solution of the OT problem are star shaped with respect to the target points and by exploiting this fact in our algorithmic development. We gave a detailed description of all algorithms we implemented and provided quantitative testing on several examples, as well as comparison with competing approaches, well beyond simply showing some qualitatively faithful figures. Our algorithms proved much more robust and accurate than other methods, being based on rigorously justified adaptivity and error control, which—in principle—allows to solve the OT problem at any desired level of accuracy. The price we pay for this is the need for the continuous density to be sufficiently smooth. Lastly, we illustrated application of our proposed algorithm through the delineation of boundaries for the congressional districts in the state of Georgia. And we explored how the Monte Carlo algorithm can effectively tackle semi-discrete optimal transport problems in higher dimensions.

Appendices

APPENDIX A

ALGORITHMS PSEUDOCODES

Algorithm 1 *Shooting* – Shooting Technique

Input: $\theta_0, \{Y, W\}, \{y_j, w_j\}, \forall j, \text{TOL}$ $\triangleright$ Default value $\text{TOL} = 10^{-14}$

Output: r value up to a tolerance TOL and index k

$r_0 \leftarrow 0, d \leftarrow \frac{\min_j \|Y - y_j\|_p}{10}, \forall j \neq i$

while $d \geq \text{TOL}$ **do**

$r_0 \leftarrow r_0 + d$

$x \leftarrow Y + r_0 \begin{pmatrix} \cos(\theta_0) \\ \sin(\theta_0) \end{pmatrix}$

if $x \in A(y_j)$ **then**

$r^* \leftarrow \text{BisectR}(\theta_0, \{r_0 - d, r_0\}, \{Y, W\}, \{y_j, w_j\}, \text{TOL})$ and $k \leftarrow j$

$x \leftarrow Y + (r^* - \text{TOL}) \begin{pmatrix} \cos(\theta_0) \\ \sin(\theta_0) \end{pmatrix}$

if $x \in A(Y)$ **then**

return r^* and $k \leftarrow j$

else

$d \leftarrow \frac{d}{2}$

end if

else if $x \notin \Omega$ **then**

$\{r^*, k\} \leftarrow \text{Bound}(\theta_0, Y)$

$x \leftarrow Y + (r^* - \text{TOL}) \begin{pmatrix} \cos(\theta_0) \\ \sin(\theta_0) \end{pmatrix}$

if $x \in A(Y)$ **then**

return r^* and k

else

$d \leftarrow \frac{d}{2}$

end if

end if

end while

Algorithm 2 *BisectR* – Bisection Method on r Value

Input: $\theta_0, \{r_a, r_b\}, \{Y, W\}, \{y_j, w_j\}, \text{TOL}$ $\triangleright$ TOL is passed by *Shooting***Output:** r value up to a tolerance TOL

```
while  $r_b - r_a \geq \text{TOL}$  do
   $r \leftarrow \frac{1}{2}(r_b + r_a)$ 
  if  $F(r, \theta_0) < 0$  then
     $r_a \leftarrow r$ 
  else if  $F_b(r) > 0$  then
     $r_b \leftarrow r$ 
  else
    return  $r$ 
  end if
end while
```

Algorithm 3 *Bound* – Boundary Point Computation

Input: θ, Y **Output:** r value and index k

```
 $r_1 \leftarrow \frac{a-Y_1}{\cos(\theta)}, y_1 = Y_2 + r_1 \sin(\theta)$ 
 $r_2 \leftarrow \frac{b-Y_1}{\cos(\theta)}, y_2 = Y_2 + r_2 \sin(\theta)$ 
 $r_3 \leftarrow \frac{a-Y_2}{\sin(\theta)}, x_3 = Y_1 + r_3 \cos(\theta)$ 
 $r_4 \leftarrow \frac{b-Y_2}{\sin(\theta)}, x_4 = Y_1 + r_4 \cos(\theta)$ 
if  $r_1 > 0 \ \& \ y_1 \geq a \ \& \ y_1 \leq b$  then
   $r \leftarrow r_1$  and  $k \leftarrow -4$ 
else if  $r_2 > 0 \ \& \ y_2 \geq a \ \& \ y_2 \leq b$  then
   $r \leftarrow r_2$  and  $k \leftarrow -2$ 
else if  $r_3 > 0 \ \& \ x_3 \geq a \ \& \ x_3 \leq b$  then
   $r \leftarrow r_3$  and  $k \leftarrow -1$ 
else if  $r_4 > 0 \ \& \ x_4 \geq a \ \& \ x_4 \leq b$  then
   $r \leftarrow r_4$  and  $k \leftarrow -3$ 
end if
```

Algorithm 4 *PredCorr* – Predictor-Corrector Step

Input: $r_0, \theta_0, \Delta, \{Y, W\}, \{y_j, w_j\}, \text{TOL}$ $\triangleright$ Default value $\text{TOL} = 10^{-14}$ **Output:** $r_1 = r(\theta_0 + \Delta)$ value up to a tolerance TOL and increment Δ_{new}

```
 $r_1^0 \leftarrow r_0 - \Delta \frac{F_\theta(r_0, \theta_0)}{F_r(r_0, \theta_0)}, r_1 \leftarrow r_1^0$ 
while  $F(r_1, \theta + \Delta) \geq \text{TOL}$  do
   $r_1 \leftarrow r_1 - \frac{F(r_1, \theta + \Delta)}{F_r(r_1, \theta + \Delta)}$ 
end while
 $\Delta_{new} \leftarrow \frac{10^{-3}}{2\sqrt{r_1 - r_1^0}} \Delta$ 
```

 $\triangleright$ Tangent Predictor Step $\triangleright$ Newton Corrector

Algorithm 5 *BisectTheta* – Bisection Method on θ Value

Input: $r_0, \theta_0, \theta_1, \{Y, W\}, \{y_j, w_j\}, \text{TOL}$ $\triangleright$ Default value $\text{TOL} = 10^{-14}$ **Output:** r and θ values up to a tolerance TOL

```
while  $\theta_1 - \theta_0 \geq \text{TOL}$  do
   $\theta \leftarrow \frac{1}{2}(\theta_0 + \theta_1)$ 
   $r \leftarrow \text{PredCorr}(r_0, \theta_0, \theta - \theta_0, \{Y, W\}, \{y_j, w_j\}, \text{TOL})$ 
   $x \leftarrow Y + r \left( \frac{\cos(\theta)}{\sin(\theta)} \right)$ 
  if  $x \in A(Y)$  then
     $\theta_0 \leftarrow \theta$  and  $r_0 \leftarrow r$ 
  else
     $\theta_1 \leftarrow \theta$ 
  end if
end while
```

Algorithm 6 *CurveBound* – Breakpoint Between Curve and Domain Boundary

Input: $k, \theta_0, \theta_1, \{Y, W\}, \{y_j, w_j\}, \text{TOL}$ $\triangleright k$ is the index of domain boundary**Output:** r^* and θ^* values up to a tolerance TOL $\triangleright$ Default value $\text{TOL} = 10^{-14}$

```
if  $k = -4$  then
   $r(\theta) \leftarrow \frac{a-Y_1}{\cos(\theta)}$ 
else if  $k = -2$  then
   $r(\theta) \leftarrow \frac{b-Y_1}{\cos(\theta)}$ 
else if  $k = -1$  then
   $r(\theta) \leftarrow \frac{a-Y_2}{\sin(\theta)}$ 
else if  $k = -3$  then
   $r(\theta) \leftarrow \frac{b-Y_2}{\sin(\theta)}$ 
end if
 $\theta^* \leftarrow \text{mod}(\text{fzero}(F(r(\theta), \theta), [\theta_0, \theta_1], \text{TOL}), 2\pi)$ 
 $r^* \leftarrow r(\theta^*)$ 
```

Algorithm 7 *BoundBound* – Breakpoint Between Two Domain Boundary Branches

Input: k_1, k_2 **Output:** r^* and θ^* values

```
if  $(k_1 = -4 \ \& \ k_2 = -1) \text{ --- } (k_1 = -1 \ \& \ k_2 = -4)$  then
   $x \leftarrow \begin{pmatrix} a \\ a \end{pmatrix}$ 
else if  $(k_1 = -1 \ \& \ k_2 = -2) \text{ --- } (k_1 = -2 \ \& \ k_2 = -1)$  then
   $x \leftarrow \begin{pmatrix} b \\ a \end{pmatrix}$ 
else if  $(k_1 = -2 \ \& \ k_2 = -3) \text{ --- } (k_1 = -3 \ \& \ k_2 = -2)$  then
   $x \leftarrow \begin{pmatrix} b \\ b \end{pmatrix}$ 
else if  $(k_1 = -3 \ \& \ k_2 = -4) \text{ --- } (k_1 = -4 \ \& \ k_2 = -3)$  then
   $x \leftarrow \begin{pmatrix} a \\ b \end{pmatrix}$ 
end if
 $\theta^* \leftarrow \text{mod}(\arctan(\frac{x_2-Y_2}{x_1-Y_1}), 2\pi)$ 
 $r^* \leftarrow \frac{x_1-Y_1}{\cos(\theta^*)}$ 
```

Algorithm 8 *AdaptSimpson* – Adaptive Integral Computation

Input: $f(x)$, $[a, b]$, TOL ▷ Default value TOL = 10^{-12}
Output: $I = \int_a^b f(x)dx$ up to a tolerance TOL
 $check \leftarrow 0$; $N_{max} \leftarrow 1024$; $i_{max} \leftarrow \lfloor \log_2(\text{TOL}/\text{eps}) \rfloor$ ▷ eps is machine precision
 $I \leftarrow 0$; $Err \leftarrow 0$
for $i \leftarrow 0$ to i_{max} **do**
 $x_i \leftarrow [a : (b - a)/(2^i) : b]$
 $\text{TOL}_i \leftarrow \text{TOL}/(2^i)$
 for $j \leftarrow 1$ to 2^i **do**
 if $check(j) = 0$ **then**
 $[I_j, err_j] \leftarrow \text{CompSimpson}(f(x), x_i(j), x_i(j + 1), \text{TOL}_i, N_{max})$
 if $Err_j < \text{TOL}_i$ **then**
 $I \leftarrow I + I_j$; $Err \leftarrow Err + err_j$; $check(j) \leftarrow 1$
 end if
 end if
 end for
 if $\min(check) = 1$ **then**
 return I
 else
 $check \leftarrow \text{repelem}(check, 2)$ ▷ Repeat each of $check$ vector's elements two times
 end if
 if $\text{mod}(i, 3) = 2$ **then**
 $N_{max} \leftarrow 8 \times N_{max}$
 end if
end for

Algorithm 9 *CompSimpson* – Integral Computation using Composite Simpson's 1/3 Rule

Input: $f(x)$, $[a, b]$, TOL, N_{max} ▷ TOL is passed by *AdaptSimpson*
Output: $I = \int_a^b f(x)dx$ up to a tolerance TOL and relative error value err
 $i_{max} \leftarrow \lfloor \log_2(N_{max}) \rfloor$; $n \leftarrow 2$
 $I_n \leftarrow \text{Simpson}(f(x), [a, b], n)$ ▷ Computes integral using Simpson's 1/3 Rule with n sub-intervals
for $i \leftarrow 1$ to i_{max} **do**
 $I_{2n} \leftarrow \text{Simpson}(f(x), [a, b], 2n)$
 $err \leftarrow 16|I_{2n} - I_n|/15$; $N \leftarrow n(err/\text{TOL})^{1/4}$; $N \leftarrow 2\lfloor N/2 \rfloor$
 if $err < \text{TOL} \vee N > N_{max}$ **then**
 return I_{2n} and err
 else
 $n \leftarrow 2n$; $I_n \leftarrow I_{2n}$
 end if
end for

Algorithm 10 *Simpson* – Integral Computation using Simpson’s 1/3 Rule

Input: $f(x)$, $[a, b]$, N **Output:** $I = \int_a^b f(x)dx$ approximation using Simpson’s 1/3 Rule with N sub-intervals $h \leftarrow \frac{b-a}{N}$, $I \leftarrow \frac{1}{3}hf(a)$ **for** $i \leftarrow 2$ to N **do** **if** $\text{mod}(i, 2) = 0$ **then** $I \leftarrow I + \frac{4}{3}hf(a + (i-1)h)$ **else if** $\text{mod}(i, 2) = 1$ **then** $I \leftarrow I + \frac{2}{3}hf(a + (i-1)h)$ **end if****end for** $I \leftarrow I + \frac{1}{3}hf(b)$

Algorithm 11 *GridInitialGuess* – Initial Guess Computation on a Grid

Input: h , (a, b) , $\{y_i, \nu_i\}, \forall i$, MAXIT $\triangleright$ Default is MAXIT = 50**Output:** w^0 value $w^0 \leftarrow 0$, $N_h \leftarrow \frac{b-a}{h}$, $\triangleright N_h$ is the number of squares on the grid $X_1 \leftarrow [a + \frac{h}{2} : h : b - \frac{h}{2}]$, $X_2 \leftarrow [a + \frac{h}{2} : h : b - \frac{h}{2}]$, $\triangleright$ Coordinates of centers of squares**for** $iter \leftarrow 1$ to MAXIT **do** $M \leftarrow \text{GridMeasure}(h, N_h, \{X_1, X_2\}, \{y_i, w_i^0\}, \forall i)$ **for** $k \leftarrow 1$ to N **do** $\triangleright N$ is the number of target points $M^0 \leftarrow \text{GridMeasure}(h, N_h, \{X_1, X_2\}, \{y_i, w_i^0\}, \forall i)$ $incr \leftarrow \sqrt{h}$ **while** $|M^0(k) - \nu_k| > h^2 \wedge incr > h^4$ **do** $w^0(i) \leftarrow w^0(i) + \text{sign}(M^0(k) - \nu_k) \frac{incr}{N-1}, \forall i \neq k$ $M^1 \leftarrow \text{GridMeasure}(h, N_h, \{X_1, X_2\}, \{y_i, w_i^0\}, \forall i)$ **if** $|M^1(k) - \nu_k| > |M^0(k) - \nu_k|$ **then** $incr \leftarrow \sqrt{h} \times incr$ **end if** $M^0 \leftarrow M^1$ **end while** **end for** **if** $\max_i |M^0(i) - \nu_i| < h^2 \vee \max_i |M^0(i) - \nu_i| \geq \max_i |M(i) - \nu_i|$ **then** **return** w^0 **end if****end for**

Algorithm 12 *GridMeasure* – Measure Approximation on the Grid

Input: $h, N_h, \{X_1, X_2\}, \{y_i, w_i\}, \forall i$

Output: M values, which approximate $\mu(A(y_i))$ values on the grid

```

 $M \leftarrow 0$ 
for  $l \leftarrow 1$  to  $N_h$  do
  for  $m \leftarrow 1$  to  $N_h$  do
     $x \leftarrow \begin{pmatrix} X_1^{(l)} \\ X_2^{(m)} \end{pmatrix}$ 
     $ind \leftarrow \arg \min_i c(x, y_i) - w_i$ 
     $M(ind) \leftarrow M(ind) + \rho(x)h^2$ 
  end for
end for
  
```

Algorithm 13 *DampedNewton* – Damped Newton's Method

Input: $w^0, \mu, \{y_i, \nu_i\}, \forall i, \text{MAXIT}, \text{TOL}$

▷ Default MAXIT = 20, TOL = 10^{-8}

Output: w^k value such that $\max_i |\mu(A(y_i, w^k)) - \nu_i| \leq \text{TOL}$

```

 $err^0 \leftarrow \mu(A(y_i, w^0)) - \nu_i, \forall i$ 
for  $k \leftarrow 1$  to MAXIT do
   $H^0 \leftarrow \frac{\partial^2 \Phi}{\partial w_i^0 \partial w_j^0}, \forall i, j$ 
   $s \leftarrow -(H^0)^I err^0, w^1 \leftarrow w^0 + s$ 
  while  $\min_{i,j} \left\| 1 - |w_i^1 - w_j^1|/c(y_i, y_j) \right\| \leq 0$  do
     $s \leftarrow \frac{1}{2}s, w^1 \leftarrow w^0 + s$ 
  end while
   $err^1 \leftarrow \mu(A(y_i, w^1)) - \nu_i, \forall i$ 
  while  $\|err^1\|_2^2 > \|err^0\|_2^2 \wedge \|s\|_\infty \geq \frac{\text{TOL}}{2^{\text{MAXIT}}}$  do
     $s \leftarrow \frac{1}{2}s, w^1 \leftarrow w^0 + s$ 
     $err^1 \leftarrow \mu(A(y_i, w^1)) - \nu_i, \forall i$ 
  end while
  if  $\|err^1\|_\infty \leq \text{TOL} \vee \|s\|_\infty \leq \frac{\text{TOL}}{2^{\text{MAXIT}}}$  then
    return  $w^1$ 
  else
     $w^0 \leftarrow w^1, err^0 \leftarrow err^1$ 
  end if
end for
  
```

▷ See (4.22)

▷ Damped Newton

Algorithm 14 *GradientDescent* – Gradient Descent Method

Input: $w^0, \mu, \{y_i, \nu_i\}, \forall i, \text{MAXIT}, \text{TOL}$ $\triangleright$ Default MAXIT = 100, TOL = 10^{-4}
Output: w^k value such that $\max_i |\mu(A(y_i, w^k)) - \nu_i| \leq \text{TOL}$
 $\phi^0 \leftarrow \Phi(w^0), \text{err}^0 \leftarrow \mu(A(y_i, w^0)) - \nu_i, \forall i$
for $k \leftarrow 1$ to MAXIT **do**
 $s \leftarrow -\text{err}^0, w^1 \leftarrow w^0 + s, \phi^1 \leftarrow \Phi(w^1)$
 while $\phi_1 \geq \phi_0$ **do**
 $s \leftarrow \frac{1}{2}s, w^1 \leftarrow w^0 + s, \phi^1 \leftarrow \Phi(w^1)$
 end while
 $\text{err}^1 \leftarrow \mu(A(y_i, w^1)) - \nu_i, \forall i$
 if $\|\text{err}^1\|_\infty \leq \text{TOL}$ **then**
 return w^1
 else
 $w^0 \leftarrow w^1, \text{err}^0 \leftarrow \text{err}^1, \phi^0 \leftarrow \phi^1$
 end if
end for

Algorithm 15 *DistrictsStep1* – Step 1 of Boundary Drawing

Input: $\{B_j, x_j, P_j\}, j = 1, 2, \dots, 7433; \{y_i, \nu_i\}, i = 1, 2, \dots, 7; \text{TOL}, e^{(0)}$ $\triangleright$ TOL = 10^{-4}
Output: $A_i, i = 1, 2, \dots, 7$
1. Solve the semi-discrete problem: $\rho(x) = \frac{1}{|\Omega|} \{w_i, A_i\} \{y_i, \nu_i\}, i = 1, 2, \dots, 7$
2. Compute the population in A_i : $p_i = \sum_{x_j \in A_i} P_j, i = 1, 2, \dots, 7$
3. Compute validation error: $e_i^{(1)} = \frac{p_i - \bar{p}}{\bar{p}}, i = 1, 2, \dots, 7$ $\triangleright \bar{p} = \frac{1}{7} \sum_j P_j$
4. If $\|e^{(1)}\|_\infty \leq \text{TOL}$, then return $A_i, i = 1, 2, \dots, 7$
5. If $\|e^{(1)}\|_2 > \|e^{(0)}\|_2$, then center target points y_i and go to Step 1
6. Otherwise, update $\nu_i \leftarrow \nu_i - \frac{1}{7k} e_i^{(1)}, e^{(0)} \leftarrow e^{(1)}$, and go to Step 1 $\triangleright k$ is a positive integer

Algorithm 16 *DistrictsStep2* – Step 2 of Boundary Drawing

Input: $\{B_j, x_j, P_j\}, j = 1, 2, \dots, 7433; \{A_i, e_i\}, i = 1, 2, \dots, 7, \text{MAXIT}, n = 0$ $\triangleright$ MAXIT = 100
Output: $A_i, i = 1, 2, \dots, 7$
while $\|p_i - \bar{p}\|_\infty > 100 \wedge n \geq \text{MAXIT}$ **do** $\triangleright \bar{p} = \frac{1}{7} \sum_j P_j$
 $n \leftarrow n + 1$
 for $i = 1 : 7$ **do**
 if $e_i < 0$ **then**
 $\text{error} \leftarrow \bar{p} e_i$
 1. Create a list of block groups B_j , such that $B_j \cap A_i \neq \emptyset$ and $x_j \notin A_i$.
 2. Sort the list w.r.t. P_j , placing the closest value to error at the top.
 3. If $x_1 \in A_k$ such that $e_k > 0$, then move block group B_1 to A_i cell.
 Otherwise, move to the next element in the list and repeat this step.
 4. Update population p_i and validation error $e_i, i = 1, 2, \dots, 7$.
 end if
 end for
end while

Algorithm 17 *DistrictsStep3* – Step 3 of Boundary Drawing

Input: $\{B_j, x_j, P_j\}, j = 1, 2, \dots, 7433; D_i, i = 1, 2, \dots, 7$

Output: $D_i^j, i = 1, 2, \dots, 7, j = 1, 2$

for $i = 1 : 7$ **do**

1. Pick two points y_a and y_b , such that $(a, b) = \arg \max_{k,l: B_k, B_l \in D_i} \|x_k - x_l\|_2$.
2. Split district D_i into two districts using MATLAB function `intlinprog`, such that
 - Sub-districts D_i^1 and D_i^2 have equal population.
 - Each block group is assigned to the unique sub-district.

end for

REFERENCES

- [1] E. Al Aawar, S. El Mohtar, I. Lakkis, A. K. Alduwais, and I. Hoteit, “Bayesian source identification of urban-scale air pollution from point and field concentration measurements,” *Computational Geosciences*, pp. 1–22, 2023.
- [2] L. Ambrosio, “Lecture notes on optimal transport problems,” in *Mathematical Aspects of Evolving Interfaces: Lectures given at the C.I.M.-C.I.M.E. joint Euro-Summer School held in Madeira, Funchal, Portugal, July 3-9, 2000*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 1–52.
- [3] F. Aurenhammer, “Power diagrams: Properties, algorithms and applications,” *SIAM Journal on Computing*, vol. 16, no. 1, pp. 78–96, 1987.
- [4] A. Barvinok, *A course in convexity*. American Mathematical Soc., 2002, vol. 54.
- [5] T. Bazeille, H. Richard, H. Janati, and B. Thirion, “Local optimal transport for functional brain template estimation,” in *Information Processing in Medical Imaging: 26th International Conference, IPMI 2019, Hong Kong, China, June 2–7, 2019, Proceedings 26*, Springer, 2019, pp. 237–248.
- [6] J.-D. Benamou, C. Cotter, and H. Malamut, “Entropic optimal transport solutions of the semigeostrophic equations,” *Journal of Computational Physics*, p. 112 745, 2024.
- [7] J.-D. Benamou and Y. Brenier, “A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem,” *Numerische Mathematik*, vol. 84, no. 3, pp. 375–393, 2000.
- [8] J.-D. Benamou and Y. Brenier, “A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem,” *Numerische Mathematik*, vol. 84, no. 3, pp. 375–393, 2000.
- [9] J.-D. Benamou, B. D. Froese, and A. M. Oberman, “Numerical solution of the optimal transportation problem using the Monge–Ampère equation,” *Journal of Computational Physics*, vol. 260, pp. 107–126, 2014.
- [10] D. Bertsekas, *Network optimization: continuous and discrete models*. Athena Scientific, 1998, vol. 8.
- [11] D. P. Bertsekas, “A new algorithm for the assignment problem,” *Mathematical Programming*, vol. 21, no. 1, pp. 152–171, 1981.

- [12] D. P. Bertsekas and J. Eckstein, “Dual coordinate step methods for linear network flow problems,” *Mathematical Programming*, vol. 42, no. 1-3, pp. 203–243, 1988.
- [13] B. Bollobás, “Linear Analysis, Cambridge Mathematical Textbooks,” in Cambridge University Press, 1999, p. 21.
- [14] N. Bonneel and J. Digne, “A survey of optimal transport for computer graphics and computer vision,” in *Computer Graphics Forum*, Wiley Online Library, vol. 42, 2023, pp. 439–460.
- [15] G. Bouchitté and G. Buttazzo, “Characterization of optimal shapes and masses through Monge-Kantorovich equation,” *Journal of the European Mathematical Society*, vol. 3, no. 2, pp. 139–168, 2001.
- [16] G. Bouchitté, G. Buttazzo, and P. Seppecher, “Shape optimization solutions via Monge-Kantorovich equation,” *Comptes Rendus de l’Académie des Sciences-Series I-Mathematics*, vol. 324, no. 10, pp. 1185–1191, 1997.
- [17] A. Bounceur, M. Bezoui, and R. Euler, “Boundaries and hulls of Euclidean graphs: From theory to practice,” in CRC Press, 2018, p. 34.
- [18] D. P. Bourne and S. M. Roper, “Centroidal power diagrams, Lloyd’s algorithm, and applications to optimal location problems,” *SIAM Journal on Numerical Analysis*, vol. 53, no. 6, pp. 2545–2569, 2015.
- [19] D. P. Bourne, B. Schmitzer, and B. Wirth, “Semi-discrete unbalanced optimal transport and quantization,” *arXiv: Optimization and Control*, 2018.
- [20] Y. Brenier, “Décomposition polaire et réarrangement monotone des champs de vecteurs,” *CR Acad. Sci. Paris Sér. I Math.*, vol. 305, pp. 805–808, 1987.
- [21] Y. Brenier, “A homogenized model for vortex sheets,” *Archive for Rational Mechanics and Analysis*, vol. 138, no. 4, pp. 319–353, 1997.
- [22] L. A. Caffarelli, S. A. Kochengin, and V. I. Oliker, “On the numerical solution of the problem of reflector design with given far-field scattering data,” in *Monge–Ampère equation: applications to geometry and optimization (NSF-CBMS Conference on the Monge–Ampère Equation, Applications to Geometry and Optimization, July 9–13, 1997, Florida Atlantic University)*, ser. Contemporary Mathematics, vol. 226, Providence, R.I.: American Mathematical Society, 1999, pp. 13–32.
- [23] R. E. Caflisch, “Monte Carlo and quasi-Monte Carlo methods,” *Acta numerica*, vol. 7, pp. 1–49, 1998.

- [24] R. Chartrand, B. Wohlberg, K. Vixie, and E. Bollt, “A gradient descent solution to the Monge-Kantorovich problem,” *Applied Mathematical Sciences*, vol. 3, pp. 1071–1080, 21–24 2009.
- [25] L. Chizat, G. Peyré, B. Schmitzer, and F.-X. Vialard, “Scaling algorithms for unbalanced optimal transport problems,” *Mathematics of Computation*, vol. 87, no. 314, pp. 2563–2609, 2018.
- [26] V. Chvátal, *Linear programming*. Macmillan, 1983.
- [27] N. Courty, R. Flamary, and D. Tuia, “Domain adaptation with regularized optimal transport,” in *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2014, Nancy, France, September 15-19, 2014. Proceedings, Part I 14*, Springer, 2014, pp. 274–289.
- [28] J. A. Cuesta-Albertos and A. Tuero-Díaz, “A characterization for the solution of the Monge-Kantorovich mass transference problem,” *Statistics and Probability Letters*, vol. 16, no. 2, pp. 147–152, 1993.
- [29] J. Cui, L. Dieci, and H. Zhou, “A continuation multiple shooting method for Wasserstein geodesic equation,” *SIAM Journal on Scientific Computing*, vol. 44, no. 5, A2918–A2943, 2022.
- [30] M. Cuturi, “Sinkhorn distances: Lightspeed computation of optimal transport,” in *Advances in Neural Information Processing Systems*, vol. 26, Curran Associates, Inc., 2013, pp. 2292–2300.
- [31] F. De Gournay, J. Kahn, and L. Lebrat, “Differentiation and regularity of semi-discrete optimal transport with respect to the parameters of the discrete measure,” *Numerische Mathematik*, vol. 141, pp. 429–453, 2019.
- [32] L. Dieci and J. Walsh III, “The boundary method for semi-discrete optimal transport partitions and Wasserstein distance computation,” *Journal of Computational and Applied Mathematics*, vol. 353, pp. 318–344, 2019.
- [33] A. Dobrin, “A review of properties and variations of Voronoi diagrams,” *Whitman College*, vol. 10, no. 1.453, p. 9156, 2005.
- [34] L. L. Doskolovich, D. A. Bykov, A. A. Mingazov, and E. A. Bezus, “Optimal mass transportation and linear assignment problems in the design of freeform refractive optical elements generating far-field irradiance distributions,” *Optics express*, vol. 27, no. 9, pp. 13 083–13 097, 2019.
- [35] Q. Du, V. Faber, and M. Gunzburger, “Centroidal Voronoi tessellations: Applications and algorithms,” *SIAM Review*, vol. 41, no. 4, pp. 637–676, 1999.

- [36] H. Federer, *Geometric measure theory*. Springer, 2014.
- [37] S. Ferradans, N. Papadakis, J. Rabin, G. Peyré, and J.-F. Aujol, “Regularized discrete optimal transport,” in *Scale Space and Variational Methods in Computer Vision: 4th International Conference, SSVN 2013, Schloss Seggau, Leibnitz, Austria, June 2-6, 2013. Proceedings 4*, Springer, 2013, pp. 428–439.
- [38] S. Ferradans, G.-S. Xia, G. Peyré, and J.-F. Aujol, “Static and dynamic texture mixing using optimal transport,” in *Scale Space and Variational Methods in Computer Vision: 4th International Conference, SSVN 2013, Schloss Seggau, Leibnitz, Austria, June 2-6, 2013. Proceedings 4*, Springer, 2013, pp. 137–148.
- [39] S. Fortune, “A sweepline algorithm for Voronoi diagrams,” in *Proceedings of the Second Annual Symposium on Computational Geometry*, ser. SCG ’86, New York: ACM, 1986, pp. 313–322.
- [40] B. D. Froese, “A numerical method for the elliptic Monge–Ampère equation with transport boundary conditions,” *SIAM Journal on Scientific Computing*, vol. 34, no. 3, A1432–A1459, 2012.
- [41] B. D. Froese and A. M. Oberman, “Convergent finite difference solvers for viscosity solutions of the elliptic Monge–Ampère equation in dimensions two and higher,” *Journal of Computational Physics*, vol. 260, pp. 107–126, 2014.
- [42] A. Galichon, *Optimal transport methods in economics*. Princeton University Press, 2018.
- [43] J. Gao and R. Gupta, “Efficient proximity search for 3-d cuboids,” in *Computational Science and Its Applications — ICCSA 2003*, V. Kumar, M. L. Gavrilova, C. J. K. Tan, and P. L’Ecuyer, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, pp. 817–826.
- [44] D. Geiß, R. Klein, R. Penninger, and G. Rote, “Optimally solving a transportation problem using Voronoi diagrams,” *Computational Geometry*, vol. 46, no. 8, pp. 1009–1016, 2013.
- [45] C. Godsil and G. F. Royle, “Algebraic Graph Theory,” in Springer Science & Business Media, 2001, vol. 207, p. 279.
- [46] S. Haker and A. Tannenbaum, “Optimal mass transport and image registration,” in *Proceedings IEEE Workshop on Variational and Level Set Methods in Computer Vision*, IEEE, 2001, pp. 29–36.

- [47] V. N. Hartmann and D. Schuhmacher, “Semi-discrete optimal transport: A solution procedure for the unsquared Euclidean distance case,” *Mathematical Methods of Operations Research*, pp. 1–31, 2020.
- [48] R. Jordan, D. Kinderlehrer, and F. Otto, “The variational formulation of the Fokker–Planck equation,” *SIAM journal on mathematical analysis*, vol. 29, no. 1, pp. 1–17, 1998.
- [49] K. G. Kachar and A. A. Gorodetsky, “Dynamic multiagent assignment via discrete optimal transport,” *IEEE Transactions on Control of Network Systems*, vol. 9, no. 1, pp. 151–162, 2022.
- [50] L. V. Kantorovich, “On the translocation of masses,” *C.R. (Doklady) Acad. Sci. URSS (N.S.)*, vol. 37, pp. 199–201, 1942.
- [51] H. B. Keller, “Numerical methods in bifurcation problems,” in Springer-Verlag, Bombay, 1987, vol. Tata Institute of Fundamental Research.
- [52] J. Kitagawa, Q. Mérigot, and B. Thibert, “Convergence of a Newton algorithm for semi-discrete optimal transport,” *Journal of the European Mathematical Society*, vol. 21, no. 9, pp. 2603–2651, 2019.
- [53] S. Kolouri, S. R. Park, M. Thorpe, D. Slepcev, and G. K. Rohde, “Optimal mass transport: Signal processing and machine-learning applications,” *IEEE signal processing magazine*, vol. 34, no. 4, pp. 43–59, 2017.
- [54] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval research logistics quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [55] C. Laclau, I. Redko, B. Matei, Y. Bennani, and V. Brault, “Co-clustering through optimal transport,” in *International conference on machine learning*, PMLR, 2017, pp. 1955–1964.
- [56] C. Léonard, “A survey of the Schrödinger problem and some of its connections with optimal transport,” *Discrete and continuous dynamical systems*, vol. 34, no. 4, pp. 1533–1574, 2014.
- [57] B. Lévy, “A numerical algorithm for L_2 semi-discrete optimal transport in 3D,” *ESAIM: Mathematical Modelling and Numerical Analysis*, vol. 49, no. 6, pp. 1693–1715, 2015.
- [58] B. Lévy and E. L. Schwindt, “Notions of optimal transport theory and how to implement them on a computer,” *Computers & Graphics*, vol. 72, pp. 135–148, 2018.

- [59] W. Li, P. Yin, and S. Osher, “Computations of optimal transport distance with Fisher information regularization,” *Journal of Scientific Computing*, vol. 75, no. 3, pp. 1581–1595, 2018.
- [60] G. Loeper and F. Rapetti, “Numerical solution of the Monge–Ampère equation by a Newton’s algorithm,” *Comptes Rendus Mathématique*, vol. 340, no. 4, pp. 319–324, 2005.
- [61] L. Luo, N. Chakraborty, and K. Sycara, “Provably-good distributed algorithm for constrained multi-robot task assignment for grouped tasks,” *IEEE Transactions on Robotics*, vol. 31, no. 1, pp. 19–30, 2014.
- [62] X.-N. Ma, N. S. Trudinger, and X.-J. Wang, “Regularity of potential functions of the optimal transportation problem,” *Archive for rational mechanics and analysis*, vol. 177, pp. 151–183, 2005.
- [63] Q. Mérigot, “A comparison of two dual methods for discrete optimal transport,” in *GSI 2013 — Geometric Science of Information, Aug 2013, Paris, France*, ser. Lecture Notes in Computer Science, F. Nielsen and F. Barbaresco, Eds., vol. 8085, Springer, 1781, pp. 389–396.
- [64] L. Métivier, R. Brossier, Q. Mérigot, E. Oudet, and J. Virieux, “Measuring the misfit between seismograms using an optimal transport distance: Application to full waveform inversion,” *Geophysical Supplements to the Monthly Notices of the Royal Astronomical Society*, vol. 205, no. 1, pp. 345–377, 2016.
- [65] J. Meyron, “Initialization procedures for discrete and semi-discrete optimal transport,” *Computer-Aided Design*, vol. 115, pp. 13–22, 2019.
- [66] J.-M. Mirebeau, “Discretization of the 3D Monge–Ampère operator, between wide stencils and power diagrams,” arXiv:1503.00947, 2015.
- [67] G. Monge, “Mémoire sur la théorie des déblais et des remblais,” pp. 666–704, 1781, In French.
- [68] F. Morgan, *Geometric measure theory: a beginner’s guide*. Academic press, 2016.
- [69] J. Munkres, “Algorithms for the assignment and transportation problems,” *Journal of the society for industrial and applied mathematics*, vol. 5, no. 1, pp. 32–38, 1957.
- [70] V. Oliker and L. Prussner, “On the numerical solution of the equation $\frac{\partial^2 z}{\partial x^2} \frac{\partial^2 z}{\partial y^2} - \left(\frac{\partial^2 z}{\partial x \partial y} \right)^2 = f$ and its discretizations, I,” *Numerische Mathematik*, vol. 54, pp. 271–293, 1988.

- [71] F. Otto, “The geometry of dissipative evolution equations: The porous medium equation,” 2001.
- [72] N. Papadakis, “Optimal transport for image processing,” Ph.D. dissertation, Université de Bordeaux, 2015.
- [73] V. Peterson *et al.*, “Transfer learning based on optimal transport for motor imagery brain-computer interfaces,” *IEEE Transactions on Biomedical Engineering*, vol. 69, no. 2, pp. 807–817, 2021.
- [74] G. Peyré and M. Cuturi, “Computational optimal transport: With applications to data science,” *Foundations and Trends® in Machine Learning*, vol. 11, no. 5-6, pp. 355–607, 2019.
- [75] A. Pratelli, “On the equality between Monge’s infimum and Kantorovich’s minimum in optimal mass transportation,” *Annales de l’Institut Henri Poincaré (B) Probability and Statistics*, vol. 43, no. 1, pp. 1–13, 2007.
- [76] J. Rabin, S. Ferradans, and N. Papadakis, “Adaptive color transfer with relaxed optimal transport,” in *2014 IEEE international conference on image processing (ICIP)*, IEEE, 2014, pp. 4852–4856.
- [77] L. Rout, A. Korotin, and E. Burnaev, “Generative modeling with optimal transport maps,” *arXiv preprint arXiv:2110.02999*, 2021.
- [78] L. Rüschendorf, “Monge-Kantorovich transportation problem and optimal couplings,” *Jahresbericht der DMV*, vol. 3, pp. 113–137, 2007.
- [79] L. Rüschendorf and L. Uckelmann, “Numerical and analytical results for the transportation problem of Monge-Kantorovich,” *Metrika*, vol. 51, no. 3, pp. 245–258, 2000.
- [80] F. Santambrogio, “Optimal transport for applied mathematicians,” *Birkhäuser, NY*, vol. 55(58-63):94, 2015.
- [81] M. Sharir, “Intersection and closest-pair problems for a set of planar discs,” *SIAM Journal on Computing*, vol. 14, no. 2, pp. 448–468, 1985.
- [82] R. Sinkhorn, “Diagonal equivalence to matrices with prescribed row and column sums,” *The American Mathematical Monthly*, vol. 74, no. 4, pp. 402–405, 1967.
- [83] J. Solomon *et al.*, “Convolutional Wasserstein distances: Efficient optimal transportation on geometric domains,” *ACM Transactions on Graphics*, vol. 34, Jul. 2015.

- [84] M. Sulman, J. Williams, and R. D. Russell, “An efficient approach for the numerical solution of the Monge–Ampère equation,” *Applied Numerical Mathematics*, vol. 61, no. 3, pp. 298–307, 2011.
- [85] G. Tartavel, Y. Gousseau, and G. Peyré, “Constrained sparse texture synthesis,” in *Scale Space and Variational Methods in Computer Vision: 4th International Conference, SSVM 2013, Schloss Seggau, Leibnitz, Austria, June 2-6, 2013. Proceedings 4*, Springer, 2013, pp. 186–197.
- [86] L. C. Torres, L. M. Pereira, and M. H. Amini, “A survey on optimal transport for machine learning: Theory and applications,” *arXiv preprint arXiv:2106.01963*, 2021.
- [87] C. Villani, *Optimal transport: old and new*. Springer, 2009, vol. 338.
- [88] J. D. Walsh III and L. Dieci, “A real-valued auction algorithm for optimal transport,” *Statistical Analysis and Data Mining: The ASA Data Science Journal*, vol. 12, no. 6, pp. 514–533, 2019.